



รายงานการทดลองที่ 2 รายวิชา FRA233 Control Engineering for Robotics

สถาบันวิทยาการหุ่นยนต์ภาคสนาม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

Lab 2 Controller Design

จัดทำโดย

นายกัณฑ์ธีร์	เนตรวิศุทธิ์	รหัสนักศึกษา	67340500002
นางสาวเบญญา	บุรณะบุญยัติ	รหัสนักศึกษา	67340500026
นายณัฏฐวัฒน์	จันทรเทพ	รหัสนักศึกษา	67340500054
นายปกรณ์	บัวงาม	รหัสนักศึกษา	67340500055
นายปรีดี	พาหิระ	รหัสนักศึกษา	67340500057
นายสมานสิน	เจตนาเจริญชัย	รหัสนักศึกษา	67340500075

คำสัญลักษณ์และคำอธิบาย

K_t	=	ค่าคงที่แรงบิดมอเตอร์ (Motor Torque Constant)
K_e	=	ค่าคงที่แรงเคลื่อนไฟฟ้าต้านกลับ (Back EMF Constant)
R_m	=	ความต้านทานภายในขดลวดของมอเตอร์ (Armature Resistance)
L_m	=	ความเหนี่ยวนำภายในขดลวดของมอเตอร์ (Armature Inductance)
J_m	=	Moment of Inertia ของมอเตอร์
J_p	=	Moment of Inertia ของ Pendulum
$J_{รวม}$	=	Moment of Inertia รวมของทั้ง Pendulum และเฟลวมอเตอร์
B_m	=	ค่าสัมประสิทธิ์ความหนืดหรือแรงเสียดทาน (Viscous Damping Coefficient)
T_s	=	คาบเวลาในการสุ่มตัวอย่าง (Sampling Time)
$U[n]$	=	สัญญาณ Input ณ เวลาปัจจุบัน
$\omega[n]$	=	สัญญาณ Output ณ เวลาปัจจุบัน
K_p	=	ค่าคงที่ที่นำไปคูณกับ "ค่าความผิดพลาด" (Error) ที่เกิดขึ้น ณ ขณะนั้นโดยตรง
K_i	=	ค่าที่มีค่าความคลาดเคลื่อนที่สถานะคงตัวเพื่อให้ตัวแปรที่ถูกควบคุมเข้าสู่ค่าเป้าหมาย
K_d	=	ค่าตัวหน่วงเชิงกล ซึ่งช่วยลด การพุ่งเกินและลดระยะเวลาที่ระบบจะเข้าสู่สภาวะคงตัว
θ_s	=	ตำแหน่งเชิงมุมของระบบ (Angular Position) ในโดเมน s
V_s	=	แรงดันไฟฟ้า Input ที่เข้ามายังระบบ ในโดเมน s

1. Control Design for DC Motor in STM32G474RE

การทดลองที่ 1.1 การทดลองเพื่อศึกษาผลที่เกิดจากการเปลี่ยนความถี่การจ่ายแรงดันให้กับแบบจำลอง DC Motor ใน STM32G474RE

จุดประสงค์

1. เพื่อให้สามารถ Discretize สมการ Second Order Transfer Function ของ DC Motor ได้
2. เพื่อให้สามารถเขียนสมการ Discretization ของ DC Motor ลงไปบน STM32G474RE ได้
3. เพื่อให้สามารถเปลี่ยนความถี่ของการจ่ายแรงดันรูปแบบ Sine Wave และ Ramp ให้กับแบบจำลอง DC Motor ใน STM32G474RE
4. เพื่อให้สามารถสร้างสัญญาณตอบสนองของแบบจำลอง DC Motor ใน STM32G474RE รูปแบบ Sine Wave และ Ramp ได้
5. เพื่อให้สามารถวิเคราะห์สัญญาณตอบสนองรูปแบบ Sine Wave และ Ramp ของแบบจำลอง DC Motor ใน STM32G474RE ที่เกิดจากความถี่ต่าง ๆ ที่ใช้จ่ายแรงดันได้
6. เพื่อให้สามารถเปรียบเทียบความแตกต่างของความถี่ Input (แรงดัน) ที่ส่งผลต่อสัญญาณ Output (ความเร็วเชิงมุม) รูปแบบ Sine Wave และ Ramp ที่ได้จากแบบจำลอง DC Motor ได้

สมมติฐาน

ความถี่ของการจ่ายแรงดันให้กับแบบจำลอง DC Motor ที่ต่ำเกินไปจะทำให้ความเร็วเชิงมุมของแบบจำลองมอเตอร์มีความไม่ต่อเนื่องสูง ส่วนความถี่ที่สูงเกินไปจะทำให้แบบจำลองมอเตอร์ไม่สามารถตอบสนองต่อแรงดันที่จ่ายได้ทัน

ตัวแปร

1. ตัวแปรต้น:
 - ความถี่ของการจ่ายแรงดันให้กับแบบจำลอง DC Motor
 - รูปแบบของสัญญาณที่เป็น Input = Sine Wave และ Ramp
2. ตัวแปรตาม:
 - ความเร็วเชิงมุมของแบบจำลอง DC Motor

3. ตัวแปรควบคุม:

- บอร์ด STM32G474RE
- พารามิเตอร์ของมอเตอร์

เอกสารและงานวิจัยที่เกี่ยวข้อง

การจำลองระบบ DC Motor โดยใช้ Microcontroller จำเป็นต้องมีการทำ Discretization เนื่องจาก Transfer Function ของ DC Motor เป็นรูปแบบ Continuous แต่ Microcontroller มีการประมวลผลในรูปแบบ Discrete จึงจำเป็นต้องแปลง Transfer Function จาก S-Domain ให้เป็น Z-Domain ซึ่งสามารถทำได้ 3 วิธี คือ

1. Euler Discretization Method (Forward Rectangular Rule) โดย $s = \frac{z-1}{T_s}$
2. Backward Euler Method (Backward Rectangular Rule) โดย $s = \frac{z-1}{zT_s}$
3. Tustin Method (Trapezoidal Method) โดย $s = \frac{2(z-1)}{T_s(z+1)}$

ขั้นตอนการดำเนินงาน

1. หา Transfer Function ของ DC Motor ที่มีแรงดันไฟฟ้าเป็น Input และความเร็วเชิงมุมเป็น Output

จะได้ $\frac{\omega(s)}{V(s)} = \frac{K_m}{L_m J_m s^2 + (L_m B_m + R_m J_m)s + R_m B_m + K_m K_e}$ และเพื่อให้ง่ายต่อการจัดรูป

จึงกำหนดให้ $A = L_m J_m$, $B = L_m B_m + R_m J_m$ และ $C = R_m B_m + K_m K_e$

2. Discretize Transfer Function ของ DC Motor จากข้อ 1 ด้วย 3 วิธี ดังนี้

- 2.1 Euler Discretization Method แทน $s = \frac{z-1}{T_s}$ ในสมการข้อ 1. จะได้

$$\frac{\omega(z)}{V(z)} = \frac{K_m}{A\left(\frac{z-1}{T_s}\right)^2 + B\left(\frac{z-1}{T_s}\right) + C}$$

จากนั้นจัดรูปสมการจะได้สมการดังนี้

$$\omega[n] = \frac{(k_m T_s^2)U[n-2] + (2A + B T_s)Y[n-1] - (A + C T_s^2)Y[n-2]}{A + B T_s}$$

- 2.2 Backward Euler Method แทน $s = \frac{z-1}{zT_s}$ ในสมการข้อ 1 จะได้

$$\frac{\omega(z)}{V(z)} = \frac{K_m}{A\left(\frac{z-1}{zT_s}\right)^2 + B\left(\frac{z-1}{zT_s}\right) + C}$$

จากนั้นจัดรูปสมการจะได้สมการดังนี้

$$\omega[n] = \frac{(k_m T_s^2)U[n] + (2A + B T_s)Y[n-1] - (A)Y[n-2]}{A + B T_s + C T_s^2}$$

- 2.3 Tustin Method แทน $s = \frac{2(z-1)}{T_s(z+1)}$ ในสมการข้อ 1 จะได้

$$\frac{\omega(z)}{V(z)} = \frac{K_m}{A\left(\frac{2(z-1)}{T_s(z+1)}\right)^2 + B\left(\frac{2(z-1)}{T_s(z+1)}\right) + C}$$

จากนั้นจัดรูปสมการจะได้สมการดังนี้

$$\omega[n] = \frac{(k_m T_s^2)(U[n] + 2U[n-1] + U[n-2]) + (8A - 2CT_s^2)Y[n-1] - (2BT_s - 4A - CT_s^2)Y[n-2]}{4A + 2BT_s + CT_s^2}$$

3. นำทุกสมการที่ได้จากข้อ 2. ไปเขียนลงในโปรแกรม STM32CubeIDE เพื่อสร้างแบบจำลองมอเตอร์ด้วย Nucleo G474RE โดยใช้ค่า Parameter ของมอเตอร์เป็นค่าที่ได้จากการทดลองที่ 1 และค่า T_s ที่เลือกใช้มีค่าเท่ากับส่วนกลับของความถี่ในการจ่ายสัญญาณไปยังแบบจำลองมอเตอร์ ดังนี้

```
/* USER CODE BEGIN 4 */
float Method(char type, float Un) {
    if (type == 'f') {
        divi = (_A + (_B * Ts));
        CoEff_U = (Km * Ts * Ts);
        CoEff_Y1 = ((2 * _A) + (_B * Ts));
        CoEff_Y2 = (_A + (_C * Ts * Ts));
        return ((u_n2 * CoEff_U) + (y_n1 * CoEff_Y1) - (y_n2 * CoEff_Y2)) / divi;
    } else if (type == 'b') {
        divi = (_A + (_B * Ts) + (_C * Ts * Ts));
        CoEff_U = (Km * Ts * Ts);
        CoEff_Y1 = ((2 * _A) + (_B * Ts));
        CoEff_Y2 = _A;
        return ((Un * CoEff_U) + (y_n1 * CoEff_Y1) - (y_n2 * CoEff_Y2)) / divi;
    } else {
        divi = ((4 * _A) + (2 * _B * Ts) + (_C * Ts * Ts));
        CoEff_U = (Km * Ts * Ts);
        CoEff_Y1 = ((8 * _A) - (2 * _C * Ts * Ts));
        CoEff_Y2 = ((2 * _B * Ts) - (4 * _A) - (_C * Ts * Ts));
        return ((CoEff_U * (Un + (2 * u_n1) + u_n2)) + (y_n1 * CoEff_Y1) + (y_n2 * CoEff_Y2)) / divi;
    }
}
```

ภาพที่ 1 Code สมการ Discretization ของ DC Motor จากการ Discretize แต่ละวิธี

```
float Apply_Filter(float u_n) {
    float y_n;
    y_n = Method(method, u_n);

    u_n2 = u_n1;
    u_n1 = u_n; // เก็บ Input ปัจจุบันไว้เป็น n-1 ในรอบหน้า

    y_n2 = y_n1;
    y_n1 = y_n; // เก็บ Output ปัจจุบันไว้เป็น n-1 ในรอบหน้า

    return y_n;
}
```

ภาพที่ 2 Code เก็บค่าและแทนที่ Input และ Output ของครั้งก่อนหน้า

```
/* USER CODE BEGIN PD */
#define Pi (22.0f/7.0f)

#define Ts 0.02f
#define J 0.0000126856024302855f
#define L 0.002766949086f
#define R 3.376477398f
#define B 0.0000550590431422389f
#define Ke 0.0496487140384686f
#define Km 0.04963704582f

#define _A (J*L)
#define _B ((J*R)+(B*L))
#define _C ((R*B)+(Ke*Km))

/* USER CODE END PD */
```

ภาพที่ 3 Code เก็บค่า Parameter ของมอเตอร์ โดยค่าที่เลือกใช้เป็นค่าที่ได้จากการทดลองที่ 1

```
char method = 't';
float divi = 0;
float CoEff_U = 0;
float CoEff_Y1 = 0;
float CoEff_Y2 = 0;

float u_n1 = 0, u_n2 = 0;
float y_n1 = 0, y_n2 = 0;
/* USER CODE END PV */
```

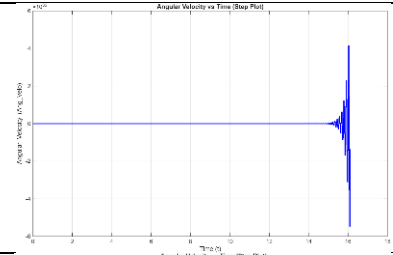
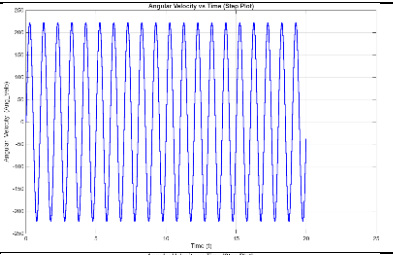
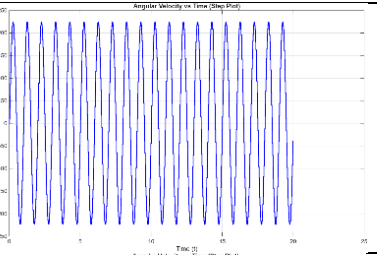
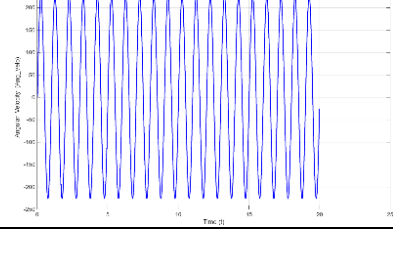
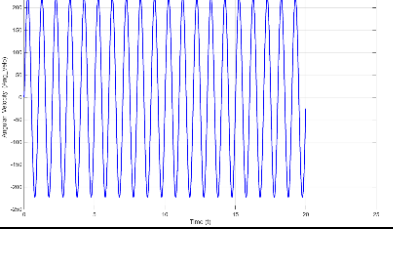
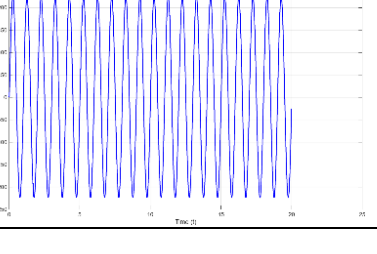
ภาพที่ 4 Code ประกาศตัวแปรที่เกี่ยวข้องกับสมการ Discretization

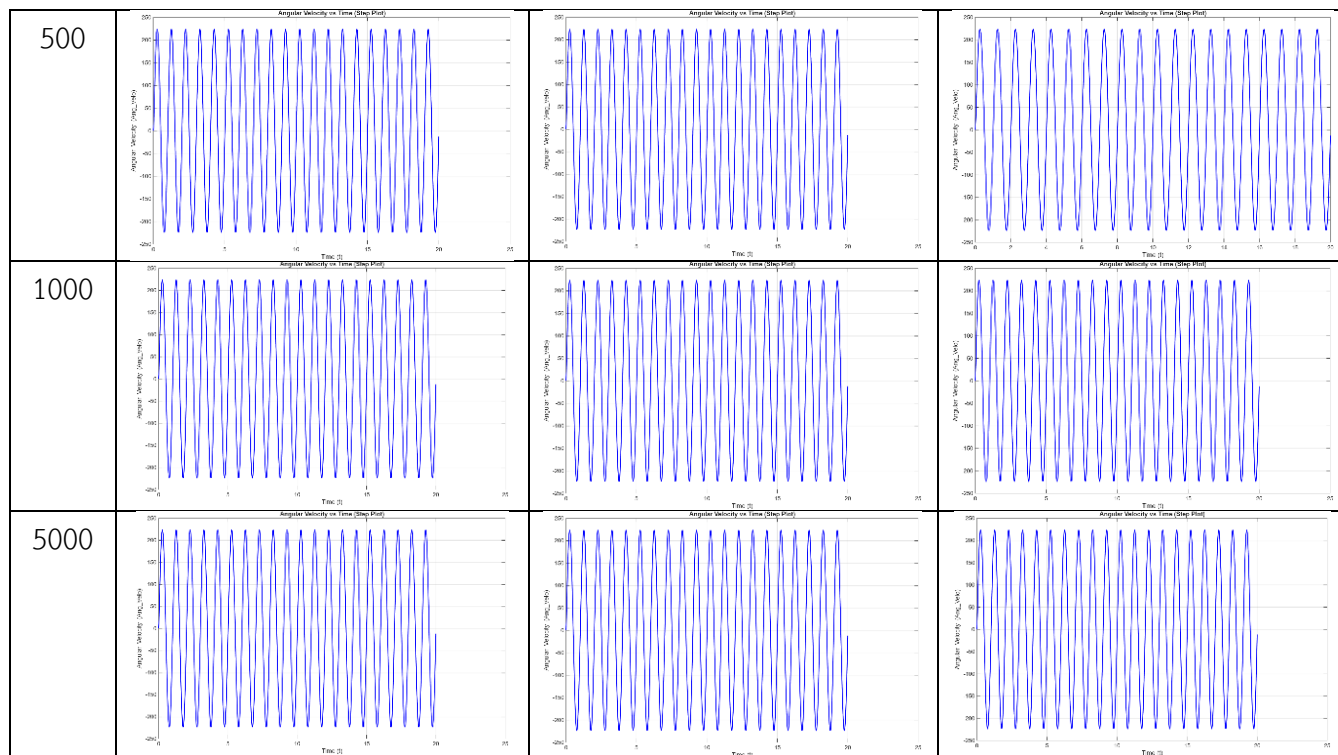
- เขียนโค้ดจ่ายสัญญาณ Input ที่อยู่ในช่วง -12 ถึง 12 โวลต์ ที่มีลักษณะสัญญาณเป็น Sine Wave : Frequency 1 Hz และ Ramp : Slope 1 Start Time 1
- ตั้งค่า Timer ใน IOC ที่เป็นตัวกำหนดความถี่ของแรงดันที่ Input ให้กับมอเตอร์แบบจำลอง โดยกำหนดความถี่เป็น 50 Hz, 100 Hz, 500 Hz, 1000 Hz และ 5000 Hz
- เก็บค่าสัญญาณความเร็วเชิงมุม Output ของมอเตอร์แบบจำลอง เป็นเวลา 20 วินาที และนำไป Plot Graph
- นำข้อมูลความเร็วเชิงมุม Output ที่ได้จากแบบจำลองมอเตอร์มาวิเคราะห์ และเปรียบเทียบความแตกต่างที่เกิดขึ้น

ผลการทดลอง

- สัญญาณรูปแบบ Sin Wave

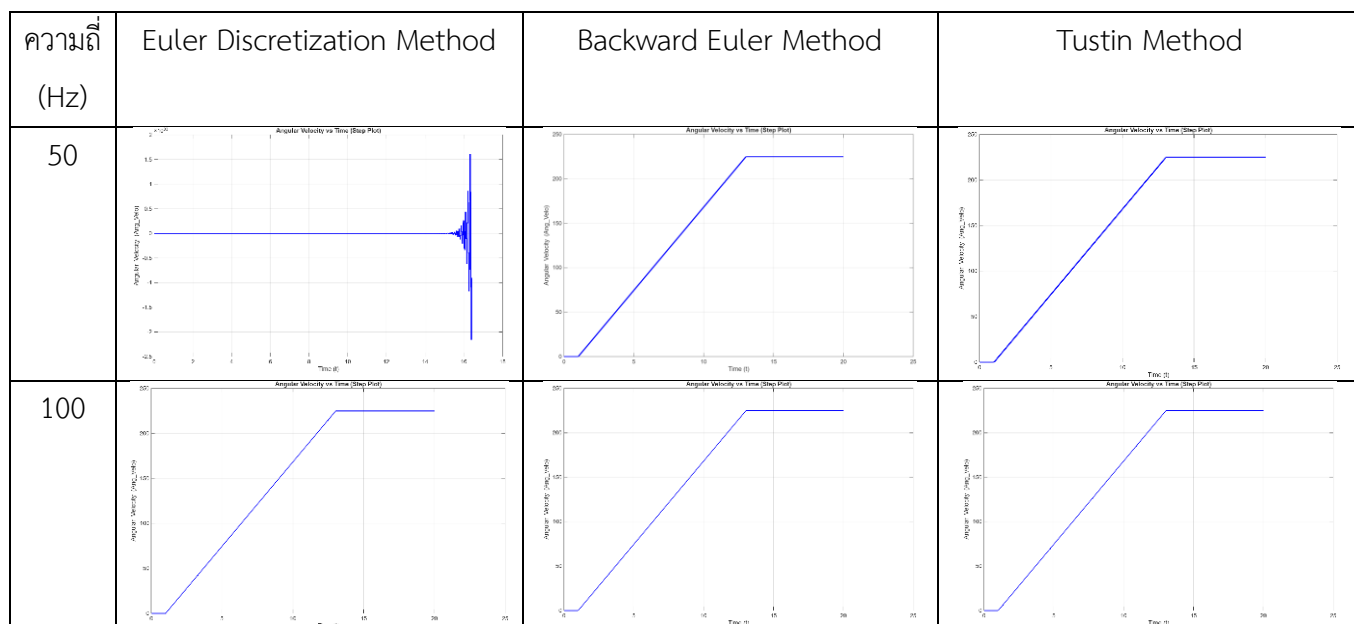
ตารางที่ 1 ตารางแสดงกราฟความเร็วเชิงมุมของมอเตอร์แบบจำลองรูปแบบ Sin Wave ที่สร้างจากวิธี Euler Discretization Method, Backward Euler Method และ Tustin Method ที่ความถี่ต่าง ๆ

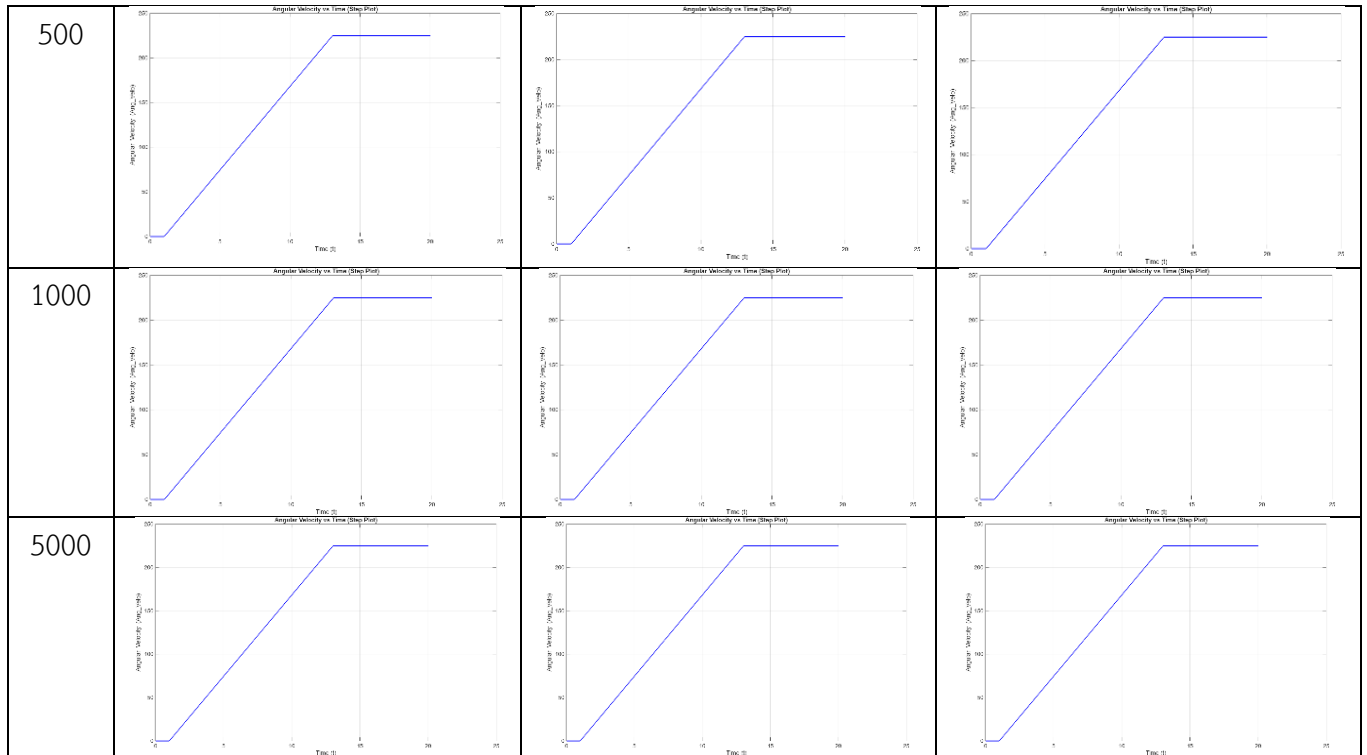
ความถี่ (Hz)	Euler Discretization Method	Backward Euler Method	Tustin Method
50			
100			



- สัญญาณรูปแบบ Ramp

ตารางที่ 2 ตารางแสดงกราฟความเร็วเชิงมุมของมอเตอร์แบบจำลองรูปแบบ Ramp ที่สร้างจากวิธี Euler Discretization Method, Backward Euler Method และ Tustin Method ที่ความถี่ต่าง ๆ

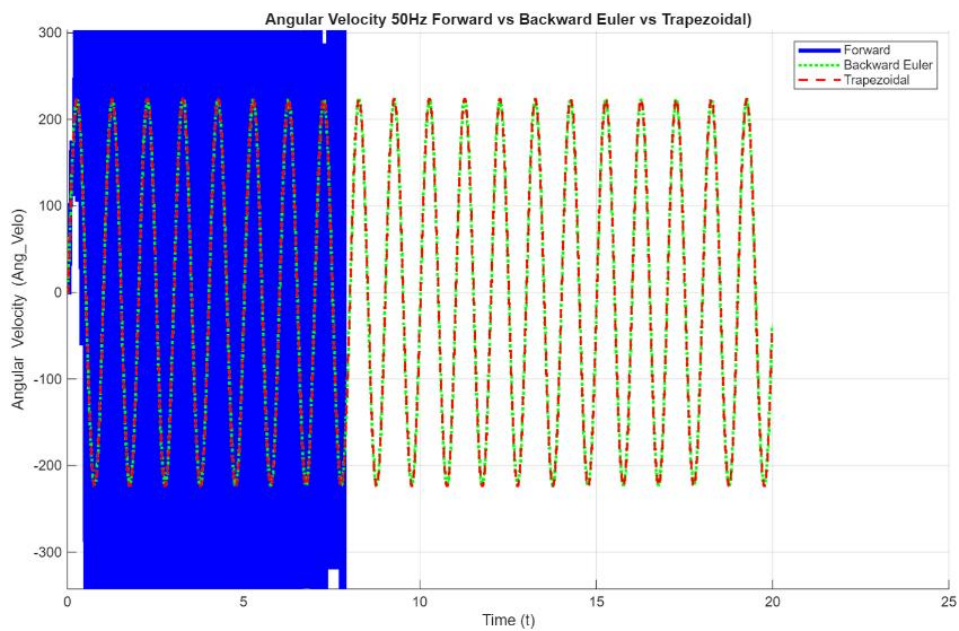




สรุปผลการทดลอง

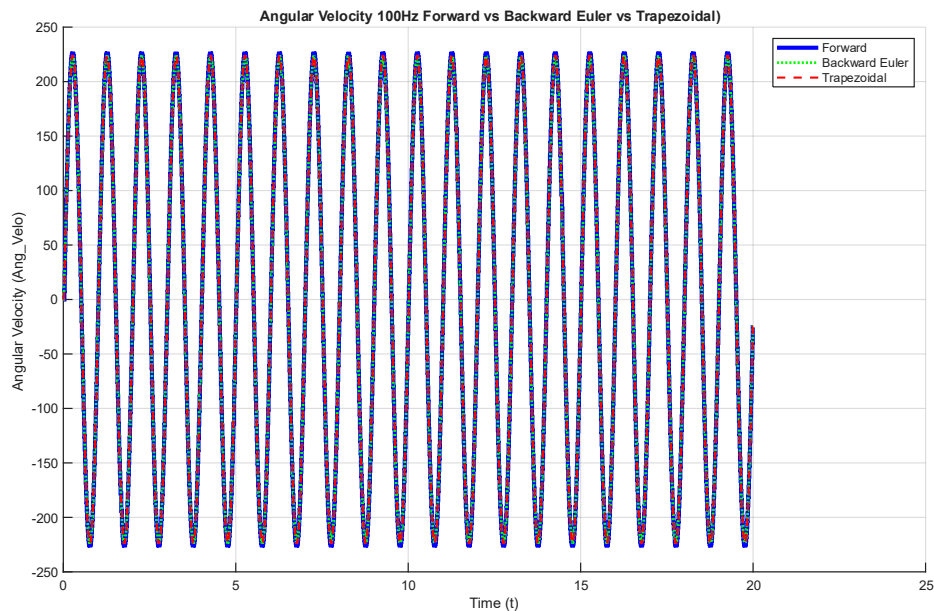
จากทดลองสามารถนำสัญญาณ Sine Wave แต่ละความถี่มาสร้างกราฟเทียบการคำนวณแต่ละวิธี ดังนี้

- ที่ 50 Hz



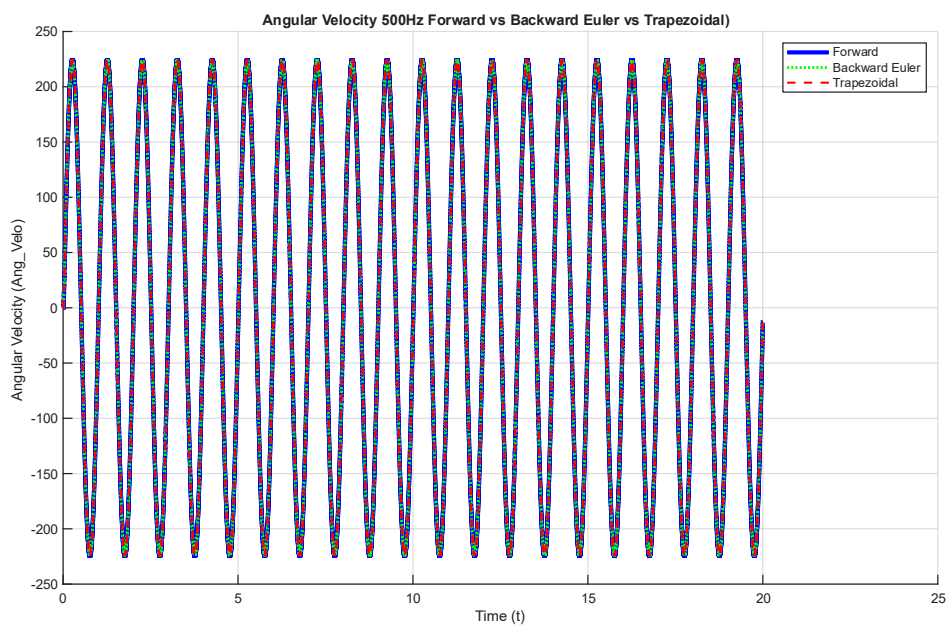
ภาพที่ 5 กราฟ Sine Wave เปรียบเทียบ Angular Velocity ของการคำนวณทั้ง 3 แบบ ที่ 50 Hz

- ที่ 100 Hz



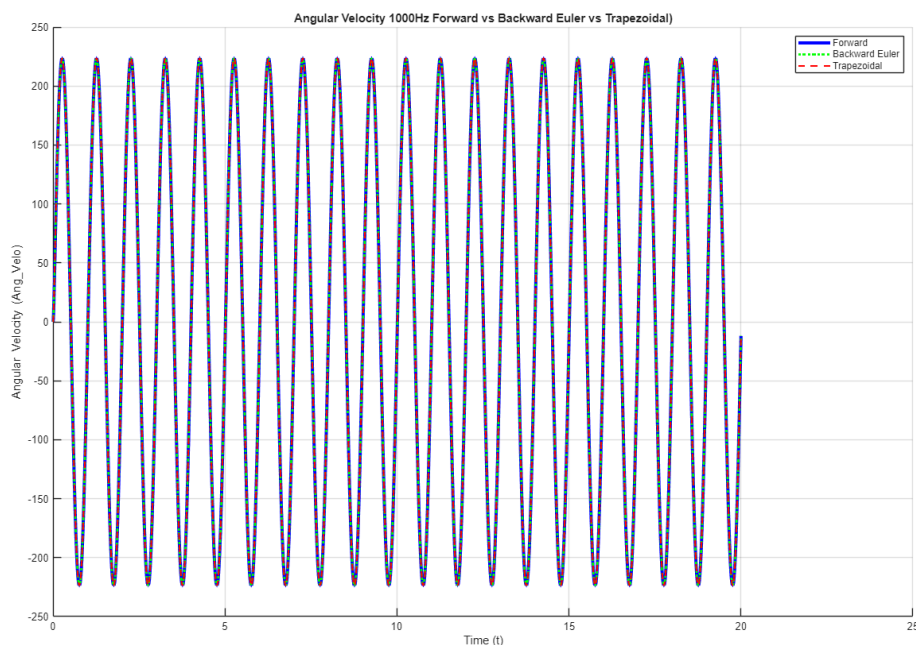
ภาพที่ 6 กราฟ Sine Wave เปรียบเทียบ Angular Velocity ของการคำนวณทั้ง 3 แบบ ที่ 100 Hz

- ที่ 500 Hz



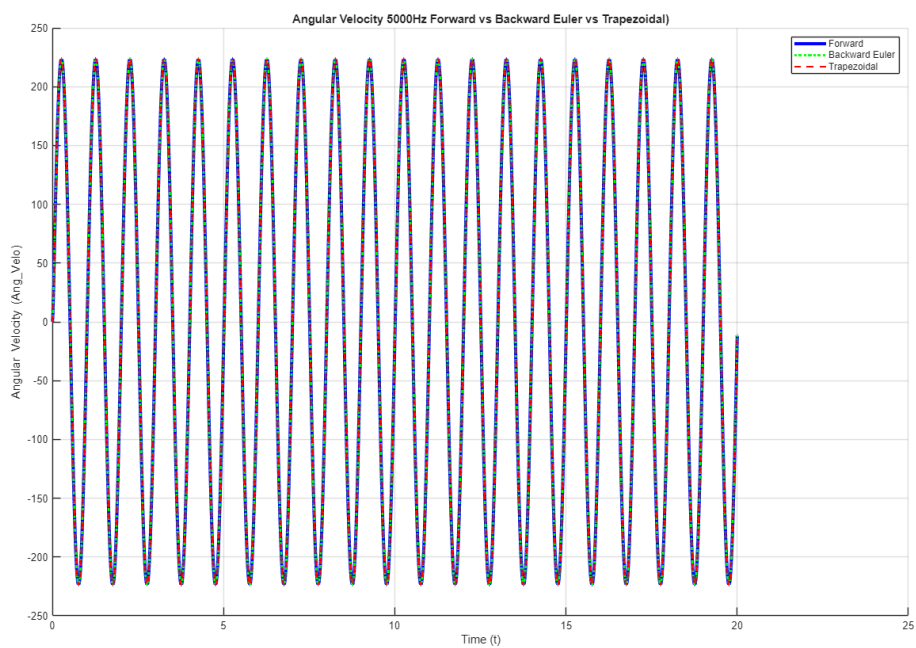
ภาพที่ 7 กราฟ Sine Wave เปรียบเทียบ Angular Velocity ของการคำนวณทั้ง 3 แบบ ที่ 500 Hz

- ที่ 1000 Hz



ภาพที่ 8 กราฟ Sine Wave เปรียบเทียบ Angular Velocity ของการคำนวณทั้ง 3 แบบ ที่ 1000 Hz

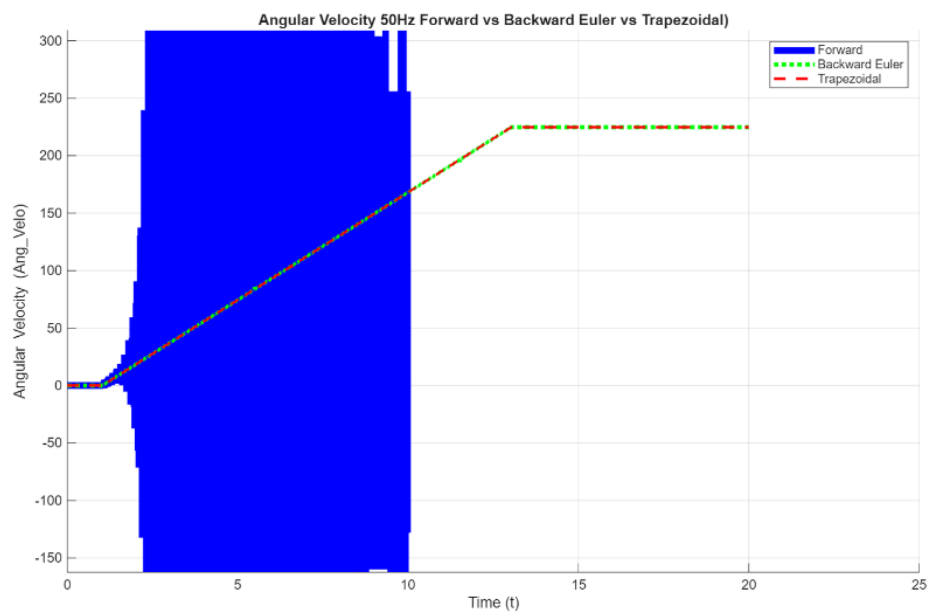
- ที่ 5000 Hz



ภาพที่ 9 กราฟ Sine Wave เปรียบเทียบ Angular Velocity ของการคำนวณทั้ง 3 แบบ ที่ 5000 Hz

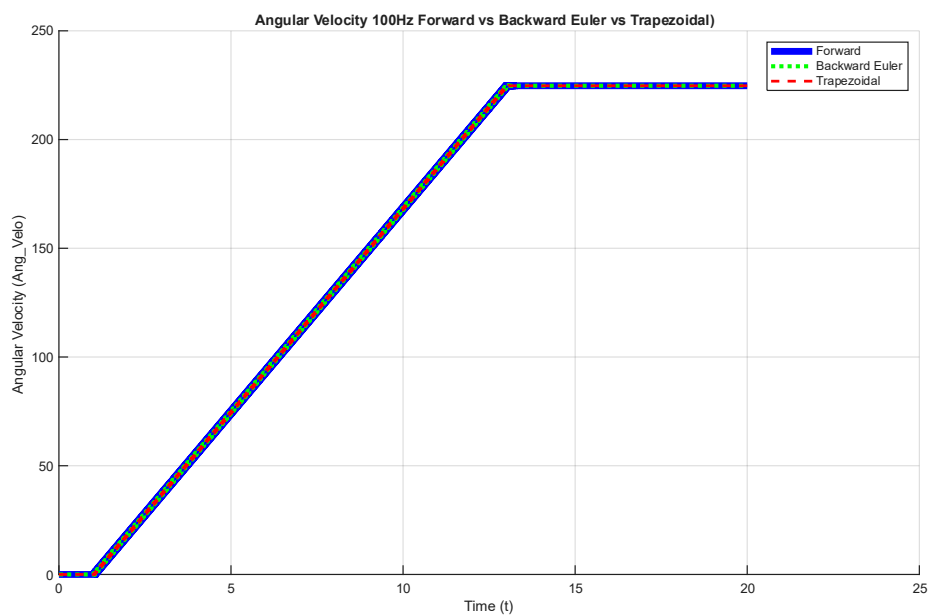
และสามารถนำสัญญาณ Ramp แต่ละความถี่มาสร้างกราฟเทียบการคำนวณแต่ละวิธี ดังนี้

- ที่ 50 Hz



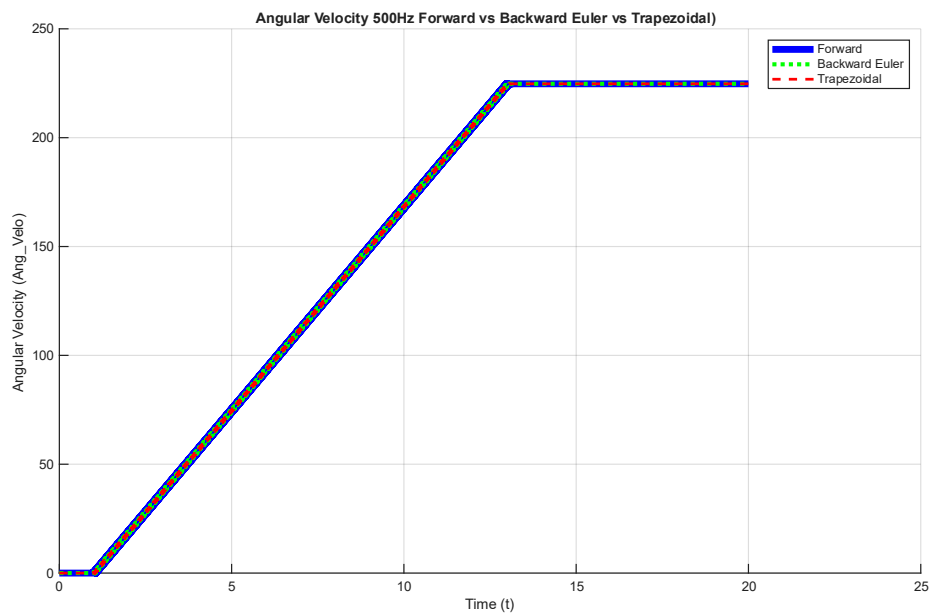
ภาพที่ 10 กราฟ Ramp เปรียบเทียบ Angular Velocity ของการคำนวณทั้ง 3 แบบ ที่ 50 Hz

- ที่ 100 Hz



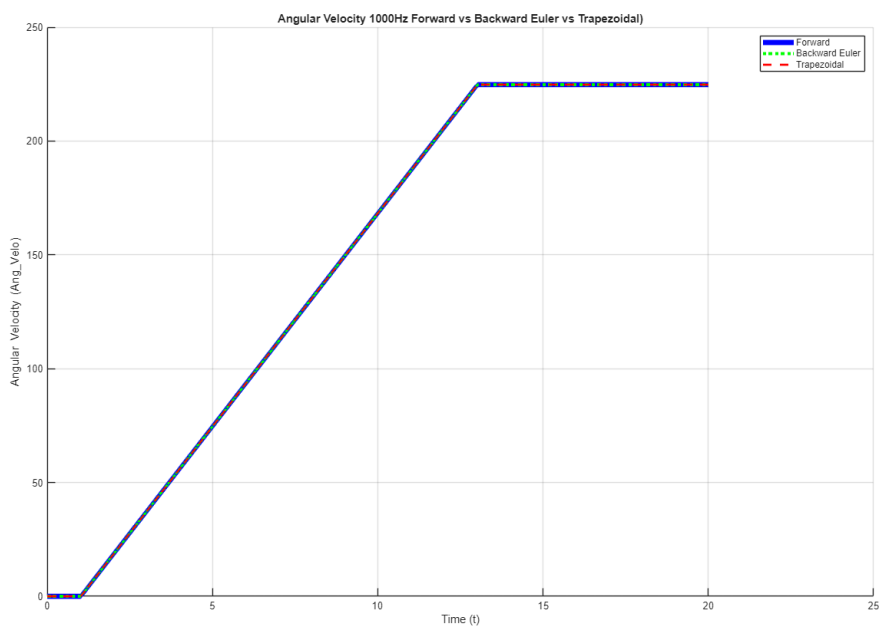
ภาพที่ 11 กราฟ Ramp เปรียบเทียบ Angular Velocity ของการคำนวณทั้ง 3 แบบ ที่ 100 Hz

- ที่ 500 Hz



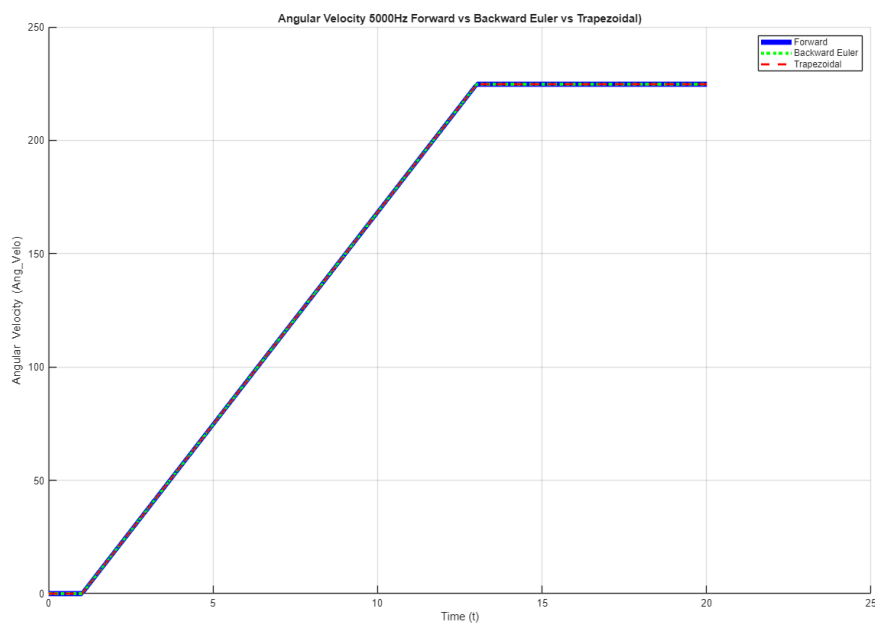
ภาพที่ 12 กราฟ Ramp เปรียบเทียบ Angular Velocity ของการคำนวณทั้ง 3 แบบ ที่ 500 Hz

- ที่ 1000 Hz



ภาพที่ 13 กราฟ Ramp เปรียบเทียบ Angular Velocity ของการคำนวณทั้ง 3 แบบ ที่ 1000 Hz

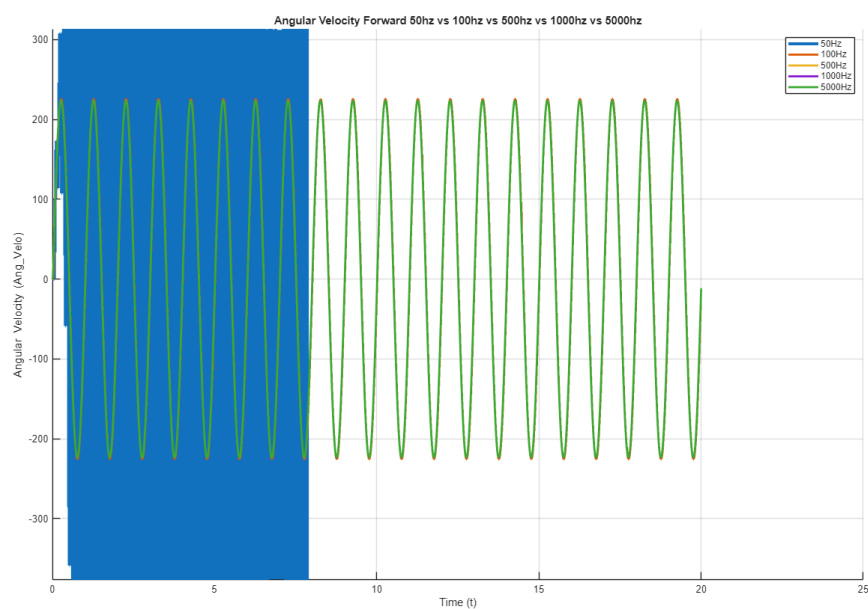
- ที่ 5000 Hz



ภาพที่ 14 กราฟ Ramp เปรียบเทียบ Angular Velocity ของการคำนวณทั้ง 3 แบบ ที่ 5000 Hz

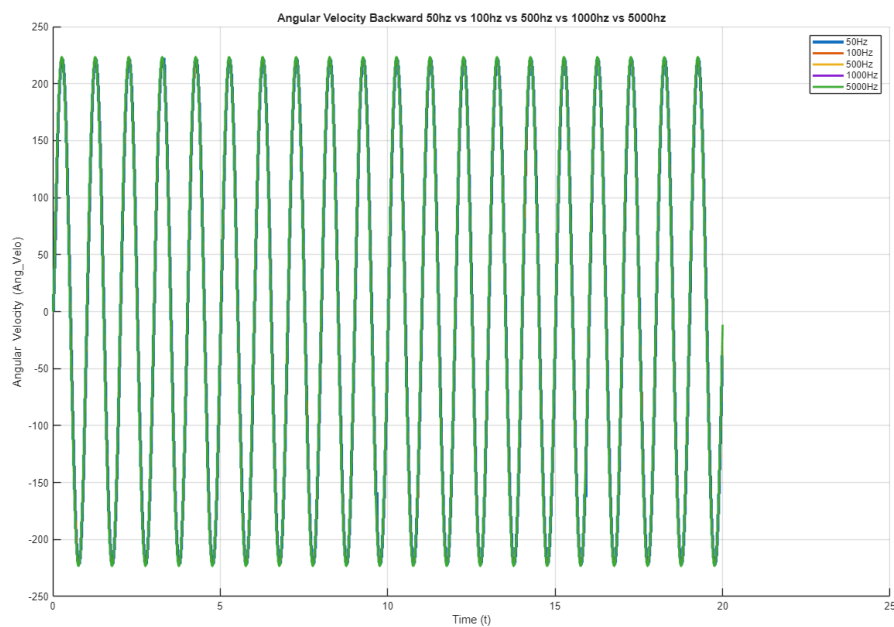
นอกจากนี้สามารถเปรียบเทียบกราฟ Sin Wave ที่ใช้วิธีคำนวณแบบเดียวกันแต่มีความถี่ต่างกันได้ ดังนี้

- Euler Discretization Method



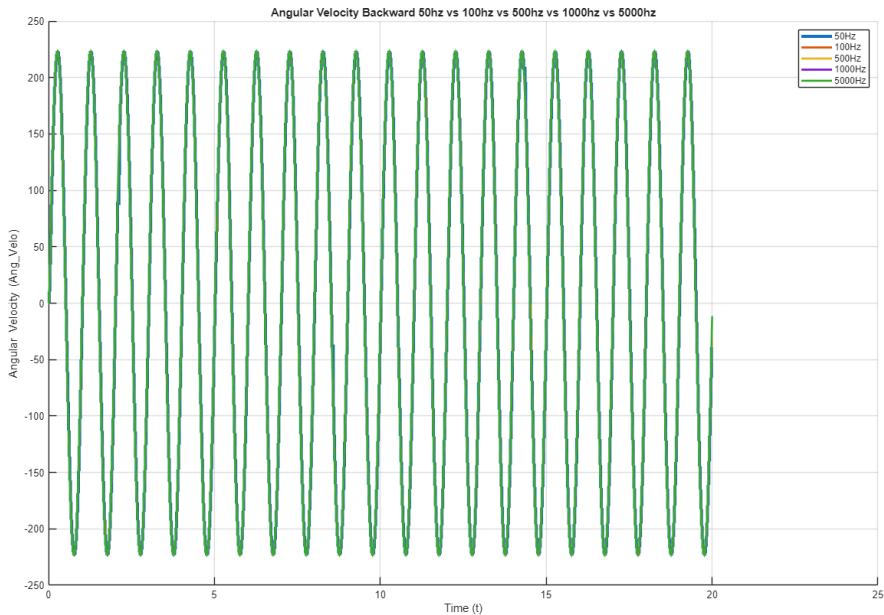
ภาพที่ 15 กราฟ Sin Wave เปรียบเทียบ Angular Velocity ของการคำนวณแบบ Forward ที่ Hz ต่างๆ

- Backward Euler



ภาพที่ 16 กราฟ Sin Wave เปรียบเทียบ Angular Velocity ของการคำนวณแบบ Backward ที่ Hz ต่างๆ

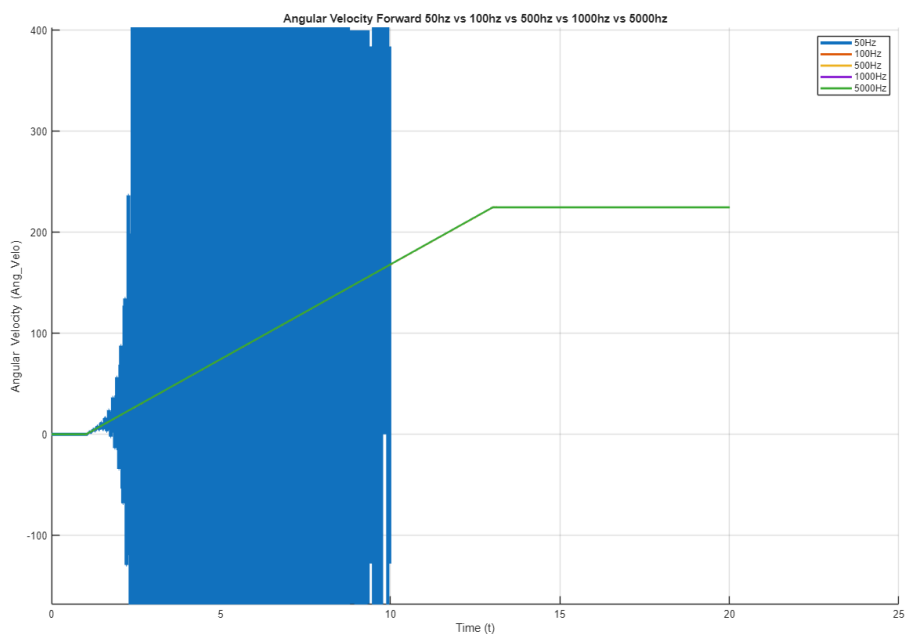
- Tustin Method



ภาพที่ 17 กราฟ Sin Wave เปรียบเทียบ Angular Velocity ของการคำนวณแบบ Trapezoidal ที่ Hz ต่างๆ

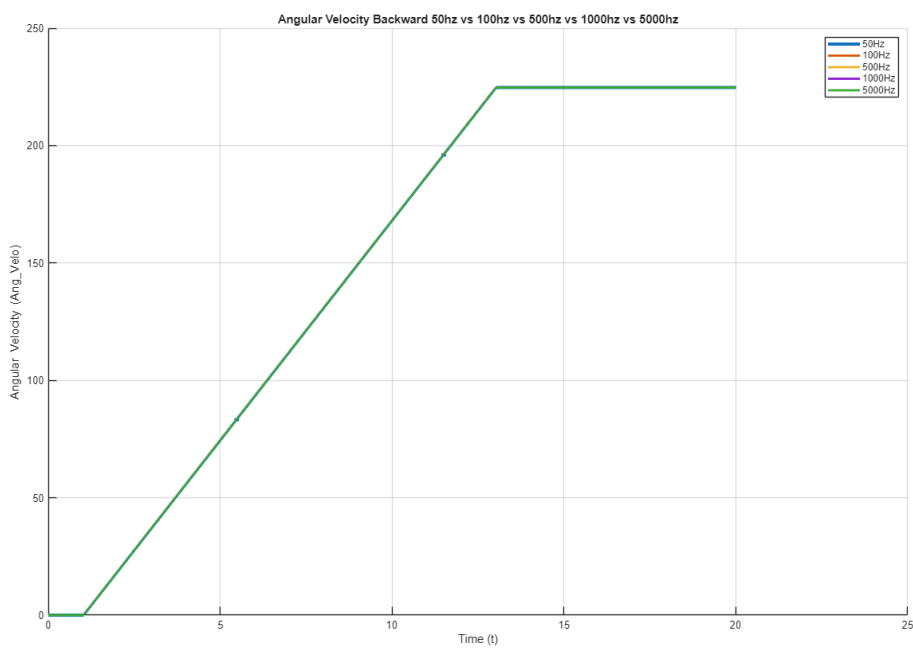
และเปรียบเทียบกราฟ Ramp ที่ใช้วิธีคำนวณแบบเดียวกันแต่มีความถี่ต่างกันได้ ดังนี้

- Euler Discretization Method



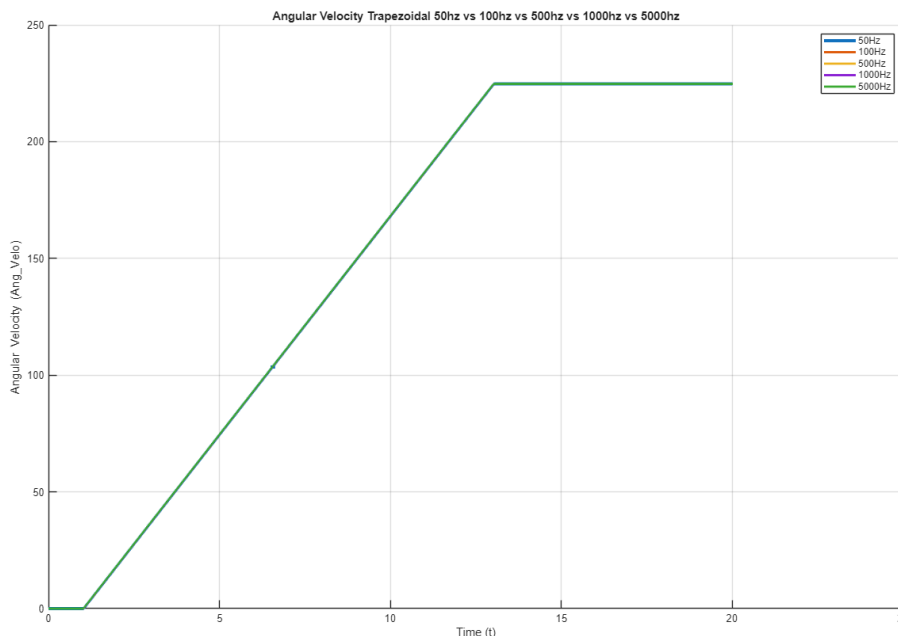
ภาพที่ 18 กราฟ Ramp เปรียบเทียบ Angular Velocity ของการคำนวณแบบ Forward ที่ Hz ต่างๆ

- Backward Euler Method



ภาพที่ 19 กราฟ Ramp เปรียบเทียบ Angular Velocity ของการคำนวณแบบ Backward Euler ที่ Hz ต่างๆ

- Tustin Method



ภาพที่ 20 กราฟ Ramp เปรียบเทียบ Angular Velocity ของการคำนวณแบบ Trapezoidal ที่ Hz ต่างๆ

อภิปรายผล

จากผลการทดลองและการสรุปผลการทดลอง จะพบว่าทั้งกราฟ Sin Wave และ Ramp ทุกๆ ความถี่ และทุกๆ วิธีการคำนวณ ให้ผลลัพธ์ที่ใกล้เคียง จนถึงเหมือนกันในทุกแบบ แต่จะมีความแตกต่างอยู่เพียงค่าเดียว คือ ที่ความถี่ 50 Hz และคำนวณแบบ Forward จะเห็นจากกราฟได้ว่ากราฟมีความไม่เสถียร และไม่สามารถพล็อตค่าออกมาได้

ข้อเสนอแนะ

1. ควรเลือกความถี่ในการจ่ายแรงดันให้เหมาะสมกับระบบ โดยไม่ควรต่ำหรือสูงเกินไป เนื่องจากความถี่ที่ต่ำเกินไปจะทำให้สัญญาณ Output มีความไม่ต่อเนื่องสูง ในขณะที่ความถี่สูงเกินไปจะทำให้แบบจำลองตอบสนองไม่ทัน
2. ควรเลือกใช้วิธี Tustin Method ในการ Discretize Transfer Function เนื่องจากให้ผลลัพธ์ที่มีความเสถียรและแม่นยำกว่า Euler Method ทั้งแบบ Forward และ Backward โดยเฉพาะที่ความถี่ต่ำ
3. ควรทดสอบแบบจำลองกับสัญญาณ Input หลายรูปแบบนอกเหนือจาก Sine Wave และ Ramp เพื่อให้เข้าใจพฤติกรรมของแบบจำลองได้ครอบคลุมมากยิ่งขึ้น

อ้างอิง

Discrete Transfer Function https://www.typhoon-hil.com/documentation/typhoon-hil-software-manual/References/discrete_transfer_function.html (2026)

ทฤษฎีการควบคุมระบบแบบไม่ต่อเนื่อง (Discrete Control Theory) [Didactic platform for DC motor speed and position control in Z-plane - ScienceDirect](#) (2021)

การทดลองที่ 1.2 การทดลองเพื่อวิเคราะห์ความแตกต่างระหว่างสัญญาณของแบบจำลอง DC Motor ที่ได้จาก STM32G474RE กับสัญญาณที่ได้จาก DC Motor จริง

จุดประสงค์

1. เพื่อให้สามารถวิเคราะห์สัญญาณตอบสนองรูปแบบ Sine Wave และ Ramp ของแบบจำลอง DC Motor ใน STM32G474RE และมอเตอร์จริงได้
2. เพื่อให้สามารถเปรียบเทียบความแตกต่างของสัญญาณที่ได้จากแบบจำลอง DC Motor ใน STM32G474RE และ DC Motor จริงได้

สมมติฐาน

สัญญาณที่ได้จากแบบจำลอง DC Motor ใน STM32G474RE มีความแตกต่างกับสัญญาณที่ได้จาก DC Motor จริง เนื่องจากความเป็น Non-Linear ที่เกิดขึ้นในมอเตอร์จริง และความต่อเนื่องของมอเตอร์จริง

ตัวแปร

1. ตัวแปรต้น:
 - มอเตอร์จริงและมอเตอร์แบบจำลอง
2. ตัวแปรตาม:
 - ความเร็วเชิงมุมของมอเตอร์
3. ตัวแปรควบคุม:
 - สัญญาณ Input ที่ใส่ให้กับแบบจำลองมอเตอร์ และมอเตอร์จริง
 - เวลาที่จ่ายสัญญาณ Input
 - Sample Time ของแบบจำลองมอเตอร์ และมอเตอร์จริง

เอกสารและงานวิจัยที่เกี่ยวข้อง

การจำลองระบบ DC Motor โดยใช้ Microcontroller จำเป็นต้องมีการทำ Discretization เนื่องจาก Transfer Function ของ DC Motor เป็นรูปแบบ Continuous แต่ Microcontroller มีการประมวลผลในรูปแบบ Discrete จึงจำเป็นต้องแปลง Transfer Function จาก S-Domain ให้เป็น Z-Domain ซึ่งสามารถทำได้ 3 วิธี คือ

1. Euler Discretization Method โดย $s = \frac{z-1}{T_s}$
2. Backward Euler Method โดย $s = \frac{z-1}{zT_s}$
3. Tustin Method โดย $s = \frac{2(z-1)}{T_s(z+1)}$

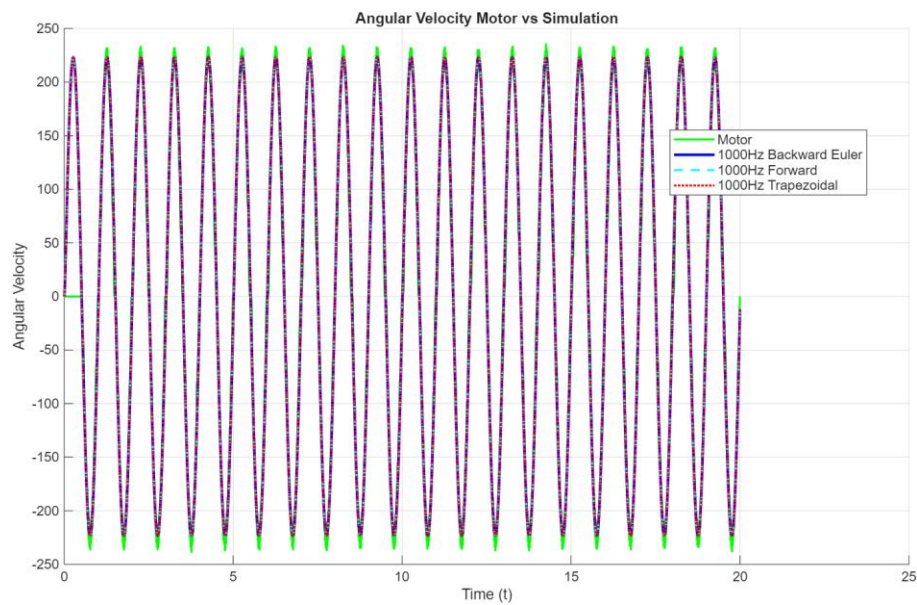
ขั้นตอนการดำเนินงาน

1. จ่ายสัญญาณ Input ที่อยู่ในช่วง -12 ถึง 12 โวลต์ ที่มีลักษณะสัญญาณเป็น Sine Wave : Frequency 1 Hz และ Ramp : Slope 1 Start Time 1 ให้กับมอเตอร์จริง โดยกำหนด Sample Time ให้เท่ากับ T_s ในสมการ Discretization ของมอเตอร์
2. เก็บค่าสัญญาณความเร็วเชิงมุม Output จากมอเตอร์จริงเป็นเวลา 20 วินาที และนำมา Plot Graph
3. นำข้อมูลความเร็วเชิงมุม Output จากแบบจำลองมอเตอร์ที่ได้จากการทดลองที่ 1 และความเร็วเชิงมุม Output จากมอเตอร์จริงมาเปรียบเทียบความแตกต่างที่เกิดขึ้น

ผลการทดลอง

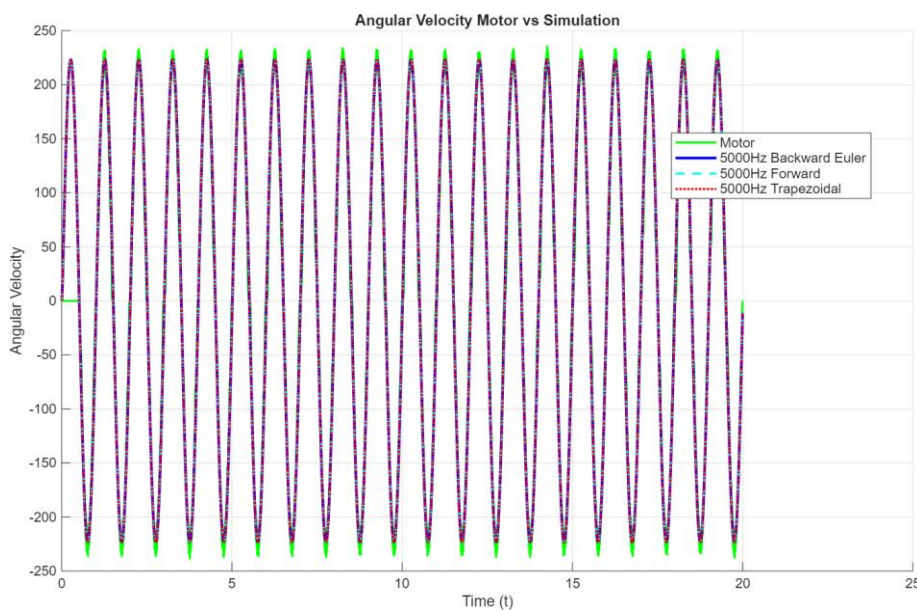
Sine Wave

- ที่ 1000 Hz



ภาพที่ 21 กราฟ Sine Wave เปรียบเทียบ Angular Velocity จากแบบจำลองและจากมอเตอร์ ที่ 1000 Hz

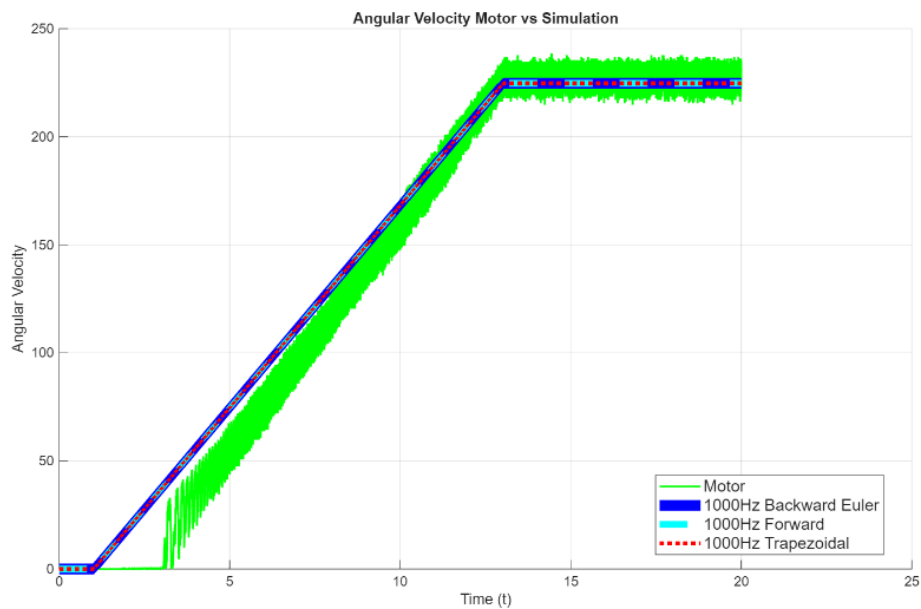
- ที่ 5000 Hz



ภาพที่ 22 กราฟ Sine Wave เปรียบเทียบ Angular Velocity จากแบบจำลองและจากมอเตอร์ ที่ 5000 Hz

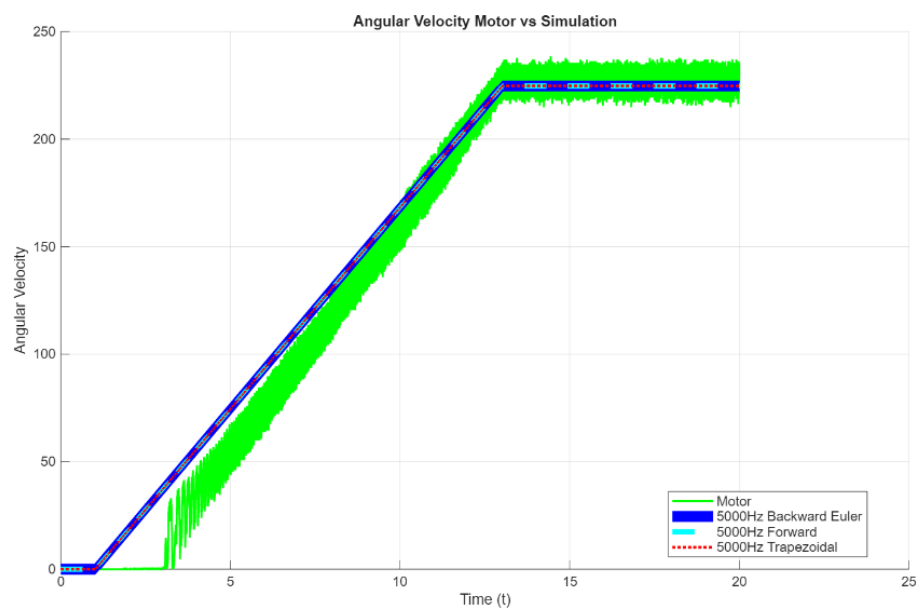
Ramp

- ที่ 1000 Hz



ภาพที่ 23 กราฟ Ramp เปรียบเทียบ Angular Velocity จากแบบจำลองและจากมอเตอร์ ที่ 1000 Hz

- ที่ 5000 Hz



ภาพที่ 24 กราฟ Ramp เปรียบเทียบ Angular Velocity จากแบบจำลองและจากมอเตอร์ ที่ 5000 Hz

สรุปผลการทดลอง

จากการทดลองเพื่อวิเคราะห์และเปรียบเทียบ Angular Velocity ระหว่างแบบจำลอง DC Motor บน STM32G474RE ที่ผ่านการ Discretization 3 วิธี (Backward Euler, Forward Euler, Tustin) กับ DC Motor ตัวจริง โดยทดสอบด้วยสัญญาณ Input แบบ Sine Wave และ Ramp ที่ความถี่ 1000 Hz และ 5000 Hz สามารถสรุปผลได้ดังนี้ :

การตอบสนองต่อสัญญาณ Sine Wave : แบบจำลองมอเตอร์ทั้ง 3 วิธี สามารถจำลองความเร็วเชิงมุมออกมาได้ใกล้เคียงกับพฤติกรรมของมอเตอร์จริงเป็นอย่างมาก ทั้ง Amplitude และ Phase ไม่ว่าจะใช้ความถี่ที่ 1000 Hz หรือ 5000 Hz

การตอบสนองต่อสัญญาณ Ramp : แบบจำลองแสดงผลการไต่ระดับความเร็วเป็นเส้นตรงที่ราบเรียบตามทฤษฎี แต่มอเตอร์จริง พบความแตกต่างอย่างชัดเจน 2 ส่วน คือ

1. มีความหน่วงในช่วงเริ่มต้น มอเตอร์จริงไม่ได้เริ่มหมุนทันทีที่เริ่มจ่ายสัญญาณ Ramp แต่ต้องรอให้แรงดันไฟฟ้าถึงจุดๆ หนึ่งก่อนจึงจะเริ่มหมุน
2. มีสัญญาณรบกวน สัญญาณจากมอเตอร์จริง มีความกว้างและไม่เรียบเนียนเหมือนแบบจำลองตลอดช่วงการทำงาน

อภิปรายผล

จากกราฟทดสอบด้วยสัญญาณ Ramp จะเห็นพฤติกรรม Non-linear ของมอเตอร์จริงได้อย่างชัดเจน ในช่วงเริ่มต้น มีสาเหตุหลักมาจากการที่มอเตอร์จริงมี แรงเสียดทานสถิต และมีลักษณะของ Dead Zone คือหากแรงดันไฟฟ้าที่จ่ายให้ยังมีค่าน้อยกว่าแรงเสียดทานภายใน มอเตอร์จะไม่สามารถเริ่มหมุนได้ ในขณะที่สมการของแบบจำลองเป็นแบบ Linear

เมื่อมอเตอร์จริงเริ่มหมุน สัญญาณที่อ่านได้ยังคงมีความกว้าง ซึ่งอาจเกิดจากปัจจัยทางกายภาพ เช่น ความคลาดเคลื่อนของ Encoder แรงเสียดทานจลน์ที่ไม่คงที่ หรือการสั่นสะเทือนทางกล ซึ่งแบบจำลองไม่ได้ครอบคลุมตัวแปรรบกวนเหล่านี้

จากการแปลงสมการด้วยวิธี Forward Euler, Backward Euler และ Tustin ให้ผลลัพธ์แทบไม่ต่างกันเลย เนื่องจากความถี่ที่ใช้ในการทดลองมีค่าสูงมาก ทำให้ผลลัพธ์ที่ได้เข้าใกล้ระบบแบบ Continuous

จากกราฟสัญญาณ Sine Wave ของมอเตอร์จริงและแบบจำลองตรงกันมากกว่า เพราะเป็นการหมุนแบบ มีโมเมนต์ต่อเนื่อง ซึ่งช่วยเอาชนะแรงเสียดทานได้ดีกว่าการเริ่มไต่ระดับความเร็วจากหยุดนิ่งแบบสัญญาณ Ramp

ข้อเสนอแนะ

1. ควรนำค่า Parameter ของมอเตอร์จริงมาอัปเดตในแบบจำลองอย่างสม่ำเสมอ เนื่องจากค่า Parameter อาจเปลี่ยนแปลงตามอุณหภูมิและการใช้งาน ซึ่งส่งผลให้ความแตกต่างระหว่างแบบจำลองและมอเตอร์จริงเพิ่มขึ้น
2. ควรพิจารณาเพิ่ม Non-linear Effects เช่น แรงเสียดทานสถิต และ Dead Zone เข้าไปในแบบจำลอง เพื่อให้แบบจำลองมีความใกล้เคียงกับพฤติกรรมของมอเตอร์จริงมากขึ้น
3. ควรทดสอบแบบจำลองกับสัญญาณ Input หลายรูปแบบและหลาย Sample Time เพื่อหาช่วงที่แบบจำลองให้ผลลัพธ์ใกล้เคียงกับมอเตอร์จริงมากที่สุด ก่อนนำไปใช้งานจริง

อ้างอิง

Discrete Transfer Function https://www.typhoon-hil.com/documentation/typhoon-hil-software-manual/References/discrete_transfer_function.html (2026)

ทฤษฎีการควบคุมระบบแบบไม่ต่อเนื่อง (Discrete Control Theory) [Didactic platform for DC motor speed and position control in Z-plane - ScienceDirect](#) (2021)

2. การออกแบบ K_p Controller

การทดลองที่ 2.1 การทดลองวิเคราะห์ผลของ Disturbance Feedforward แบบค่าดิบและแบบ ใส่ Low – Pass Filter

จุดประสงค์

1. เพื่อสังเกตความสามารถในการหักล้างผลของแรงโน้มถ่วงของ Disturbance Feedforward

สมมติฐาน

- การใส่ Disturbance Feedforward แบบ ค่าดิบ (assume ค่า L_m เป็น 0) จะทำให้เกิด Steady State Error มากกว่าการใช้แบบ Low Pass Filter

ตัวแปร

1. ตัวแปรต้น:
 - 1.1. การเลือกใช้ Disturbance Feedforward
 - 1.2. ค่า Cut-off Frequency ของ Low-Pass Filter
2. ตัวแปรตาม:
 - 2.1. ค่า Steady State Error ที่ได้จากการ Hold Position ของ Pendulum
3. ตัวแปรควบคุม:
 - 3.1. ตำแหน่ง Set Point ที่ต้องการ Hold Position
 - 3.2. ระยะเวลาในการ Run ผลการทดลอง

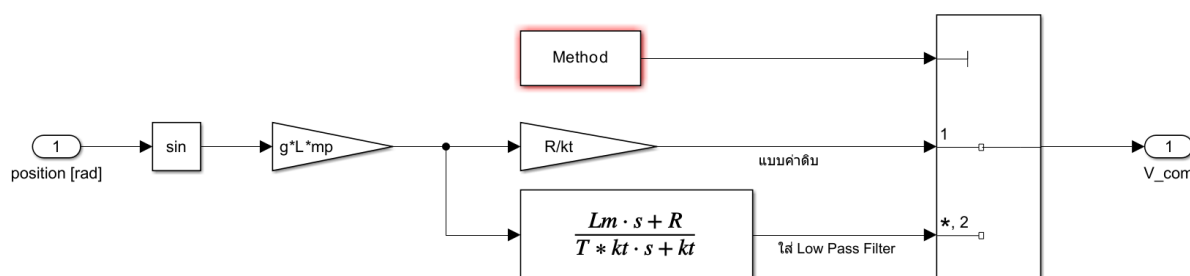
เอกสารและงานวิจัยที่เกี่ยวข้อง

Disturbance Feedforward คือเทคนิคการควบคุมที่ใช้ Sensor ตรวจจับสัญญาณรบกวนที่ส่งผลกระทบต่อระบบ แล้วนำไปคำนวณ Input เพิ่มเติมเพื่อลดผลกระทบของ Disturbance นั้น โดยจำเป็นต้องมี Mathematical Model ของระบบเพื่อทำนายผลของ Disturbance ได้อย่างถูกต้องในระบบ Vertical Pendulum แรงโน้มถ่วงที่กระทำต่อ Pendulum ถือเป็น Disturbance ที่สามารถคำนวณได้จากตำแหน่งเชิงมุม θ การนำ Feedforward มาใช้จะช่วยลดภาระของ PID Loop โดยการชดเชยแรงที่ทราบค่าได้ล่วงหน้า ทำให้ PID Loop ไม่จำเป็นต้องทำงานเพื่อต้านแรงโน้มถ่วงอีกต่อไปหรือเรียกว่า Gravity Compensation

สาเหตุที่ต้องใส่ Low-Pass Filter เข้าไปใน Disturbance Feedforward เนื่องจาก Transfer Function ของ Feedforward ในรูปแบบ $G_{ff}(s) = \frac{L_m \cdot s + R_m}{k_t}$ นั้นมีพจน์ $L_m \cdot s$ ซึ่งทำให้ Transfer Function เป็น Improper และไม่สามารถนำไปใช้งานได้จริง จึงจำเป็นต้องเพิ่ม Low-Pass Filter เข้าไปเพื่อให้ Transfer Function กลายเป็น Proper และ Realizable โดยค่า Time Constant ของ Filter ที่ต่ำจะให้การ Reject Disturbance ที่ดีกว่า แต่จะมีการขยาย Noise จาก Sensor มากขึ้นดังนั้นการเลือกค่า Time Constant ที่เหมาะสมจึงเป็น Trade-off ระหว่าง Disturbance Rejection และ Noise Suppression

ขั้นตอนการดำเนินงาน

1. Setup โปรแกรม Simulink สำหรับการเก็บค่าและบันทึกผล
2. คำนวณหา Disturbance Feedforward ของ Pendulum โดยจะได้สมการดังนี้ (การคำนวณดูได้ใน Appendix B การทดลองที่ 2.1)
3. กำหนดให้ K_p ของ Controller มีค่าเป็น 0 และกำหนดให้ Setpoint และ Initial Point ไว้ที่ 90 องศา เพื่อให้เห็นถึงประสิทธิภาพของ Disturbance Feedforward เท่านั้น
4. สร้าง Switch เลือกการใช้ Disturbance Feedforward โดยกำหนดให้ Method 1 คือ Method 2 คือ Disturbance Feedforward แบบค่าดิบ และ Method 2 คือ Disturbance Feedforward แบบใส่ Low-Pass Filter ดังภาพที่ 19



ภาพที่ 25 Block สำหรับการตัดสินใจเลือกรูปแบบ Disturbance Feedforward

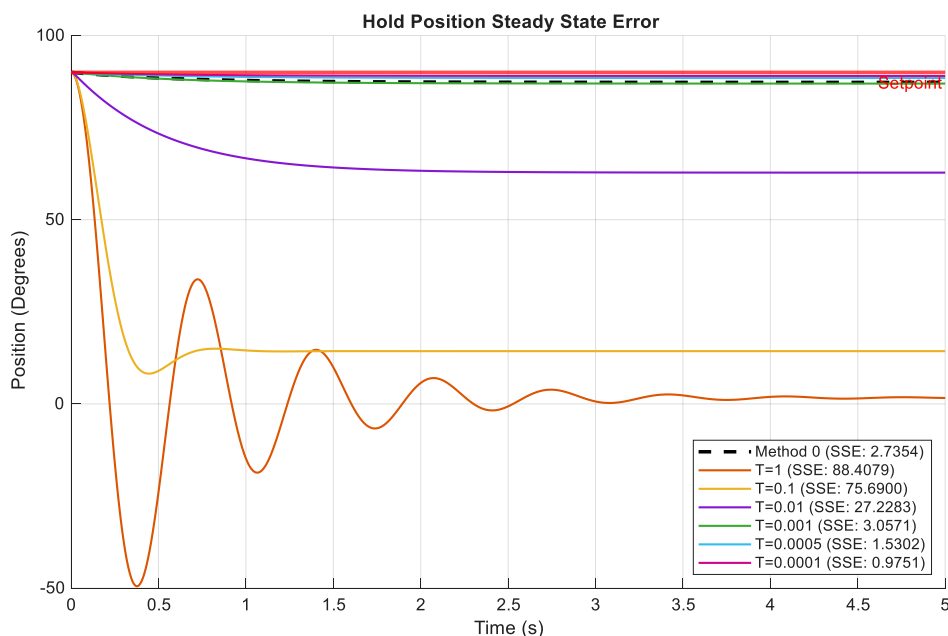
5. กำหนด Set ของ ค่า T ใน Disturbance Feedforward โดยกำหนดให้ Set ของค่า T ดังภาพที่ 26

```
T_vector = [1,0.1,0.01,0.001,0.0005,0.0001];
```

ภาพที่ 26 Vector ของค่า Tau ใน MATLAB

6. เขียนโปรแกรมเพื่อเก็บค่าตำแหน่งของ Pendulum และค่า Steady State Error

ผลการทดลอง

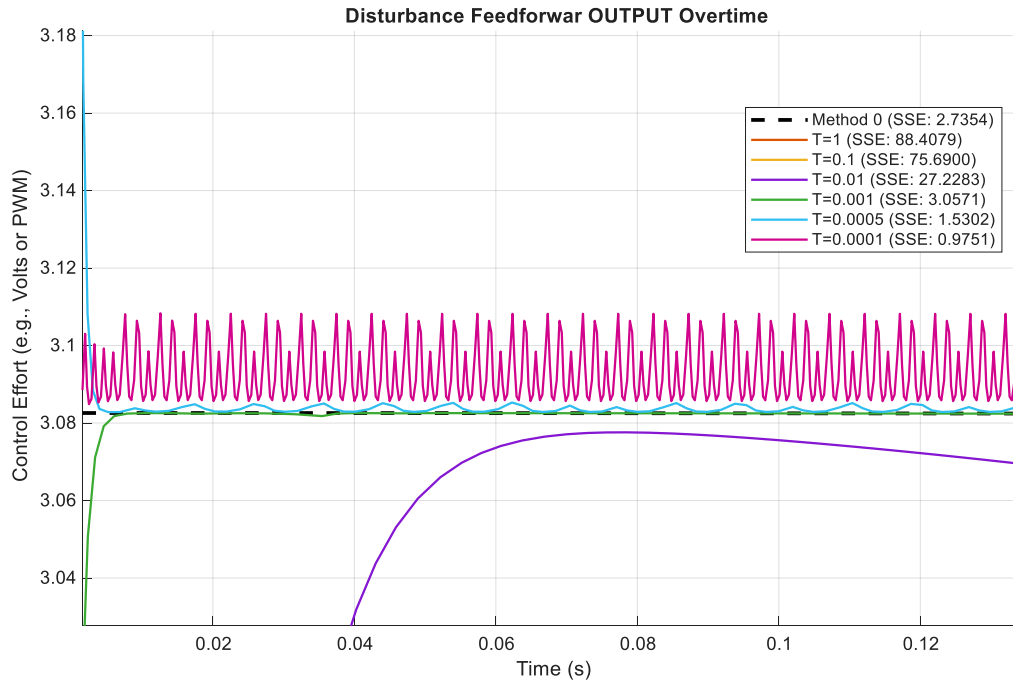


ภาพที่ 27 แสดงตำแหน่งองศาของ Pendulum ในแต่ละเวลา

สรุปผลการทดลอง

จากการทดลองแสดงให้เห็นว่าเมื่อปรับค่า T หรือ ค่า Tau ให้น้อยลงจะทำให้ค่า Steady State Error มีค่าต่ำลงเช่นเดียวกัน โดยจะพบว่าจะได้ค่า Steady State Error น้อยที่สุดที่ 0.9751 องศา เมื่อค่า Tau อยู่ที่ 0.0001 องศา และจะมีมากที่สุดเมื่อค่า Tau มีค่า 1 วินาที

จากผลการทดลองพบว่าหากค่า Tau มีค่าน้อยกว่าค่า R/kt ซึ่งจาก Parameter ของ Motor ปัจจุบันจะอยู่ที่ 1117.75 Hz ซึ่งหากนำค่าที่ได้มาแปลงเป็นค่า Tau จะอยู่ที่ $8.947e-04$ วินาที จะทำให้ค่า Steady State Error มีค่าต่ำกว่าการวิธี แทนค่าดิบ แต่ในทางกลับกันหากนำค่าของ Disturbance Feedforward มา zoom เพื่อสังเกตกราฟจะพบว่า Disturbance Feedforward จะมีลักษณะเป็นลูกคลื่นคล้าย Noise ดังภาพที่ 22 โดยที่ยังค่า Tau มีค่าน้อยกว่า $8.947e-04$ วินาทีลูกคลื่นจะมีความกว้างมากขึ้น แต่ในทางกลับกัน Voltage ภาพรวมจะมีค่ามากขึ้นเช่นกัน



ภาพที่ 28 การขยายภาพจากภาพที่ xx

อภิปรายผล

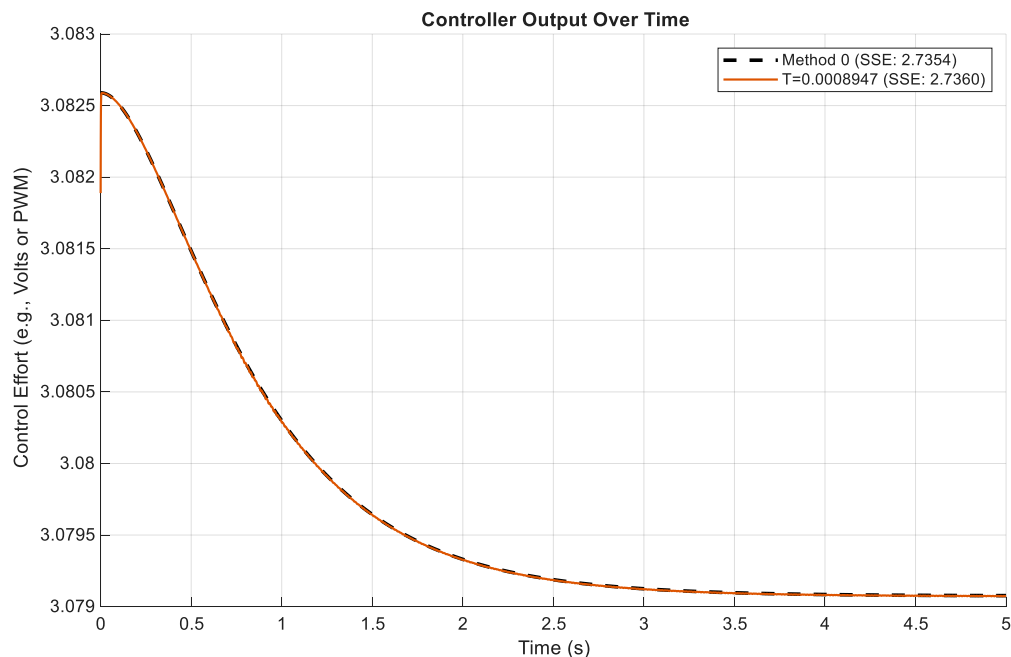
จากการทดลองแสดงให้เห็นว่า Response ของระบบอยู่ที่ 1117.75 Hz สังเกตได้จากเมื่อปรับ Tau ต่ำกว่า $8.947e-04$ วินาที ระบบจะมีลักษณะคล้ายกับ Noise เพิ่มขึ้นมา โดยหากทำการทดสอบโดยการทดลองให้ Tau ที่ $8.947e-04$ วินาที จะได้ Disturbance Feedforward และ Steady State Error เท่ากับแบบค่าดิบตั้ง ภาพที่ 33 ทั้งนี้แม้จะเลือก Tau-Constant มากกว่า Response ของระบบ ก็จะไม่ส่งผลต่อใน Position เนื่องจาก Motor นั้นประพุดติตัวคล้ายคลึงกับ Motor เช่นเดียวกันจากพจน์ โดยหาแปลงเป็น Cut-Off Frequency จะอยู่ที่

ทั้งนี้ไม่ว่าจะเป็นการใช้ Disturbance Feedforward แบบใดก็ตามจะพบว่า **Disturbance Feedforward** ไม่สามารถหักล้างผลของ **Gravity** ได้อย่างสมบูรณ์

$$\frac{1}{Js + B} = \frac{\frac{B}{J}}{s + \frac{B}{J}}$$

$$\frac{B}{J} = 0.318 \text{ Hz}$$

โดยในภาพรวมค่า Tau Constant ที่จะนำไปใช้ต่อคือ 0.0001 วินาที



ภาพที่ 29 เปรียบเทียบการจ่าย Voltage ของ Disturbance Feedforward

ข้อเสนอแนะ

1. หากตั้งค่า Fixed-Step ของ Simulink ต่ำกว่าค่า Tau Constant ใน Disturbance Feedforward จะส่งผลให้ Simulink Error ได้ เท่ากับว่าเราควรตั้งให้ Fixed-Step ของ Simulation ต่ำกว่าหรือเท่ากับ Tau Constant เสมอ

อ้างอิง

Bandwidth constraints of disturbance observer in the presence of real parametric uncertainties [Bandwidth constraints of disturbance observer in the presence of real parametric uncertainties](#) (2013)

An improved stability compensation for feedforward active noise control systems with acoustic feedback [Microsoft Word - 01-23-00821-2003.doc](#) (2024)

การทดลองที่ 2.2 การทดลองเปรียบเทียบความแตกต่างของหา K_p Controller Gain ด้วยวิธีการต่างๆ

จุดประสงค์

1. เพื่อเลือกวิธีการในการหา K_p Controller Gain เบื้องต้นในการทดลองผลของ Gravity Compensation
2. เพื่อเลือกช่วงของ K_p ไปทดลองต่อในการทดลองที่ 2.3 วิเคราะห์ผลของ Gravity Compensation ที่ส่งผลกระทบต่อระบบ

สมมติฐาน

- ค่า K_p Controller Gain ของแต่ละวิธีควรจะได้ค่าที่ใกล้เคียงกัน

ตัวแปร

1. ตัวแปรต้น:
 - 1.1. วิธีการได้มาของ K_p Controller
2. ตัวแปรตาม:
 - 2.1. ผลของการนำค่า K_p มา Validation และวัดค่า Stability ของระบบในแต่ละองศา
3. ตัวแปรควบคุม:
 - 3.1. Plant ของระบบ

เอกสารและงานวิจัยที่เกี่ยวข้อง

การหาค่า Controller Gain (K_p) เป็นขั้นตอนในการกำหนดสมรรถนะของตัวควบคุม PID โดยค่า K_p ที่เหมาะสมจะช่วยลดความผิดพลาดที่สภาวะคงที่ (Steady-state Error) และเพิ่มความเร็วในการตอบสนองของระบบ วิธีการดั้งเดิมอย่าง **Root Locus** อาศัยการวิเคราะห์ตำแหน่ง Poles S-plane เพื่อหาความสัมพันธ์ระหว่าง Gain และเสถียรภาพของระบบ ช่วยให้เห็นภาพรวมและขีดจำกัดสูงสุดของ Gain ที่ระบบยังคงเสถียรอยู่ได้ การคำนวณด้วยวิธีนี้อาจมีความคลาดเคลื่อนและไม่คล่องตัวเมื่อระบบมีความซับซ้อนมากขึ้น การใช้เครื่องมือ **Control System Design Toolbox** ช่วยให้สามารถหาค่า Gain แบบเชิงกราฟิกและเห็นผลตอบสนอง (Step Response) เปลี่ยนแปลงแบบทันที อีกทั้งยังมีการใช้ PID Controller Block ในรูปแบบของ **Continuous** และ **Discrete** เพื่อวิเคราะห์ความแตกต่างทางทฤษฎีและการ simulation ซึ่งการหาค่า Continuous จะให้พารามิเตอร์โมเดลในเวลาอุดมคติแต่หากหาค่า

Discrete จะพิจารณาผลกระทบของคาบเวลาแบบสุ่ม (T_s) และหาก T_s ไม่เหมาะสมจะทำให้เสถียรภาพของระบบคลาดเคลื่อน

ขั้นตอนการดำเนินงาน

1. กำหนด Requirements ของระบบที่ต้องการให้ K_p อยู่ ซึ่ง requirement ของระบบดังต่อไปนี้

Requirement	Value	Unit
Overshoot Percentage	≤ 10	%
Peak Time	≤ 3	seconds
Voltage Range	$[-12, 12]$	volts

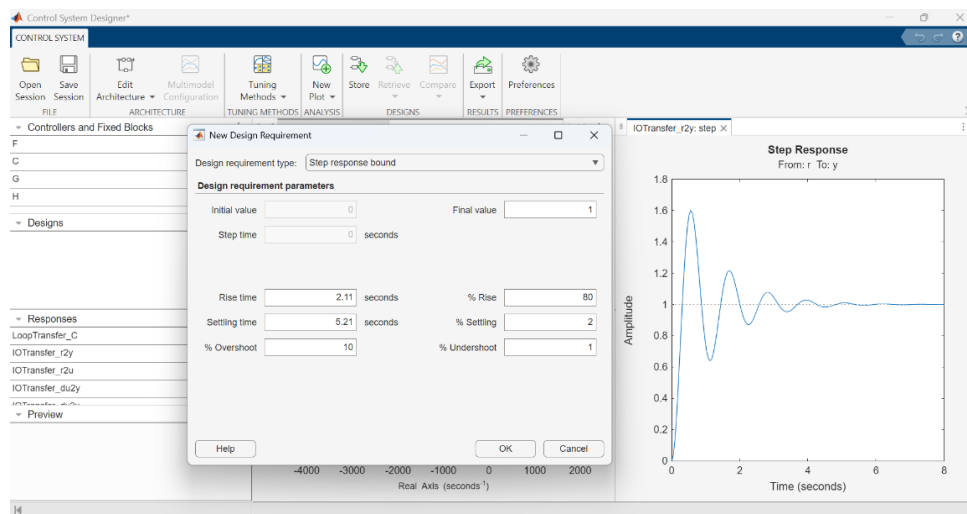
ภาพที่ 30 Requirement ของการทดลองที่ 2 การออกแบบ K_p Controller

2. คำนวณหาค่า K_p โดยใช้วิธีการ คำนวณ Root Locus Controller Gain (ดูต่อใน Appendix B การทดลองที่ 2.2)
3. คำนวณหาค่า K_p โดยใช้ Controller System Design
4. คำนวณหาค่า K_p โดยใช้ PID Controller Block แบบ Continuous
5. คำนวณหาค่า K_p โดยใช้ PID Controller Block แบบ Discrete
6. เปรียบเทียบค่า K_p ที่ได้จากแต่ละวิธีการ มา Validate เพื่อวิเคราะห์หา %Overshoot และ Peak time

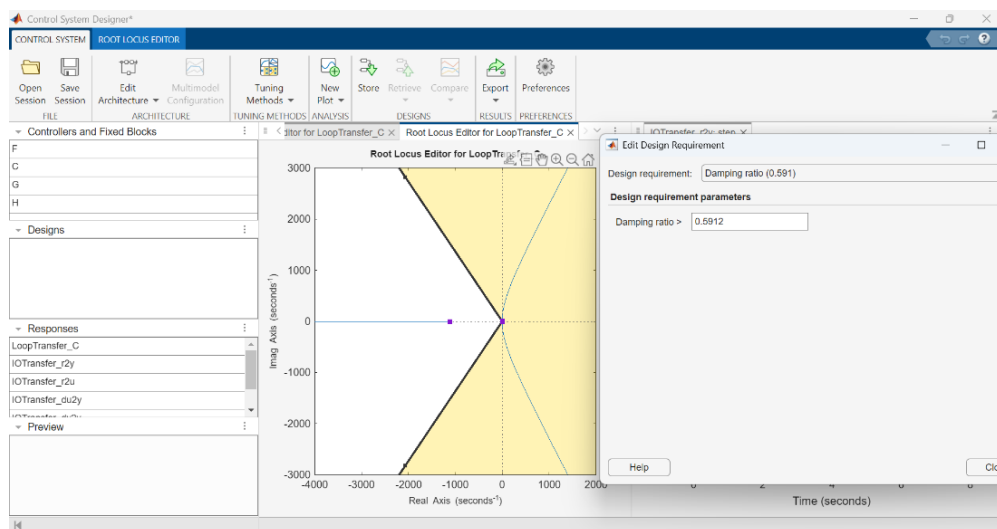
ผลการทดลอง

ตารางที่ 3 ผลการ Tuning ด้วยวิธีการต่างๆ

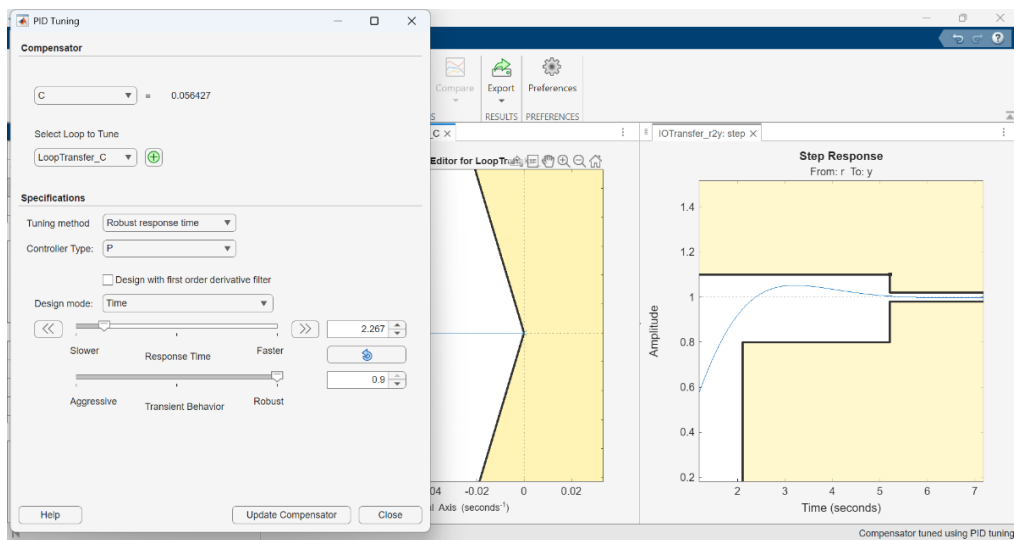
วิธีการ	คำนวณ Root Locus Controller Gain	Controller System Design	PID Controller Block แบบ Continuous	PID Controller Block แบบ Discrete
ค่า K_p ที่ได้	0.06235	0.056427	0.065754	0.069864



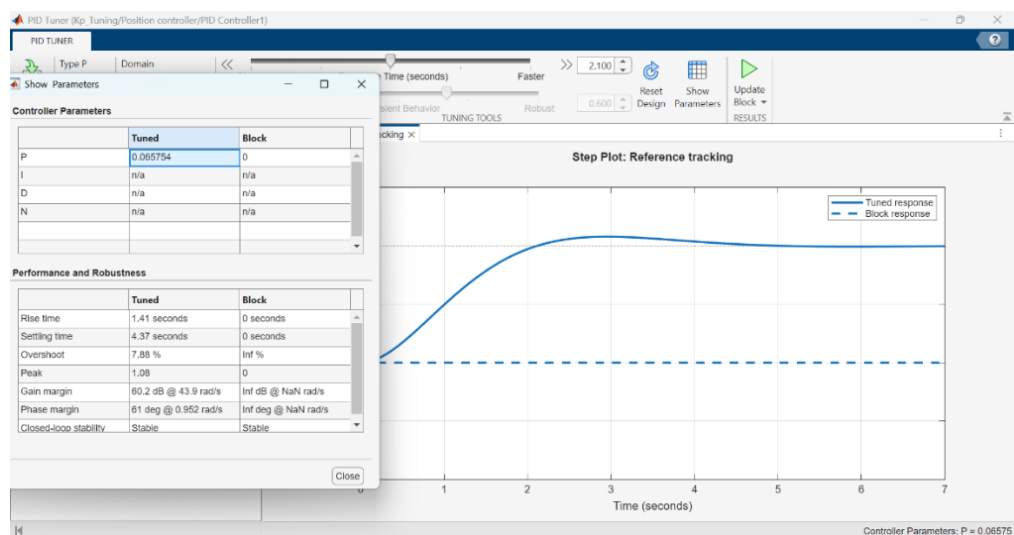
ภาพที่ 31 Tuning Controller System Design



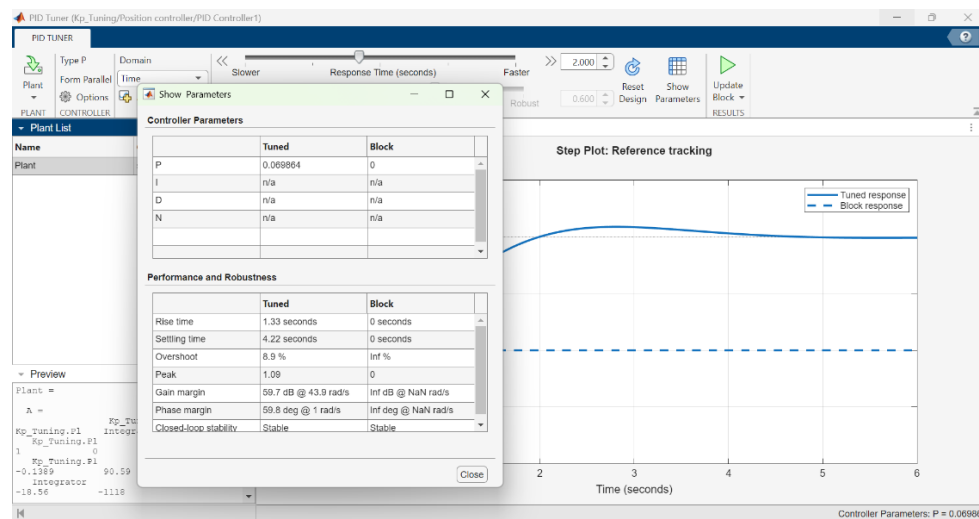
ภาพที่ 32 Tuning Controller System Design



ภาพที่ 33 Tuning Controller System Design



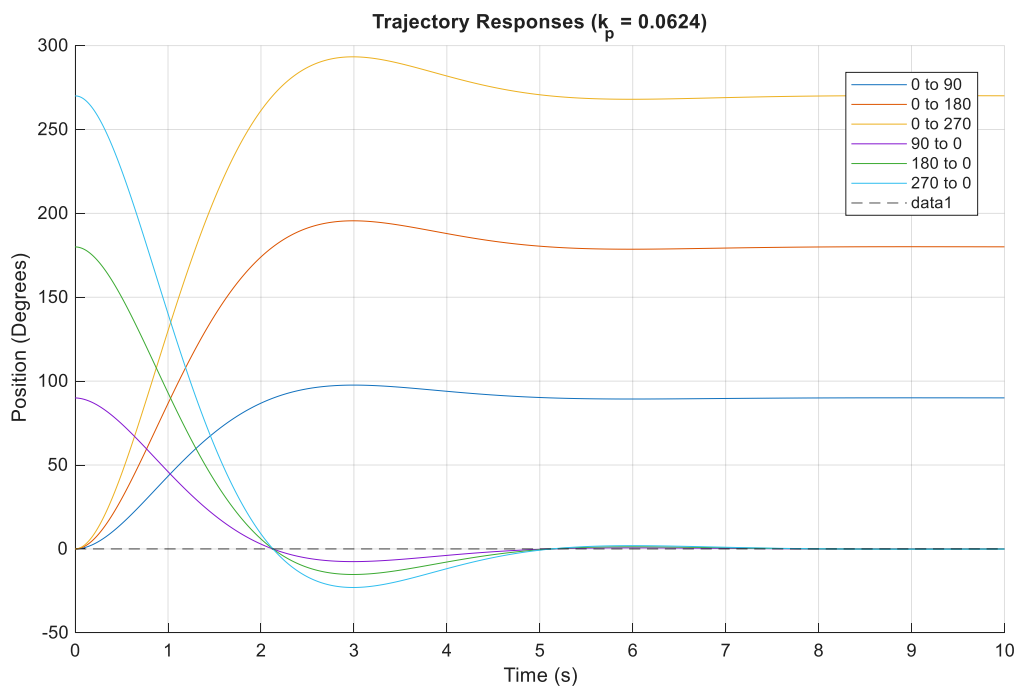
ภาพที่ 34 Tuning PID Controller Block แบบ Continuous



ภาพที่ 35 PID Controller Block แบบ Discrete

ตารางที่ 4 Validate ค่าที่ได้จากการคำนวณ Root Locus Controller Gain $K_p = 0.6235$

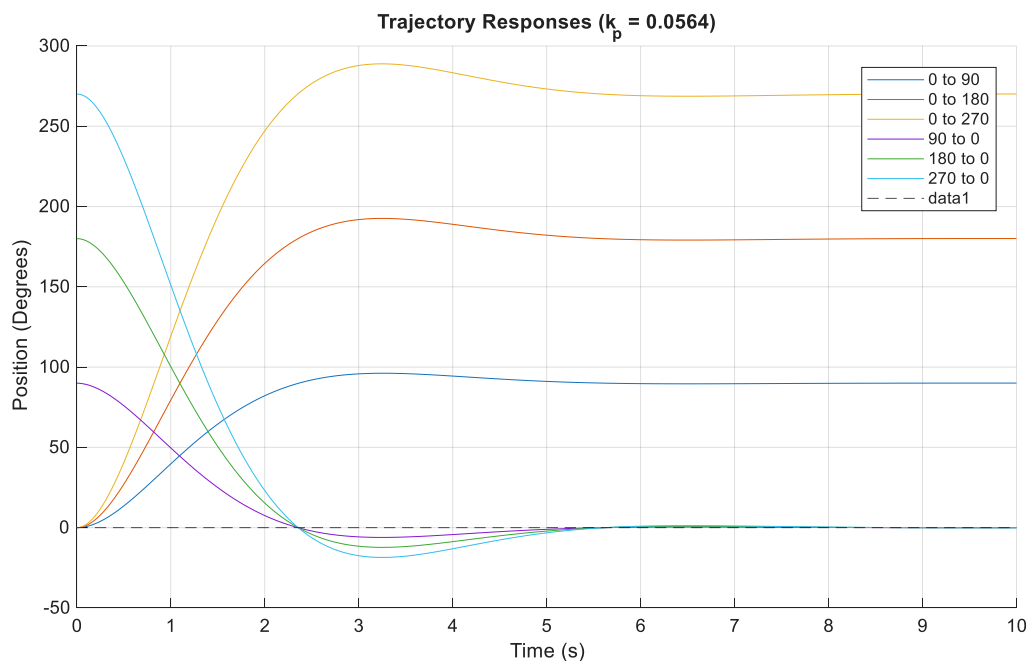
Trajectory (Initial – Setpoint) (Degree)	%Overshoot (%)	Peak Time (sec)
0 – 90	8.4896	2.9989
0 – 180	8.6493	2.9928
0 – 270	8.6341	2.9878
90 – 0	8.4088	2.9882
180 – 0	8.4700	2.9911
270 – 0	8.5173	2.9913



ภาพที่ 36 Position Response ของแต่ละองศาที่ $K_p = 0.6235$

ตารางที่ 5 Validate ค่าที่ได้จากการคำนวณ Control System Design $K_p = 0.056427$

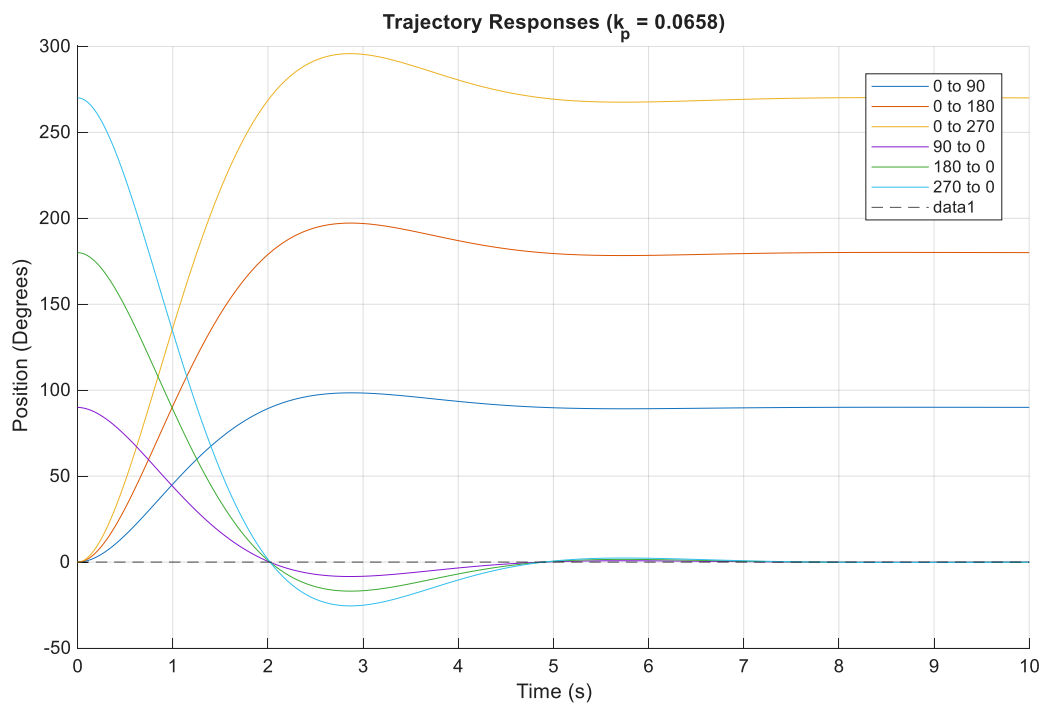
Trajectory (Initial – Setpoint) (Degree)	%Overshoot (%)	Peak Time (sec)
0 – 90	6.8490	3.2601
0 – 180	7.0008	3.2516
0 - 270	6.9875	3.2460
90 – 0	6.7719	3.2484
180 – 0	6.8279	3.2513
270 – 0	6.8721	3.2507



ภาพที่ 37 Position Response ของแต่ละองศาที่ $K_p = 0.056427$

ตารางที่ 6 Validate ค่าที่ได้จาก PID Controller Block แบบ Continuous $K_p = 0.065754$

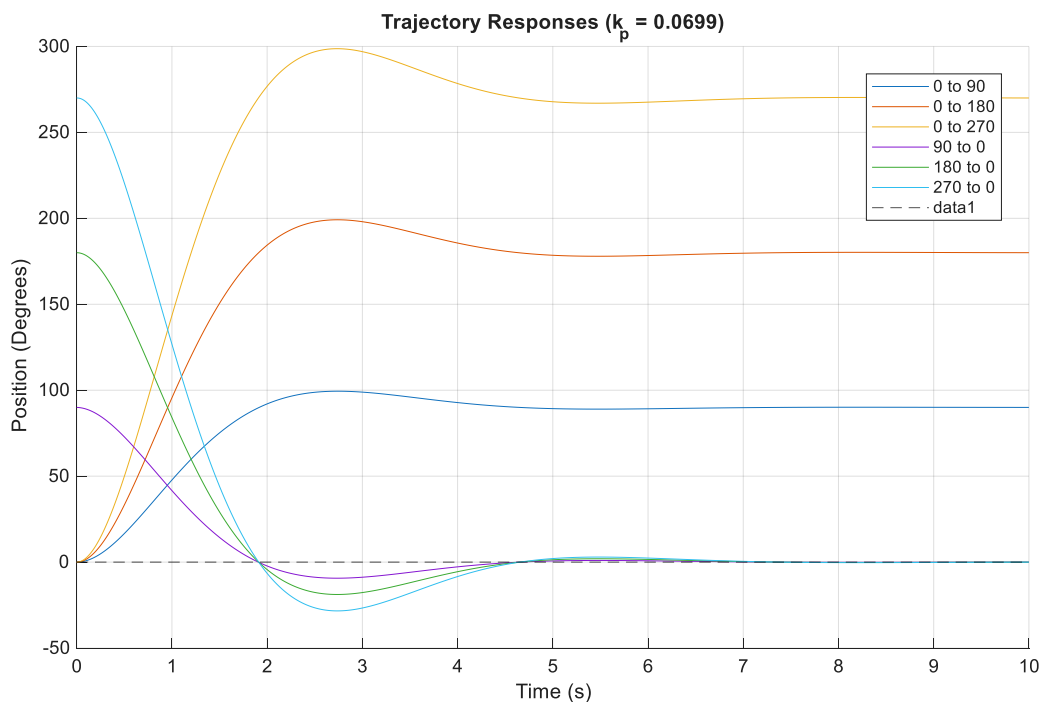
Trajectory (Initial – Setpoint) (Degree)	%Overshoot (%)	Peak Time (sec)
0 – 90	9.4041	2.8748
0 – 180	9.5673	2.8694
0 - 270	9.5507	2.8650
90 – 0	9.3214	2.8641
180 – 0	9.3851	2.8673
270 – 0	9.4337	2.8677



ภาพที่ 38 Position Response ของแต่ละองศาที่ $K_p = 0.065754$

ตารางที่ 7 Validate ค่าที่ได้จาก PID Controller Block แบบ Discrete $K_p = 0.069864$

Trajectory (Initial – Setpoint) (Degree)	%Overshoot (%)	Peak Time (sec)
0 – 90	10.4775	2.7433
0 – 180	10.6441	2.7389
0 - 270	10.6257	2.7345
90 – 0	10.3927	2.7332
180 – 0	10.4590	2.7366
270 – 0	10.5089	2.7370



ภาพที่ 39 Position Response ของแต่ละองศาที่ $K_p = 0.069864$

สรุปผลการทดลอง

จากการทดลองพบว่าการ Tune PID จาก PID Controller Block ทั้งแบบ Discrete และ Continuous นั้นให้ค่าที่ไม่ตรงกับค่าที่ Tune ได้จริงโดยสังเกตจาก ภาพที่ 29 ที่แจ้งไว้ว่าสามารถหาค่า %Overshoot ได้ที่ 8.7% ซึ่งหลังจากนำค่า K_p มา Validate พบว่า %Overshoot จริงๆอยู่ที่ 10.39% - 10.69% ซึ่งในลักษณะคล้ายกัน Tune ด้วย PID Controller แบบ Continuous Model แต่ในทางกลับกันการ Tune ด้วย Control System Design และการคำนวณด้วย Root locus Gain นั้นสามารถให้ค่าที่เมื่อนำไป Validate แล้วให้ค่าที่ใกล้เคียงกับ Requirement ของการทดลองที่ 2

ในการปรับมุมแต่ละองศาพบว่าการเคลื่อนที่จาก 0 องศา ถึง องศาใดๆนั้นไม่มีความแตกต่างอย่างนัยยะสำคัญแต่หากสังเกตจากผลกา Validate ค่า K_p แต่ละค่าจะพบว่ามุม 0 องศา ถึง 180 องศา นั้นให้ค่า %Overshoot มากที่สุดเสมอ

อภิปรายผล

การใช้ PID Controller ใช้ Tune ค่า PID พบว่า PID Controller Block จะดึง Plant จาก Block ที่ผ่าน Loop จนกระทั่ง Feedback กลับมาที่ PID Controller Block ซึ่งมีความเป็นไปได้ Block PID นั้นไม่ได้รับ Plant

ของระบบจริงๆ โดยทางผู้จัดทำได้ตั้งสมมติฐานว่า PID Controller Block นั้นเห็น Plant ของระบบถึงแค่ Output เป็น Torque ของระบบเท่านั้นซึ่งโดยคาดการณ์มาจาก เวลาจน P Control จะเห็นเป็น First Order System เท่านั้นซึ่งในระบบ Motor จริงๆนั้นเป็นระบบ Third Order จากสมการ Transfer Function หมายความว่าหากต้องการ Tune ค่า PID จาก PID Controller Block ควรสร้าง Plant ของระบบแยกออกจาก ฝายในการ Run Simulation เพื่อที่ระบบจะได้รับ Plant ของ System จริงๆ

และการที่ Trajectory จาก 0 องศาไปถึง 180 องศา นั้นให้ค่า %Overshoot สูงที่สุดเนื่องจากเมื่อระบบ เคลื่อนที่ไปถึงตำแหน่ง Setpoint Controller จะต้องจ่าย Voltage เพื่อหักล้างกับผลของ Gravity ทั้งก่อนถึง Setpoint และตำแหน่งหลัง Set Point ที่ Disturbance Feedforward หักล้างไม่หมดซึ่งสื่อได้ว่าที่ตำแหน่ง 180 องศาเป็นตำแหน่งที่รับผลกระทบจาก Gravity มากที่สุด

อ้างอิง

Comparative Analysis of PID Tuning Methods for DC Motor Speed Control [IEEE Xplore Full-Text PDF](#): (2025)

Particle Swarm Optimization and Genetic Algorithms for PID Controller Tuning [IEEE Xplore Full-Text PDF](#): (2023)

การทดลองที่ 2.3 วิเคราะห์ผลของ Gravity Compensation ที่ส่งผลต่อระบบ

จุดประสงค์

1. เพื่อสังเกตผลของ Gravity Compensation ที่ส่งผลต่อการควบคุม Pendulum
2. เพื่อสังเกตความแตกต่างระหว่างผลของ Simulation และการคำนวณจริง โดยจะดูจากความถูกต้อง

สมมติฐาน

- การใส่ Gravity Compensation เข้าไปในระบบจะทำให้ระบบสามารถทำงานได้ เนื่องจากหากไม่มี Gravity Compensation ในการหักล้างกับแรงโน้มถ่วงจะทำให้ Plant เป็นแบบ Non-linear และไม่สามารถ Model ได้และจะทำให้ Motor ไม่สามารถทำงานไปถึง Set Point ที่ต้องการได้ด้วย P control
- คาดว่าผลจากการทดลองจริงจะมีความแตกต่างจากสิ่งที่ได้จากการทดลองเล็กน้อยเนื่องจาก feedforward อาจไม่สามารถทำ gravity compensation ได้อย่างสมบูรณ์ ในขณะที่การคำนวณเราจะตั้ง Assumption ไว้ว่า Feedforward สามารถหักล้างกับ แรงโน้มถ่วงได้ทั้งหมด

ตัวแปร

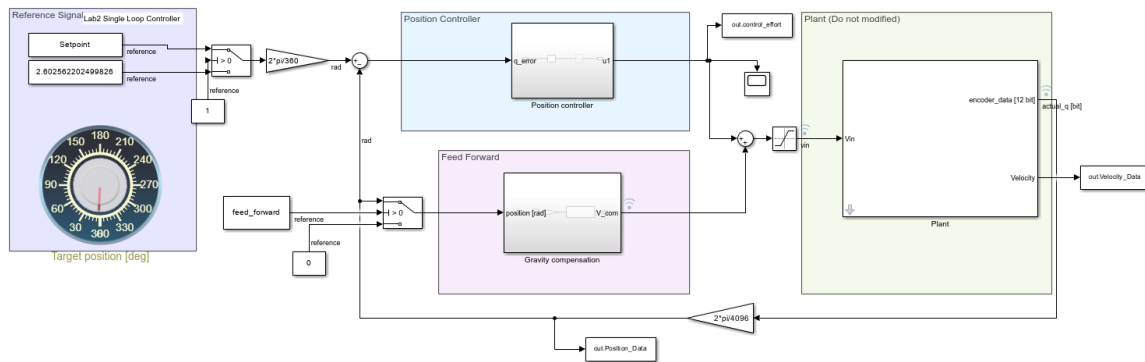
1. ตัวแปรต้น:
 - 1.1. การเปิดและปิด Feedforward
 - 1.2. การได้มาของค่า Peak time และ %Overshoot ระหว่างการคำนวณและ Simulation
2. ตัวแปรตาม:
 - 2.1. ผลของ Peak time และ %Overshoot ที่ได้จากการคำนวณและที่ได้จาก Simulation
3. ตัวแปรควบคุม:
 - 3.1. ระยะเวลาการรันโปรแกรมของแต่ละชุดการทดลอง
 - 3.2. Plant ของระบบ

เอกสารและงานวิจัยที่เกี่ยวข้อง

การจูน PID ของชุดการทดลองจะมีมวลและจุดศูนย์ถ่วงที่เปลี่ยนแปลงตามตำแหน่งเชิงมุมซึ่ง Gravity หรือ แรงโน้มถ่วงจะกลายเป็นแรงบิดคงที่ที่มีผลกับระบบทุกครั้งที่เราเริ่มการทดลองจะส่งผลให้เกิด Stead-State Error ให้กับ PID เนื่องด้วยระบบที่ PID กำลังควบคุมอยู่นั้นจะมีส่วนหนึ่งที่ใช้ต้านแรงโน้มถ่วง ดังนั้น Gravity compensation จึงถูกนำมาใช้ในการควบคุมแบบ Feedforward Control เพื่อหักล้างแรงบิดที่เกิดขึ้นจากแรงโน้มถ่วงทำให้ระบบหลักที่ควบคุมสามารถโฟกัสไปกับ Dynamics เดิมของระบบได้ ซึ่งจากงานวิจัยอ้างอิงได้ระบุไว้ว่า Gravity compensation จะทำให้พฤติกรรมของระบบใกล้เคียงกับ Linear System มากขึ้น ทำให้การจูน K_p ใน Simulation จะมีความแม่นยำ

ขั้นตอนการดำเนินงาน

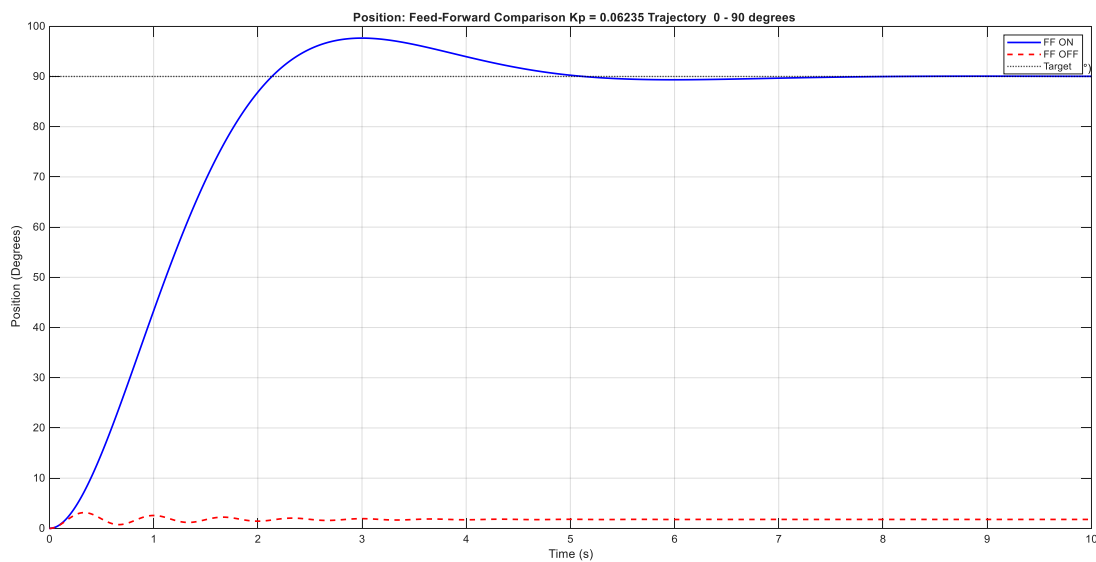
1. กำหนดค่า K_p ที่ใช้ในการทดลองโดยใช้วิธีการ System Design Toolbox $K_p = 0.06235$
2. เขียนโปรแกรมจำลองระบบใน Simulink และเขียนโปรแกรม MATLAB สำหรับการเก็บค่าในการทดลอง โดยกำหนดให้ Input เป็น K_p , Initial Point, Set Point โดยจะบันทึกค่าทั้งหมด 2 รอบและนำมา Plot เทียบกันคือ 1. แบบมี Disturbance Feed Forward 2. แบบไม่มี Disturbance Feedforward



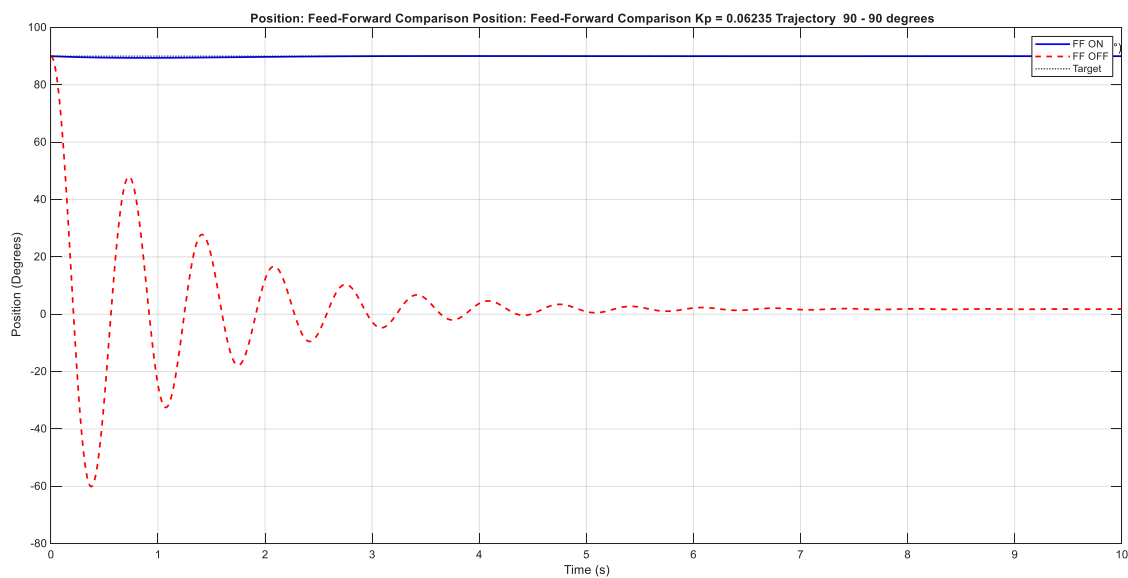
ภาพที่ 40 ภาพแสดง Simulink ที่ใช้ทำการทดลองที่ 2.3

3. แบ่งการทดลองออกเป็น 3 ชุดได้แก่
 - a. กำหนดให้ Initial Point เป็น 0 องศา และ Set point เป็น 90 องศา
 - b. กำหนดให้ Initial Point เป็น 90 องศา และ Set point เป็น 90 องศา
 - c. กำหนดให้ Initial Point เป็น 90 องศา และ Set point เป็น 0 องศา
4. เปรียบเทียบผลของการ run simulation แต่ละค่าของ gravity compensation และไม่มี gravity compensation พร้อมกับชี้ตำแหน่งของ Peak Time และระบุ %Overshoot ของระบบ
5. นำผลที่ได้จากการเปรียบเทียบมาพิจารณาเพื่อสรุปถึงหน้าที่และความจำเป็นในการมี gravity compensation

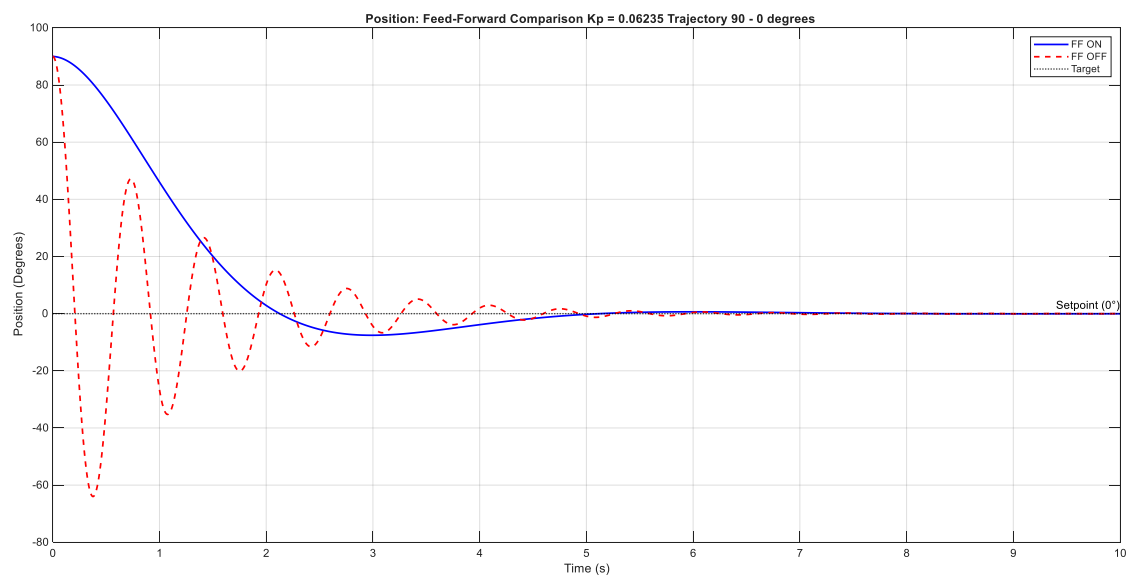
ผลการทดลอง



ภาพที่ 41 ผลการเปรียบเทียบ System Response Trajectory จาก 0 องศา ถึง 90 องศา ระหว่างมี Feed Forward และไม่มี Feedforward



ภาพที่ 42 ผลการเปรียบเทียบ System Response Trajectory จาก 90 องศา ถึง 90 องศา ระหว่างมี Feed Forward และไม่มี Feedforward



ภาพที่ 43 ผลการเปรียบเทียบ System Response Trajectory จาก 90 องศา ถึง 0 องศา ระหว่างมี Feed Forward และไม่มี Feedforward

สรุปผลการทดลอง

จากการทดลองแสดงให้เห็นว่าหากระบบไม่มี Disturbance Feedforward ระบบจะไม่สามารถเข้าสู่ตำแหน่ง Setpoint โดยเราจะเห็นว่า Controller ยังพยายามจะเข้าสู่ Setpoint ได้จากภาพที่ 40 ที่ Position ของ Motor นั้นไม่ได้หยุดที่ 0 องศาพอดี

อภิปรายผล

จากการทดลองนี้แสดงให้เห็นว่าหาก Model นั้นมี Disturbance มากเกินไป จะทำให้ Controller ไม่สามารถทำงานได้ตามที่เรากำหนดไว้

อ้างอิง

A design approach for gravity compensators using planar four-bar mechanisms and a linear spring [A design approach for gravity compensators using planar four-bar mechanisms and a linear spring](#) (2022)

Modeling and Performance Evaluation of the Dynamic Behavior of Gravity Energy Storage with a Wire Rope Hoisting System [Modeling and Performance Evaluation of the Dynamic Behavior of Gravity Energy Storage with a Wire Rope Hoisting System](#) (2020)

การทดลองที่ 2.4 สังเกตผลของการปรับค่า K_p ที่ส่งผลต่อ Overshoot และ Peak Time ของระบบ

จุดประสงค์

1. เพื่อสังเกตแนวโน้มการปรับค่า K_p ที่ส่งผลต่อระบบ

สมมติฐาน

การปรับค่า K_p ขึ้นส่งผลให้ค่า Overshoot มีค่ามากยิ่งขึ้น และทำให้ Peak Time มีค่าลดลง ในทางกลับกันการปรับค่า K_p ลงจะทำให้ค่า Over Shoot มีค่าน้อยลง และทำให้ Peak Time มีค่ามากยิ่งขึ้น

ตัวแปร

1. ตัวแปรต้น:
 - 1.1. การปรับค่า K_p ของระบบ
2. ตัวแปรตาม:
 - 2.1. ผลของ Peak time และ %Overshoot ของแต่ละค่า K_p

3. ตัวแปรควบคุม:

- 3.1. Plant ของระบบ
- 3.2. ระยะเวลาในการรัน โปรแกรม
- 3.3. ตำแหน่ง Inertial Point และตำแหน่ง Set Point

เอกสารและงานวิจัยที่เกี่ยวข้อง

ในการเปลี่ยนแปลงของค่า K_p ส่งผลทำให้ Transient Response หรือการตอบสนองชั่วขณะของระบบ ซึ่งจะส่งผลกับ Overshoot (M_p) กับ Peak Time (T_p) ที่บ่งบอกถึงความเร็วและความเสถียรของระบบ ซึ่งงานวิจัยที่อ้างอิงระบุไว้ว่า การเพิ่มค่า K_p จะทำให้ความถี่ธรรมชาติ (ω_n) เพิ่มมากขึ้นจะทำให้ระบบตอบสนองเข้าสู่เป้าหมายได้รวดเร็วขึ้น ในขณะเดียวกันจะสามารถลดค่า Damping Ratio (ζ) ของระบบทำให้เกิดการหน่วงเวลาและสัญญาณที่กระเพื่อมสูงขึ้น ตาม Secon order System ที่ K_p สัมพันธ์กับ Peak Time จาก $T_p = \frac{\pi}{\omega_n \sqrt{1-\zeta^2}}$ แต่หาก Overshoot เพิ่มขึ้นตามความสัมพันธ์ดังกล่าว

$$M_p = e^{-\left(\frac{\zeta\pi}{\sqrt{1-\zeta^2}}\right)}$$

ในการทดลองผลกระทบผ่าน Simulation จะช่วยให้เห็นความสัมพันธ์ก่อนการนำไปใช้งานจริง ซึ่งงานวิจัยอ้างอิงสรุปว่าการจูนค่า K_p ที่ดีที่สุด ระหว่างความเร็วในการเข้าสู่เป้าหมายและการควบคุมไม่ให้แรงเหวี่ยงหรือการสั่น

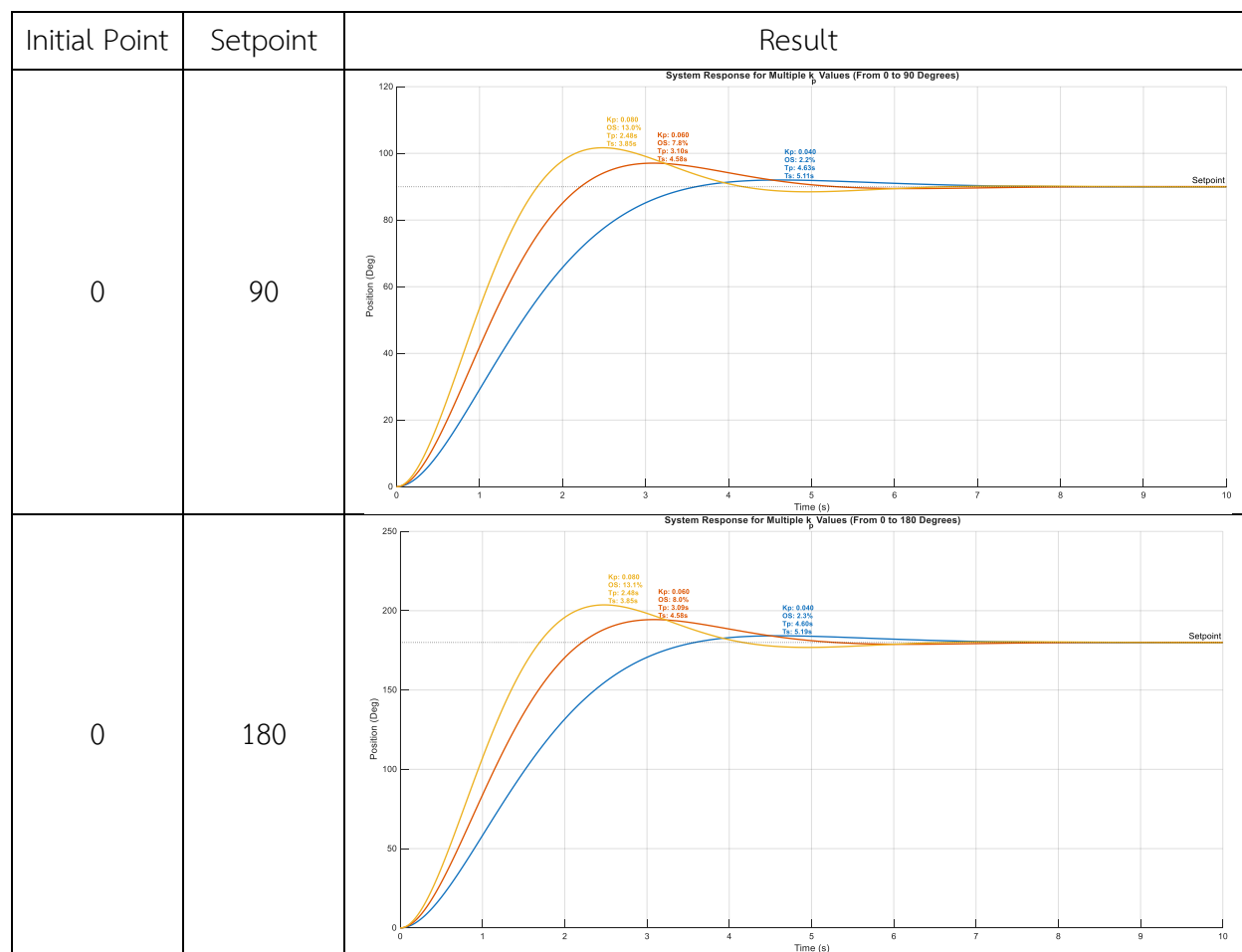
ขั้นตอนการดำเนินงาน

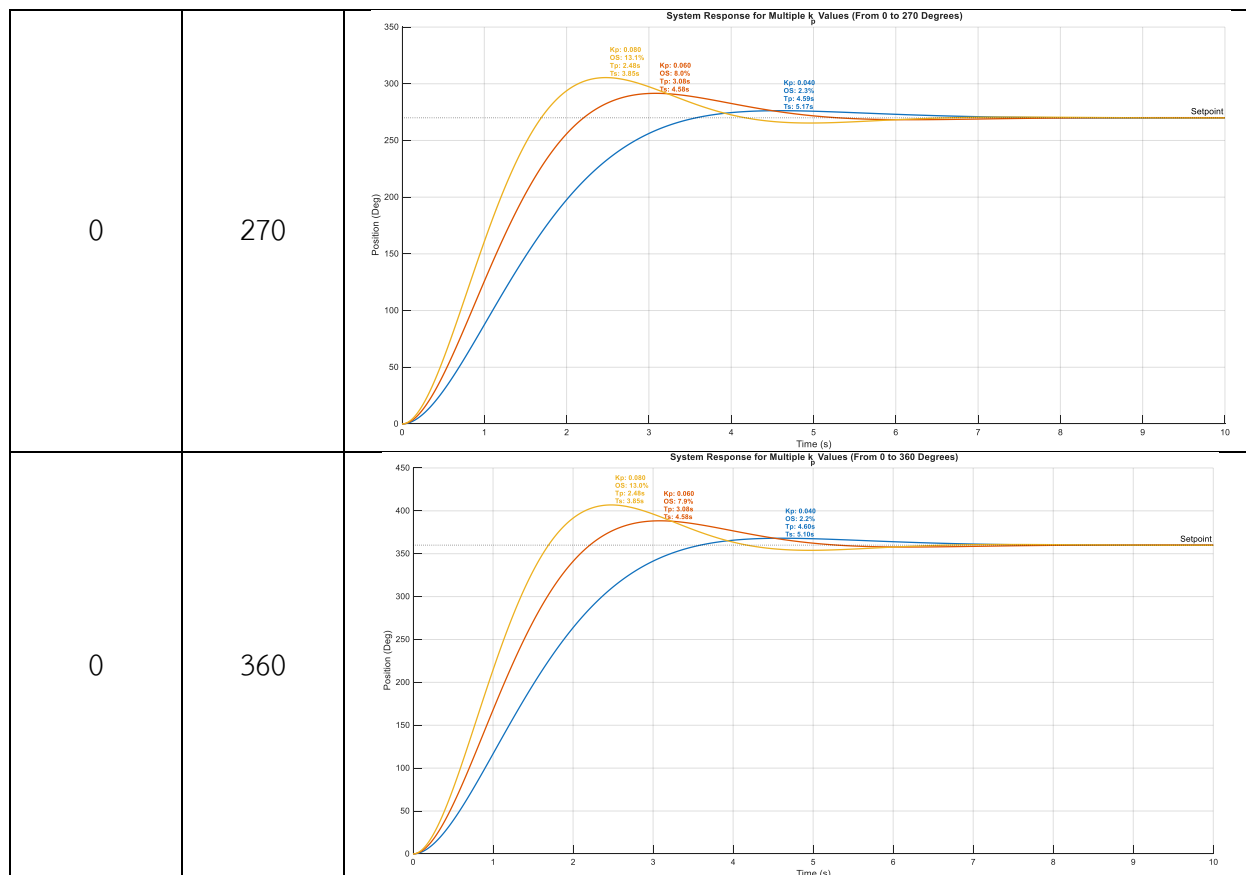
1. กำหนดชุดการทดลองย่อยโดยกำหนดจาก Set Point และ Initial Point ดังนี้
 - 1.1. Initial Point 0 องศา Setpoint 90 องศา
 - 1.2. Initial Point 0 องศา Setpoint 180 องศา
 - 1.3. Initial Point 0 องศา Set Point 270 องศา
 - 1.4. Initial Point 0 องศา Set Point 360 องศา
2. เขียนโปรแกรมจำลองระบบใน Simulink และเขียนโปรแกรม MATLAB สำหรับการเก็บค่าในการทดลอง โดยกำหนดให้ Input เป็น K_p , Initial Point, Set Point โดยจะบันทึกค่าทั้งหมด 3 รอบและนำมา Plot เทียบกันคือ K_p

- เปรียบเทียบผลของการ run simulation แต่ละค่า K_p พร้อมทั้งชี้ตำแหน่งของ Peak Time และระบบ %Overshoot ของระบบ
- นำค่าที่ได้จากการเปรียบเทียบผลของการ run simulation และจากการคำนวณมาเปรียบเทียบถึงความแตกต่างของ Peak Time และ %Overshoot จากนั้นบันทึกผล

ผลการทดลอง

ตารางที่ 8 ตารางบันทึกผลการเปรียบเทียบค่า K_p ที่แต่ละ Trajectory





สรุปผลการทดลอง

จากการทดลองแสดงให้เห็นว่าเมื่อเพิ่มค่า K_p จะส่งผลให้ %Overshoot เพิ่มขึ้น และทำให้ Peak time ลดลง ในทางกลับกันเมื่อลดค่า K_p จะส่งผลให้ %Overshoot มีค่าลดลง , Peak time มีค่าเพิ่มขึ้น

อภิปรายผล

หากวิเคราะห์ใน Root Locus จะพบว่า แนวโน้มของการเปลี่ยนแปลงค่า K_p นั้นสอดคล้องกับค่าที่ได้จากผลการทดลอง หรือก็คือหากทำการเลื่อนค่า K_p ให้มากขึ้นจะทำให้ pole ของระบบเลื่อนขึ้น (Imaginary part increase) และ pole ของระบบเลื่อนไปทางขวา (real part decrease) ซึ่งสื่อได้ว่าระบบจะส่ายแรงขึ้น (Overshoot สูงขึ้นและ Peak time น้อยลง) และระบบจะเสถียรเร็วขึ้น (Settling Time ลดลง)

หากดูจากภาพรวมตลอดการทดลองที่ 2 การออกแบบ K_p Control สามารถอนุมานได้ว่าในระยะทางสั้นๆจะไม่ส่งผลต่อค่า Stability ของระบบอย่างมีนัยยะสำคัญ ทั้งนี้จากการสังเกตการเก็บข้อมูลที่ผ่านมาจะพบว่า

การตั้ง Initial Point ไว้ที่ 0 องศา และ Set point ไว้ที่ 180 องศา จะมีค่า %Overshoot มากที่สุดเสมอดังนั้นในการทดลองที่ 3 ของทุกการทดลองจะกำหนดให้ Initial Point ที่ 0 องศา และ Set Point ที่ 180 องศา

อ้างอิง

Optimized PI/PD/PID Controller Design for DC Motor Speed Control: A Comparative Study of Random Forest-Based Tuning & Classical Methods [IEEE Xplore Full-Text PDF](#): (2025)

Sensitivity Analysis of Proportional Gain in Second-Order Electromechanical Systems [SECOND-ORDER SYSTEMS](#) (2025)

3. การออกแบบ PID Controller

การทดลองที่ 3.1 การทดลองความแตกต่างของวิธีการ ออกแบบ Controller Design

จุดประสงค์

1. เพื่อหาวิธีการ Design Controller ที่ให้ผลตรงกับค่าจริงมากที่สุด
2. เพื่อสังเกตพฤติกรรมของกราฟ Output เมื่อใส่ Controller แต่ละรูปแบบ

สมมติฐาน

คาดว่า การ Tuning ด้วย Control System Design และการคำนวณ Root Locus Controller Gain นั้นสามารถให้

ตัวแปร

1. ตัวแปรต้น:
 - 1.1. วิธีการได้มาของค่า Controller Gain
2. ตัวแปรตาม:
 - 2.1. ผลของ Controller Gain ที่ได้จากแต่ละเครื่องมือ
3. ตัวแปรควบคุม:
 - 3.1. Plant

เอกสารและงานวิจัยที่เกี่ยวข้อง

การออกแบบ PID Controller Block สามารถทำได้หลายวิธี โดยแต่ละวิธีจะให้ค่าพารามิเตอร์ที่ต่างกันและส่งผลกระทบต่อสมรรถนะของระบบ

สมการ PID Controller:

$$u(t) = K_c \left[e(t) + \frac{1}{T_i} \int e(t) dt + T_d \frac{de(t)}{dt} \right]$$

หรือในรูปแบบ Transfer Function:

$$G_c(s) = K_c \left(1 + \frac{1}{T_i s} + T_d s \right)$$

โดยที่ $e(t) = SP - PV$ คือค่าความผิดพลาดระหว่าง Setpoint และ Process Variable

ขั้นตอนการดำเนินงาน

*ตัด PI ออกจากการทดลองนี้เนื่องจาก PI ไม่สามารถทำงานได้ตาม Requirement โดยสามารถดูพิสูจน์ได้ในภาคผนวก ข**

1. PD control

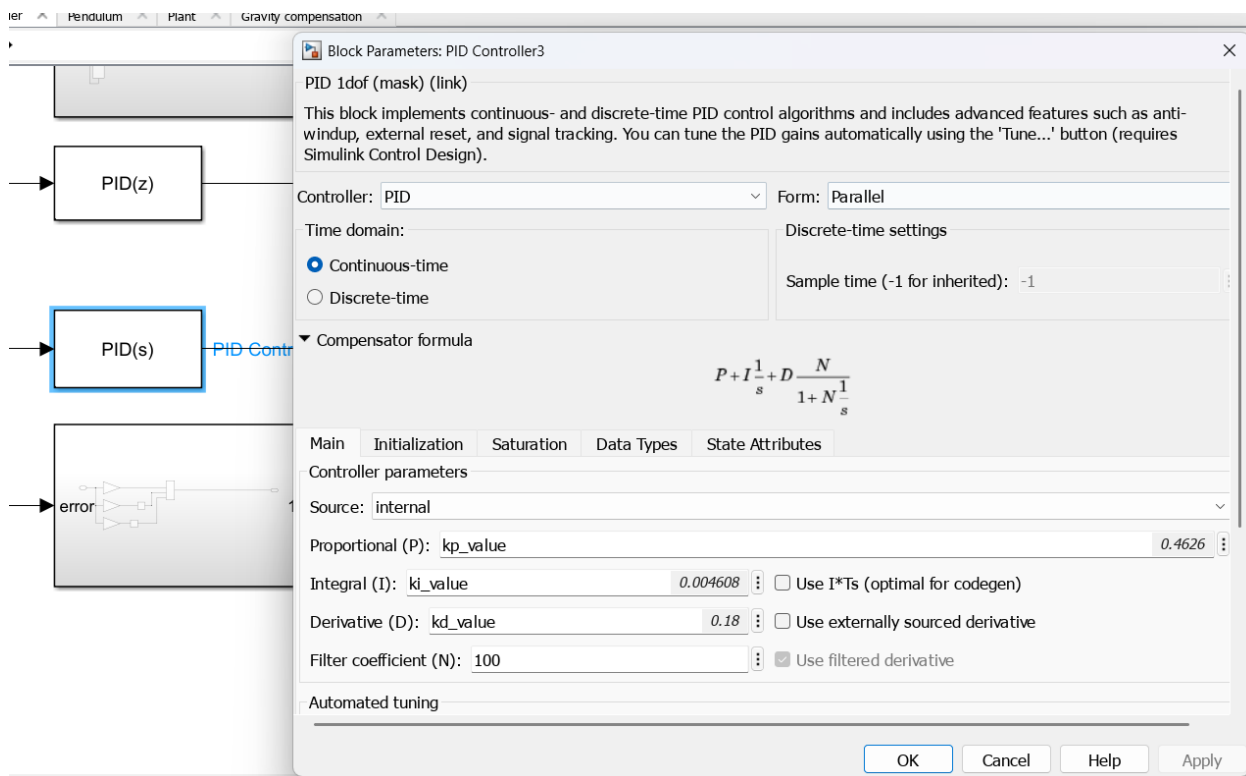
- 1.1. คำนวณค่า Controller Gain ด้วย Root Locus Controller Gain
- 1.2. หาค่า Controller Gain ผ่าน Controller System Design Toolbox
- 1.3. หาค่า Controller Gain ผ่าน PID Controller Block
 - 1.3.1. หาค่า Controller Gain โดยการใช้ PID Controller Block แบบ Continuous
 - 1.3.2. หาค่า Controller Gain โดยการใช้ PID Controller Block แบบ Discrete

2. PID control

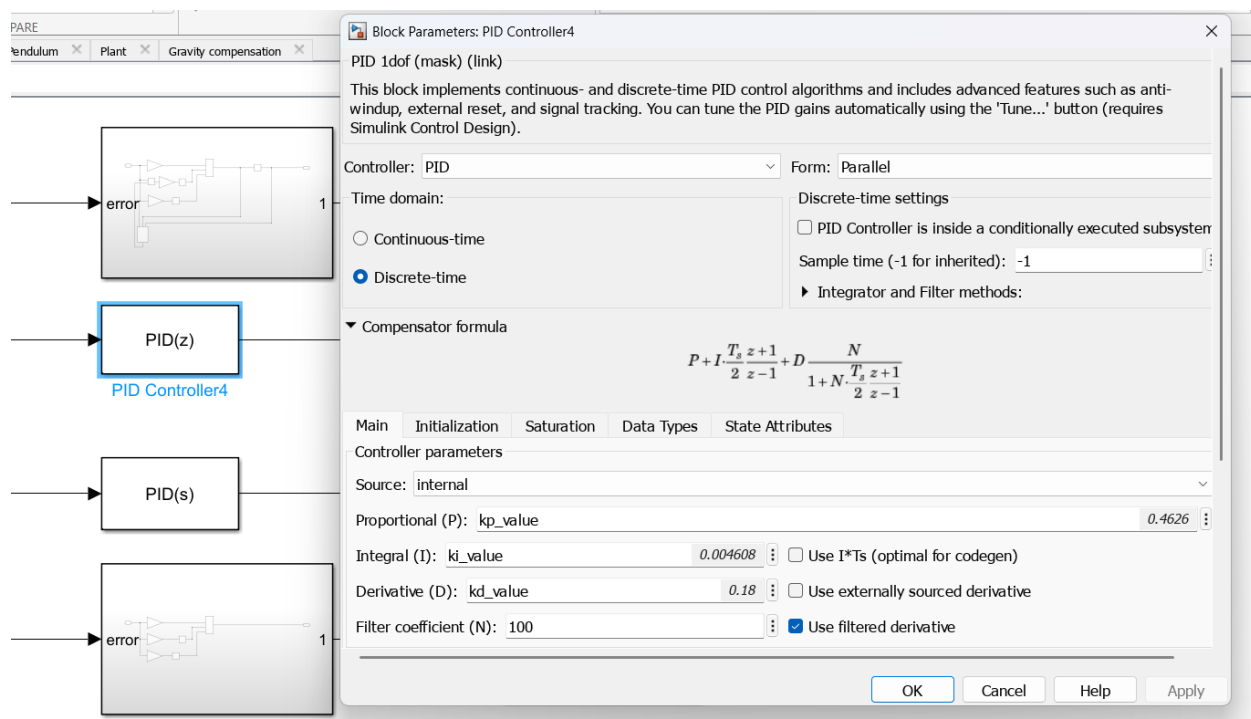
- 2.1. คำนวณค่า Controller Gain ด้วย Root Locus Controller Gain
- 2.2. หาค่า Controller Gain ผ่าน Controller System Design Toolbox
- 2.3. หาค่า Controller Gain ผ่าน PID Controller Block
 - 2.3.1. หาค่า Controller Gain โดยการใช้ PID Controller Block แบบ Continuous
 - 2.3.2. หาค่า Controller Gain โดยการใช้ PID Controller Block แบบ Discrete

3. นำค่า Controller Gain ที่ได้จากการทดลองข้อที่ 1 - 4 มา Validate ผ่านวิธีการต่อไปนี้

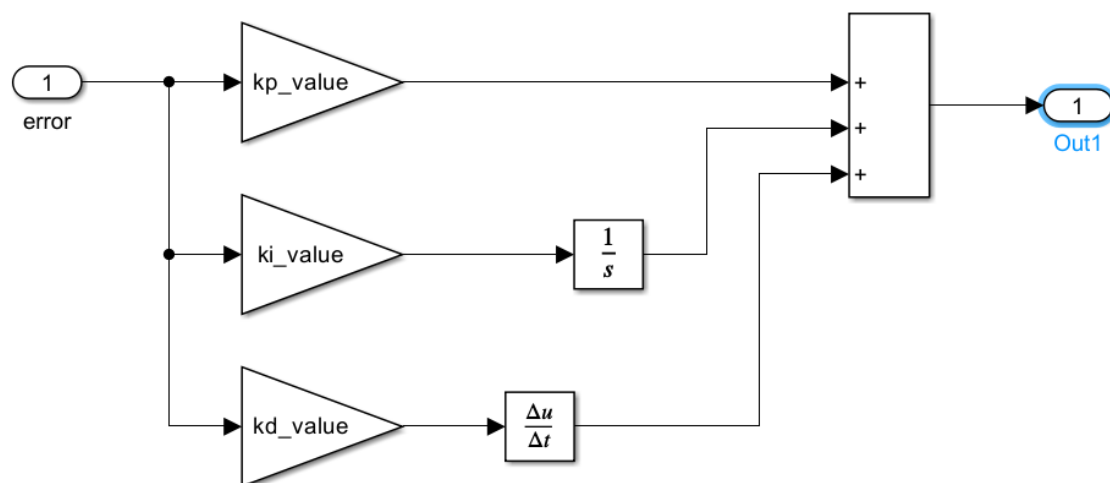
- 3.1. นำมารันผ่าน PID Controller Block แบบ Continuous – time



3.2. นำมารันผ่าน PID Controller Block แบบ Discrete - time



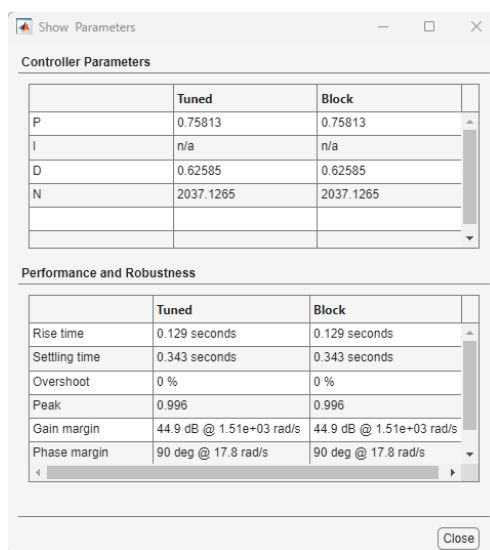
3.3. นำมารันผ่าน Constant Block



ผลการทดลอง วิธีการแสดงผล

ตารางที่ 9 ผลการ Tuning PD ด้วยวิธีการต่างๆ

วิธีการ	คำนวณ Root Locus Controller Gain	Controller System Design	PID Controller Block แบบ Continuous	PID Controller Block แบบ Discreate
ค่า K_p ที่ได้	0.18048	1.9642	0.75813	1.0393
ค่า K_d ที่ได้	0.0705	1.8856	0.62585	0.78142



ภาพที่ 44 ตารางแสดง Parameters ที่ได้จากการใช้ PID Controller Block แบบ Continue

Controller Parameters	
	Tuned
P	1.0393
I	n/a
D	0.78142
N	185.5156
Performance and Robustness	
	Tuned
Rise time	0.09 seconds
Settling time	0.19 seconds
Overshoot	0 %
Peak	1
Gain margin	18 dB @ 222 rad/s
Phase margin	82.8 deg @ 22.3 rad/s
Closed-loop stability	Stable

ภาพที่ 45 ตารางแสดง Parameters ที่ได้จากการใช้ PID Controller Block แบบ Discrete

Compensator Editor	
Compensator	
C	$= 1.9642 \times \frac{(1 + 0.96s)}{s}$
Pole-Zero	
Dynamics	
Type	Location
Real Zero	-1.05
Damping	Frequency
1	1.05
Right-click to add or delete poles/zeros	
Help	Cancel

ภาพที่ 46 ตารางแสดง Parameters ที่ได้จากการใช้ Controller System Design

ตารางที่ 10 ผลการ Tuning PID ด้วยวิธีการต่างๆ

วิธีการ	คำนวณ Root Locus Controller Gain	Controller System Design	PID Controller Block แบบ Continuous	PID Controller Block แบบ Discrete
ค่า K_p ที่ได้	0.4608	0.4626	3.6237	0.9611
ค่า K_i ที่ได้	0.000185	0.004608	1.3254	0.17965
ค่า K_d ที่ได้	0.18	0.18	2.2012	1.0258

Controller Parameters		
	Tuned	Block
P	3.6237	103
I	1.3254	1
D	2.2012	4.8
N	7165.0657	1429

Performance and Robustness		
	Tuned	Block
Rise time	0.0327 seconds	0.00942 seconds
Settling time	0.059 seconds	0.099 seconds
Overshoot	0 %	10 %
Peak	1	1.1
Gain margin	42.4 dB @ 2.83e+03 rad/s	25 dB @ 1.24e+03 rad/s
Phase margin	86.3 deg @ 62.7 rad/s	69.4 deg @ 139 rad/s
Closed-loop stability	Stable	Stable

ภาพที่ 47 ตารางแสดง Parameters ที่ได้จากการใช้ PID Controller Block แบบ Continue

Controller Parameters	
	Tuned
P	0.9611
I	0.17965
D	1.0258
N	188.8132

Performance and Robustness	
	Tuned
Rise time	0.07 seconds
Settling time	0.24 seconds
Overshoot	0 %
Peak	1
Gain margin	15.7 dB @ 226 rad/s
Phase margin	81 deg @ 29.3 rad/s
Closed-loop stability	Stable

ภาพที่ 48 ตารางแสดง Parameters ที่ได้จากการใช้ PID Controller Block แบบ Discrete

Compensator Editor

Compensator

C = 0.16 $\times \frac{(1 + 1s + 0.2s)(1 + 0.39s)}{s}$

Pole Zero

Dynamics

Type	Location	Damping	Frequency
Real Zero	0.01	1	0.01
Integrator	0	-1	0
Real Zero	2.56	1	2.56

Right-click to add or delete poles/zeros

Help Cancel

ภาพที่ 49 ตารางแสดง Parameters ที่ได้จากการใช้ Controller System Design

ตารางที่ 11 ค่าต่างๆ ที่ได้จากการ Validate ค่า PD ที่ Tune ได้ โดยมี Initial = 0 และ Setpoint = 180 องศา

Tuning Method: Tuning Block Discreate	Root Locus Controller Gain $K_p = 0.18048$ $K_d = 0.0705$	Controller System Design $K_p = 1.9642$ $K_d = 1.8856$	PID Controller Block แบบ Continuous $K_p = 0.75813$ $K_d = 0.62585$	PID Controller Block แบบ Discreate $K_p = 1.0393$ $K_d = 0.78142$
%Overshoot (%)	2.6143	0	0	0
Settling Time	2.0572	3.7418	3.1924	2.907
Steady State Error	1.0533e-06	0.0061009	0.0012771	0.00037384
Requirement Mash	☒	☐	☐	☐

ตารางที่ 12 ค่าต่างๆ ที่ได้จากการ Validate ค่า PD ที่ Tune ได้ โดยมี Initial = 0 และ Setpoint = 180 องศา

Tuning Method: Tuning Block Continue	Root Locus Controller Gain $K_p = 0.18048$ $K_d = 0.0705$	Controller System Design $K_p = 1.9642$ $K_d = 1.8856$	PID Controller Block แบบ Continuous $K_p = 0.75813$ $K_d = 0.62585$	PID Controller Block แบบ Discreate $K_p = 1.0393$ $K_d = 0.78142$
%Overshoot (%)	2.6135	0	0	0
Settling Time	2.0571	3.7417	3.1923	2.9069
Steady State Error	1.0553e-06	0.0061004	0.0012771	0.00037382
Requirement Mash	☒	☐	☐	☐

ตารางที่ 13 ค่าต่างๆ ที่ได้จากการ Validate ค่า PD ที่ Tune ได้ โดยมี Initial = 0 และ Setpoint = 180 องศา

Tuning Method: Block Constant	Root Locus Controller Gain $K_p = 0.18048$ $K_d = 0.0705$	Controller System Design $K_p = 1.9642$ $K_d = 1.8856$	PID Controller Block แบบ Continuous $K_p = 0.75813$ $K_d = 0.62585$	PID Controller Block แบบ Discreate $K_p = 1.0393$ $K_d = 0.78142$
%Overshoot (%)	1.4249	0	0	0
Settling Time	1.6721	3.8168	3.369	3.0304
Steady State Error	3.6017e-06	0.0061697	0.0014276	0.00038782

Requirement Mash	☒	☐	☐	☐
------------------	-------------------------------------	--------------------------	--------------------------	--------------------------

ตารางที่ 14 ค่าต่างๆ ที่ได้จากการ Validate ค่า PID ที่ Tune ได้ โดยมี Initial = 0 และ Setpoint = 180 องศา

Tuning Method: Tuning Block Discreate	Root Locus Controller Gain $K_p = 0.4608$ $K_i = 0.000185$ $K_d = 0.18$	Controller System Design $K_p = 0.4626$ $K_i = 0.004608$ $K_d = 0.18$	PID Controller Block แบบ Continuous $K_p = 3.6237$ $K_i = 1.3254$ $K_d = 2.2012$	PID Controller Block แบบ Discreate $K_p = 0.9611$ $K_i = 0.17965$ $K_d = 1.0258$
%Overshoot (%)	0.020813	0.32233	11.938	10.397
Settling Time	1.1007	1.0676	6.9774	NaN
Steady State Error	0.022978	0.52248	0.69144	7.3894
Requirement Mash	☒	☒	☐	☐

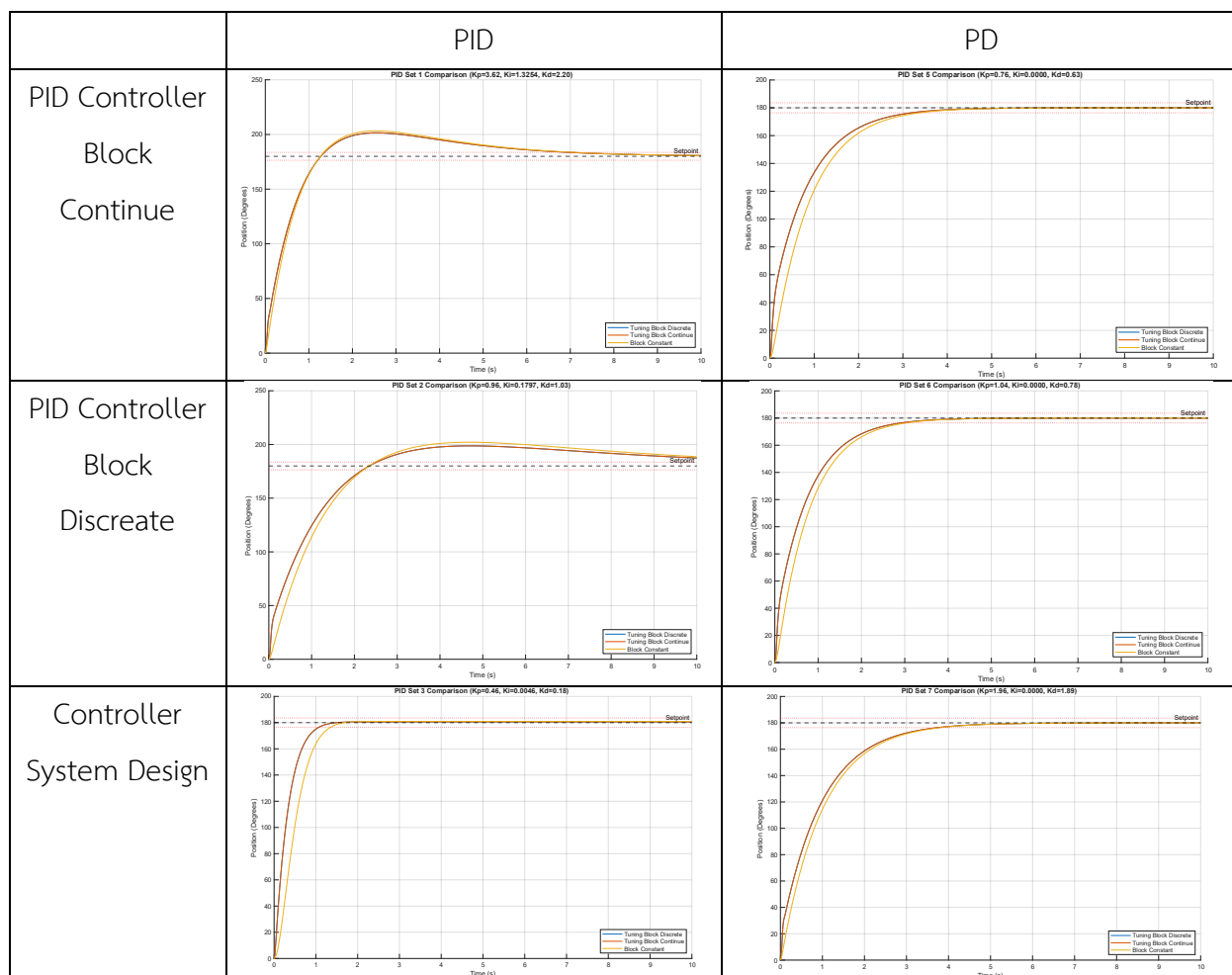
ตารางที่ 15 ค่าต่างๆ ที่ได้จากการ Validate ค่า PID ที่ Tune ได้ โดยมี Initial = 0 และ Setpoint = 180 องศา

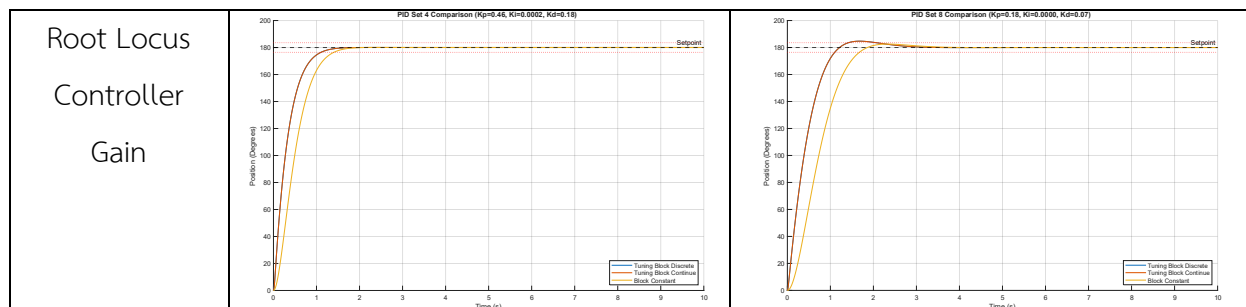
Tuning Method: Tuning Block Continue	Root Locus Controller Gain $K_p = 0.4608$ $K_i = 0.000185$ $K_d = 0.18$	Controller System Design $K_p = 0.4626$ $K_i = 0.004608$ $K_d = 0.18$	PID Controller Block แบบ Continuous $K_p = 3.6237$ $K_i = 1.3254$ $K_d = 2.2012$	PID Controller Block แบบ Discreate $K_p = 0.9611$ $K_i = 0.17965$ $K_d = 1.0258$
%Overshoot (%)	0.020855	0.32237	11.936	10.396
Settling Time	1.1008	1.0676	6.9772	NaN
Steady State Error	0.022977	0.52245	0.69144	7.3886
Requirement Mash	☒	☒	☐	☐

ตารางที่ 16 ค่าต่างๆ ที่ได้จากการ Validate ค่า PID ที่ Tune ได้ โดยมี Initial = 0 และ Setpoint = 180 องศา

Tuning Method: Block Constant	Root Locus Controller Gain $K_p = 0.4608$ $K_i = 0.000185$ $K_d = 0.18$	Controller System Design $K_p = 0.4626$ $K_i = 0.004608$ $K_d = 0.18$	PID Controller Block แบบ Continuous $K_p = 3.6237$ $K_i = 1.3254$ $K_d = 2.2012$	PID Controller Block แบบ Discrete $K_p = 0.9611$ $K_i = 0.17965$ $K_d = 1.0258$
%Overshoot (%)	0.04666	0.53426	12.928	12.271
Settling Time	1.3996	1.3475	7.0917	NaN
Steady State Error	0.037145	0.84517	0.69144	8.658
Requirement Mash	☑	☑	☒	☒

ตารางที่ 17 กราฟแสดงผลการ Validate ค่า PD และ PID ที่ได้จากการ Tune ด้วยวิธีต่างๆ





สรุปผลการทดลอง

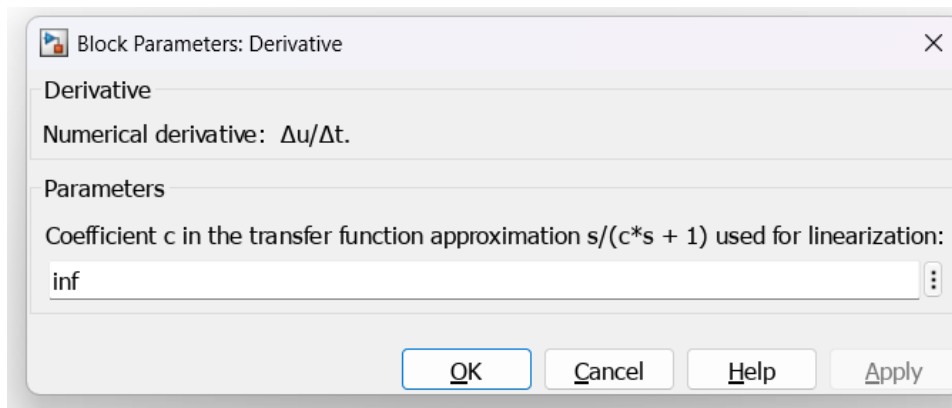
จากการทดลองแสดงให้เห็นว่าค่า PID จากที่ Tune ได้ในแต่ละวิธีนั้นให้ผลที่แตกต่างกันอย่างชัดเจน ทั้งนี้ค่าที่สามารถนำ Validate แล้วสามารถทำงานได้จริงๆคือ ค่าที่ได้จากการคำนวณด้วย Root locus เท่านั้น และในส่วนของ PID ที่ Tune ด้วย Controller System Design นั้นสามารถ Tune ให้เข้า Requirement ได้นั้น เนื่องจาก ได้นำค่าจากการคำนวณมาเป็น Initial Guess ก่อนซึ่งส่งผลให้ค่าที่ Tune ได้สามารถทำงานได้ตาม Requirement

ในส่วนของผลการทำงานของ Controller แต่ละตัวจะพบว่าการใช้ PID Controller ให้ผลที่คล้ายกันทั้งแบบ Continuous Model และแบบ Discrete Model ทั้งนี้ Controller แบบ Block Constant นั้นให้ผล Position Response ที่แตกต่างอย่างชัดเจน โดยสังเกตจาก แนวโน้มของ Constant Block ไม่ลู่เข้ากับ PID Controller ของทุกๆค่า PID

อภิปรายผล

จากการทดลองแสดงให้เห็นว่าการคำนวณ Root Locus ด้วยมือก่อนนำไป Tune ผ่านโปรแกรมสามารถนำมาใช้อ้างอิงของค่า PID เบื้องต้นได้

ในส่วนของ Block Constant นั้นสาเหตุที่แตกต่างจาก PID Controller เป็นไปได้ว่าเกิดจากการที่ใช้ Block Derivative ซึ่งหากทำการดูเข้าไปใน Block จะเห็นว่า Block Derivative นั้นก็คือ Low Pass Filter ที่มี Cut-off Frequency ที่ อินฟินิตี้ ซึ่งส่งผลให้ Model ของระบบที่ออกมาจาก Block Constant ไม่เหมือนกับ Model ของ PID Controller Block



ภาพที่ 50 Derivative Block

ข้อเสนอแนะ

1. ควรเลือกใช้วิธี Root Locus หรือ Controller System Design ในการหาค่า Controller Gain เนื่องจากให้ค่าที่ใกล้เคียง Requirement มากกว่าการใช้ PID Controller Block
2. หากใช้ PID Controller Block ควรสร้าง Plant แยกออกมาต่างหาก เพื่อให้ Block รับรู้ Plant ของระบบได้อย่างถูกต้อง

อ้างอิง

Didactic plat form for DC motor speed and position control in Z-plane [main.pdf](#) (2021)

Comparative Study for DC Motor Speed Control Using PID Controller [Comparative Study for DC Motor Speed Control Using PID Controller](#) (2017)

การทดลองที่ 3.2 การทดลองวิเคราะห์ผลกระทบของ Integration Windup และการใส่ Block Saturation

จุดประสงค์

1. เพื่อวิเคราะห์ผลกระทบของ Integration Windup ที่ส่งผลต่อการควบคุมระบบ
2. เพื่อศึกษาวิธีการแก้ไข Integration Windup ของการสร้าง PID Controller แต่ละวิธี
3. เพื่อศึกษาผลกระทบของการใส่ Saturation Block สำหรับการจำลอง Limit ของ Motor

สมมติฐาน

คาดว่า การใส่ Anti Windup สามารถช่วยลดผลของ Steady State Error ได้

ตัวแปร

1. ตัวแปรต้น:
 - 1.1. ค่า Saturation ที่กำหนดลง Integration Windup
 - 1.2. ระยะเวลาของการดึง Pendulum ที่ส่งผลต่อ KI control
2. ตัวแปรตาม:
 - 2.1. การทำงานของ Controller ที่ได้
3. ตัวแปรควบคุม:
 - 3.1. Plant ของระบบ
 - 3.2. ระยะเวลาในการทำการทดลองของแต่ละ Simulation

เอกสารและงานวิจัยที่เกี่ยวข้อง

Integration Windup หรือ Integrator Windup เป็นปัญหาที่เกิดขึ้นใน PID Controller เมื่อ Actuator ถึงขีดจำกัด (Saturation) แต่ Integral Term ยังคงสะสมค่า Error ต่อไปจนมีค่ามากเกินไป ส่งผลให้เกิด Overshoot ที่มากและระบบใช้เวลานานในการกลับสู่สภาวะปกติ

กลไกการเกิด Windup เมื่อ Controller Output ($u(t)$) ถึงขีดจำกัด (u_{max} หรือ u_{min}) แต่ระบบยังมี Error อยู่ Integral Term จะยังคงสะสมค่าตามสมการ:

$$\int e(t)dt = \int (SP - PV)dt$$

เมื่อ Error เปลี่ยนเครื่องหมาย เช่น จาก Positive เป็น Negative Integral Term ที่สะสมไว้มากเกินไป จะต้องถูก Unwind ก่อน ทำให้ระบบตอบสนองช้าและเกิด Overshoot

Cramping Method วิธีป้องกัน Anti-Windup ให้ทำการหยุดการสะสม Integral เมื่อ Output ถึงขีดจำกัด โดยใช้ Block Function Conditional Integration ที่มีโค้ด

```
if (u > u_max || u < u_min) {
    // ไม่สะสม Integral
} else {
    integral += e(t) * dt
}
```

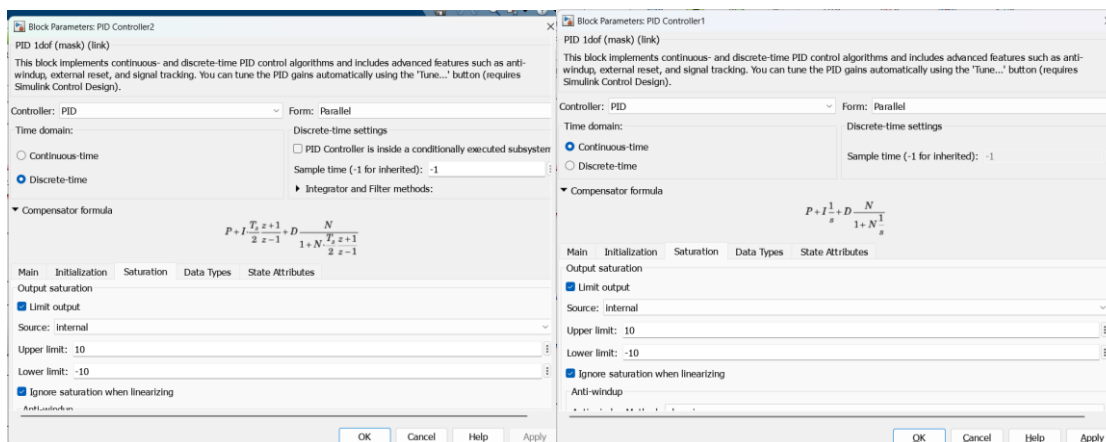
ช่วยป้องกันไม่ให้ Integral Term สะสมค่ามากเกินไปเมื่อระบบอยู่ในสถานะ Saturation

ขั้นตอนการดำเนินงาน

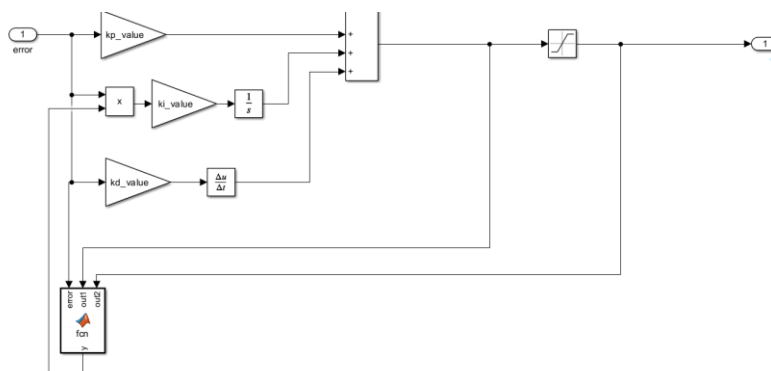
1. เขียนโปรแกรม MATLAB Function สำหรับ PID Controller โดยรองรับการทำงานทั้งแบบมี Anti-Windup และไม่มี Anti-Windup รวมถึงแบบมีและไม่มี Saturation Block เพื่อให้สามารถเลือกโหมดการทำงานได้ภายใน Function เดียว
2. กำหนด Initial Point ไว้ที่ 0 องศา และ Setpoint ไว้ที่ 3600 องศา (10 รอบ) เพื่อให้ Integral Term สะสมค่า Error เป็นเวลานานพอที่จะแสดงให้เห็นปรากฏการณ์ Windup อย่างชัดเจน หากกำหนด Setpoint ไว้เพียง 180 องศาเช่นเดียวกับการทดลองอื่น ระบบจะเข้าสู่ Steady State เร็วเกินไปจน Integration Windup ไม่มีโอกาสเกิดขึ้น
3. ทดลองรันระบบทั้งหมด 4 กรณี ได้แก่ มี Anti-Windup และไม่มี Saturation, มี Anti-Windup และมี Saturation, ไม่มี Anti-Windup และไม่มี Saturation, และไม่มี Anti-Windup และมี Saturation โดย Anti Windup จะตั้ง Saturation ไว้ที่ 10V
4. เก็บบันทึกกราฟ Controller Effort และ Position Response ของทั้ง 4 กรณี นำมาเปรียบเทียบความแตกต่างของ Overshoot และพฤติกรรมการตอบสนองของระบบ

```
%% Input Variable Section
kp_value = 0.4626;
ki_value = 0.004608;
kd_value = 0.18;
rad2deg = 180/pi;
Setpoint = 180 + 360*10;
Initial_point = 0;
```

ภาพที่ 51 ภาพแสดงการ Setup ข้อมูลสำหรับการนำไปใช้ run simulation



ภาพที่ 52 และ 53 การตั้ง Anti Windup ของ PID Controller ทั้งแบบ Continuous และ Discrete



ภาพที่ 54 การต่อ Anti Windup ของ Block Constant

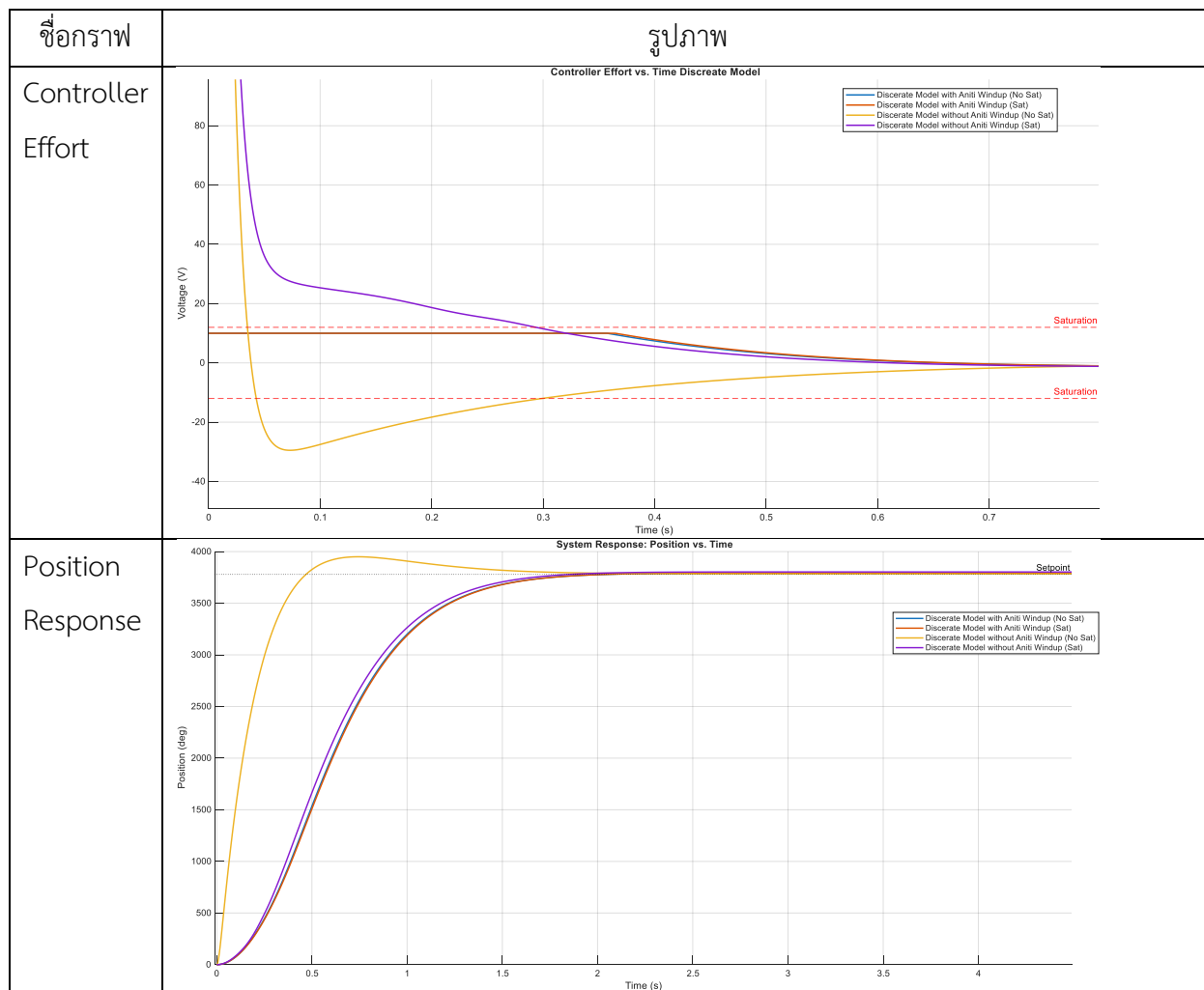
```
function y = fcn(error, out1, out2)
    is_saturated = (out1 ~= out2);
    pushing_into_saturation = (sign(error) == sign(out1));

    if is_saturated && pushing_into_saturation
        y = 0; % Clamp: Stop integration
    else
        y = 1; % Integrate normally
    end
end
```

ภาพที่ 55 การเขียนโค้ด Logic ของการเปิดปิด Anti Windup ของ Block Constant

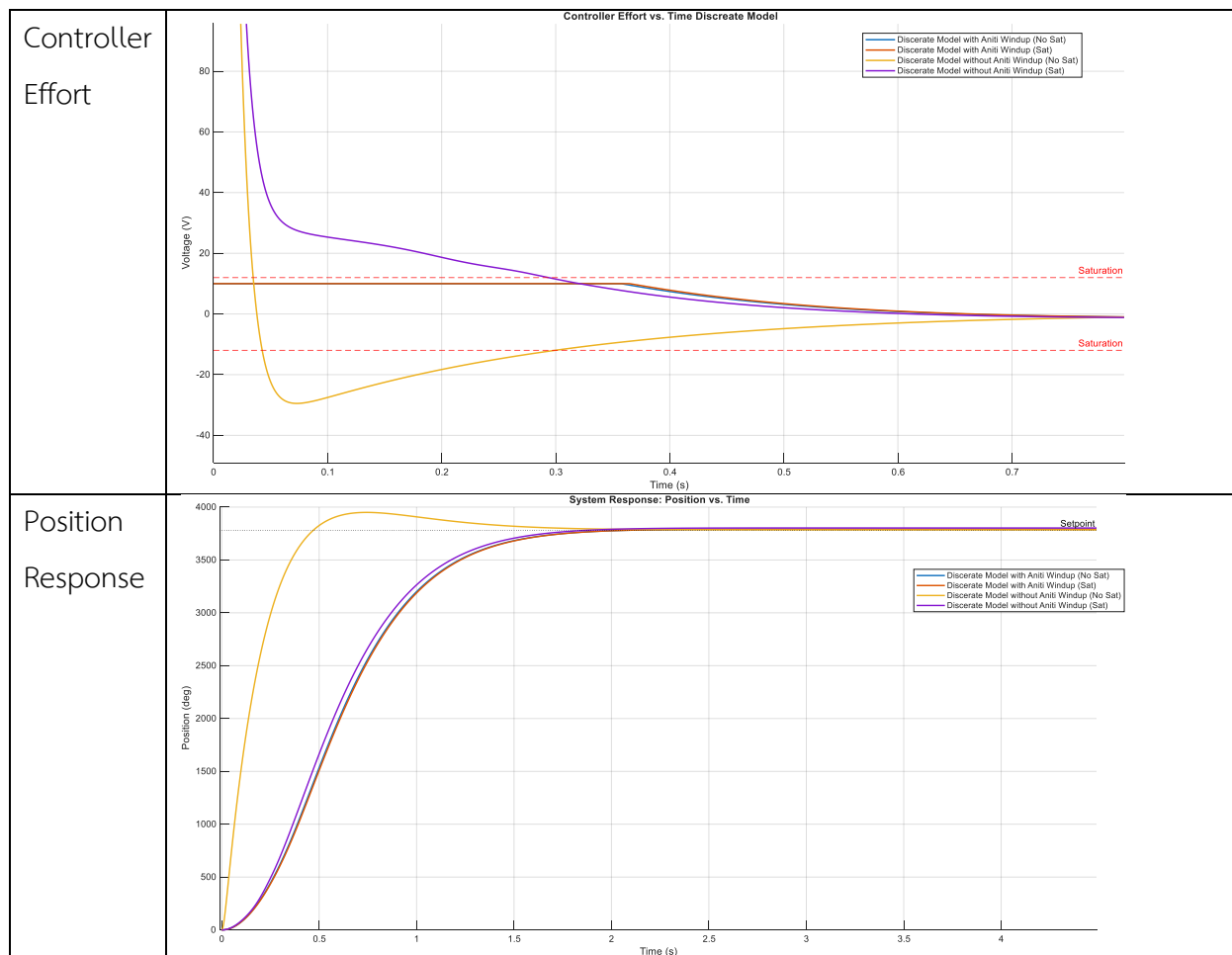
ผลการทดลอง วิธีการแสดงผล

ตารางที่ 15 เปรียบเทียบการทดลอง Anti Windup และ Saturation ของ Discrete Model

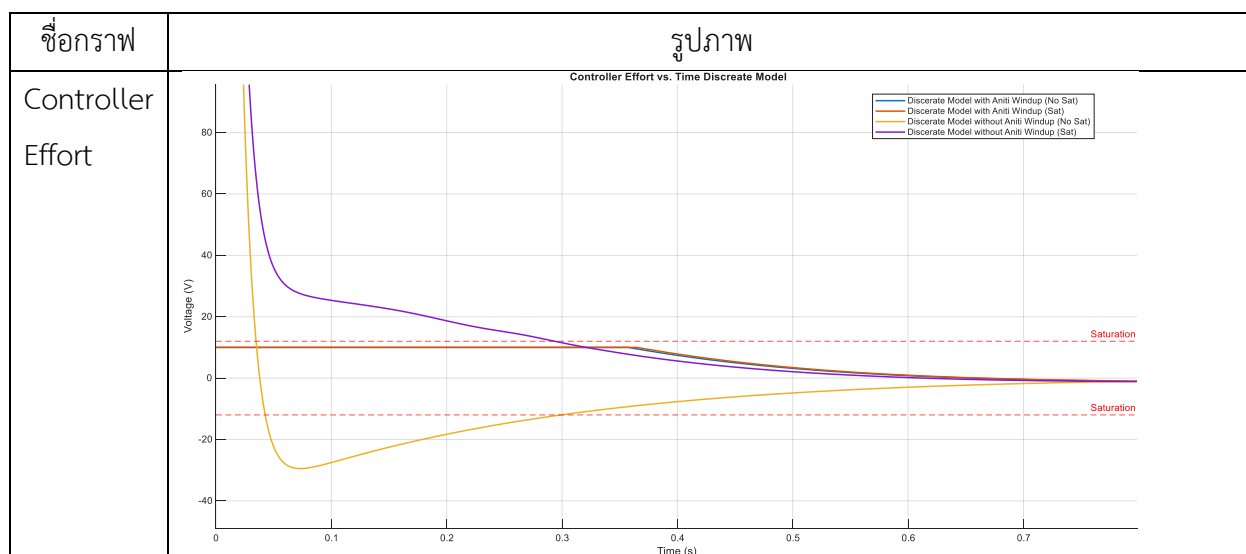


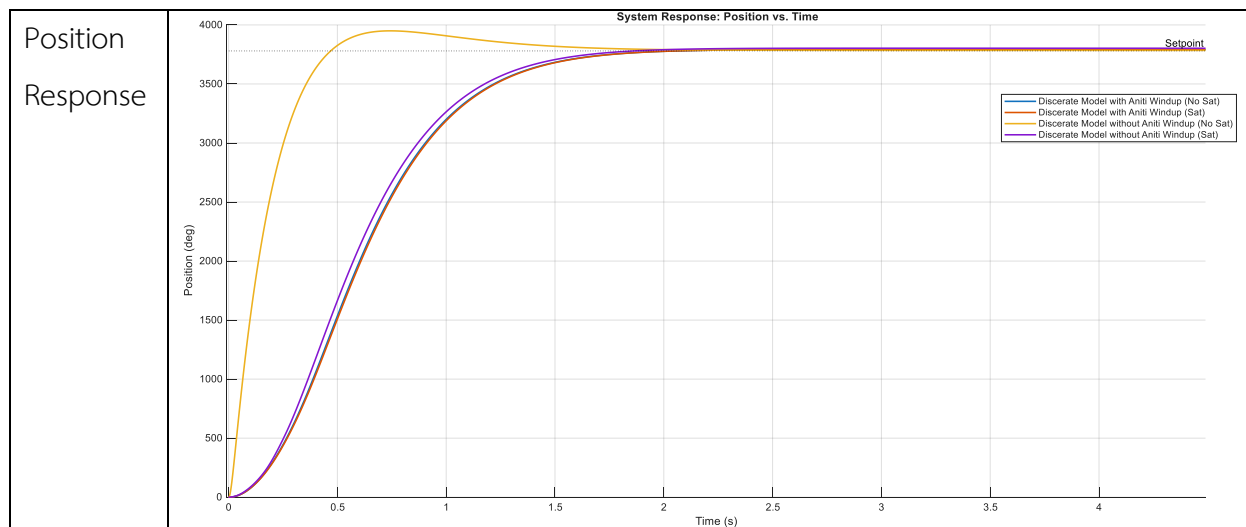
ตารางที่ 16 เปรียบเทียบการทดลอง Anti Windup และ Saturation ของ Continuous Model

ชื่อกราฟ	รูปภาพ
----------	--------



ตารางที่ 17 เปรียบเทียบการทดลอง Anti Windup และ Saturation ของ Discreate Model





ตารางที่ 18 Constant block

Configuration	PeakTime _s	Overshoot _pct	RiseTime _s	SettlingTime _s	SSError _deg
With Aniti Windup (No sat)	2.5982	0.33118	0.891	1.5185	10.792
With Aniti Windup (Sat)	2.6067	0.32988	0.895	1.5257	10.752
Without Aniti Windup (No sat)	2.4197	0.5328	0.8253	1.349	17.749
Without Aniti Windup (Sat)	2.5495	0.62816	0.8629	1.4571	21.179

ตารางที่ 19 Discerate Model

Configuration	PeakTime _s	Overshoot _pct	RiseTime _s	SettlingTime _s	SSError _deg
With Aniti Windup (No sat)	3.1133	0.30848	0.9019	1.5589	10.824
With Aniti Windup (Sat)	3.1216	0.30756	0.9058	1.5659	10.793
Without Aniti Windup (No sat)	0.743	4.4965	0.3011	1.2434	4.2871
Without Aniti Windup (Sat)	3.0196	0.60641	0.8731	1.4941	21.322

สรุปผลการทดลอง

จากการทดลองแสดงให้เห็นว่าการใส่ Anti Windup นั้นช่วยสามารถช่วยลด Overshoot ที่เกิดขึ้นในระบบได้โดยสังเกตจากเมื่อเปรียบเทียบกรณีของการใส่ Saturation แล้วเทียบกับมี Anti Windup และ ไม่มี Anti Windup นั้นจะพบว่า ค่า %Overshoot ของระบบนั้นลดลง ตัวอย่างเช่น ใน PID Controller แบบ Continuous %Overshoot ลดลงจาก 0.064% เมื่อใส่ Anti Windup ค่า %Overshoot ลดลงเหลือเพียง 0.30757% ซึ่งแนวโน้มเกิดขึ้นกับทุกรูปแบบของ Controller

เมื่อสังเกตที่ Controller Effort จะพบว่าการใส่ Anti Wire up ค่าของ Controller จะ Limit ตัวเองที่ 10 V เสมอและค่าจะลดทันทีเมื่อ Controller ต้องการลดค่า Voltage ของตัวเอง ซึ่งแตกต่างจาก Controller ที่ไม่มี Anti Wind up ที่ Controller จะหน่วงเวลาก่อนช่วงหนึ่งที่ Controller จะลดค่าของตัวเองลง

ในส่วนของการใส่ Saturation จะพบว่าการใส่ Block Saturation นั้นเพียงช่วยให้ Simulation มีลักษณะคล้ายกับของจริงเพียงเท่านั้น หรือก็คือในกรณีที่เราปลด Saturation และ Anti Windup ระบบจะ Response

อภิปรายผล

สาเหตุที่ เมื่อใส่ Block Saturation แต่ไม่ใส่ Anti Wire up ระบบจะขึ้นเร็วว่าการใส่ Anti Wire up เล็กน้อยเพราะการไม่ใส่ Anti Wire up Controller จะจ่าย Voltage เต็ม 12 V ตลอดเวลาสังเกตได้จาก กราฟ Controller Effort ซึ่งจะส่งผลให้ค่าของ Controller ไปทับกับผลจาก Gravity Compensation

ในส่วนของการใส่ Anti Wind Up สามารถลด %Overshoot ของตัวเองได้เนื่องจาก Controller สามารถตอบสนองกับ Feedback ได้ทันที

ข้อเสนอแนะ

1. ควรกำหนดค่า Saturation ให้สอดคล้องกับขีดจำกัดของ Actuator จริงในระบบ เนื่องจากหากตั้งค่า Saturation สูงเกินไปจะไม่สามารถป้องกัน Windup ได้อย่างมีประสิทธิภาพ แต่หากต่ำเกินไปจะทำให้ Controller ไม่สามารถจ่าย Voltage ได้เพียงพอต่อการควบคุมระบบ
2. ควรใช้ Anti-Windup แบบ Conditional Integration ร่วมกับ Saturation Block เสมอ เพื่อหยุดการสะสมค่าของ Integral Term เมื่อ Output ของระบบถึงขีดจำกัด ซึ่งจะช่วยลด Overshoot และทำให้ระบบเข้าสู่สภาวะคงตัวได้เร็วขึ้น

อ้างอิง

Robust Anti-Windup Control Design for PID Controllers [IEEE Xplore Full-Text PDF:](#)
(2017)

PID Anti-Windup Schemes [PID Anti-Windup Schemes | Scilab](#) (2018)

การทดลองที่ 3.3 การทดลองผลของ Noise ที่ส่งผลต่อการทำงานของ PID Controller

จุดประสงค์

1. เพื่อวิเคราะห์ผลกระทบ Noise ในแต่ละขนาดและหลากหลายรูปแบบ ที่ส่งผลกระทบต่อ PID Control
2. เพื่อศึกษาวิธีการแก้ไข Noise ที่เกิดขึ้นในระบบ

สมมติฐาน

เมื่อ Noise ของระบบเพิ่มขึ้นจะส่งผลให้ระบบตอบสนองได้แย่ลง โดยยิ่งเพิ่มความถี่ก็จะทำให้ Controller ไม่สามารถแสดง พฤติกรรมได้เช่นเดิม

ตัวแปร

1. ตัวแปรต้น:
 - 1.1. Frequency ของ Noise ในระบบ
 - 1.2. การตั้งค่า Cut-Off Frequency ใน PD และ PID Control แต่ละวิธี
2. ตัวแปรตาม:
 - 2.1. Output ของระบบ
3. ตัวแปรควบคุม:
 - 3.1. Plant ของระบบ

เอกสารและงานวิจัยที่เกี่ยวข้อง

Meaning ของแต่ละ Parameter ใน Block Band Limit White Noise ใน Simulink

ในการทดลองนี้ใช้ Band-Limited White Noise Block ใน Simulink เพื่อจำลอง Noise ที่เกิดขึ้นในระบบจริง โดย Block นี้มีพารามิเตอร์สำคัญดังนี้

Noise Power (P):

กำหนดกำลังของสัญญาณ Noise โดยมีหน่วยเป็น (unit^2) ค่านี้แสดงถึงความแรงหรือขนาดของ Noise ที่ต้องการจำลอง ยิ่งค่า Noise Power สูง สัญญาณ Noise ที่ได้จะมีแอมพลิจูดมากขึ้น

$$P = E[n^2(t)]$$

โดยที่ $n(t)$ คือสัญญาณ Noise

Sample Time (T_s):

กำหนดช่วงเวลาในการสุ่มตัวอย่างสัญญาณ Noise มีหน่วยเป็นวินาที ค่านี้กำหนดความถี่ในการสร้าง Noise โดย Bandwidth ของ Noise จะเท่ากับ:

$$B = \frac{\pi}{T_s}$$

ยิ่ง Sample Time เล็ก Bandwidth ของ Noise ก็ยิ่งสูง

Seed เป็นค่าเริ่มต้นสำหรับ Random Number Generator ใช้ในการสร้างลำดับของ Noise แบบสุ่ม การใช้ Seed เดียวกันจะได้ลำดับ Noise ที่เหมือนกันทุกครั้งที่รัน Simulation ทำให้สามารถทำซ้ำผลการทดลองได้

ผลกระทบต่อ Derivative Term

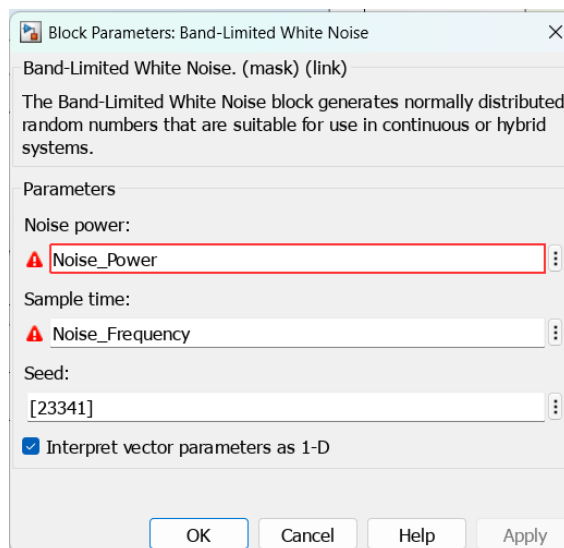
Derivative Term ของ PD และ PID Controller คำนวณจาก

$$D(t) = K_d \frac{de(t)}{dt}$$

เมื่อมี Noise ในสัญญาณ Process Variable การหาอนุพันธ์จะขยาย Noise ความถี่สูงให้มีขนาดใหญ่ขึ้น ทำให้ Controller Output มีการสั่นไหวสูง (Chatter) และส่งผลกระทบต่อสมรรถนะของระบบ

ขั้นตอนการดำเนินงาน

1. เพิ่ม Band-Limited White Noise Block เข้าไปใน Feedback Path ของ Simulink เพื่อจำลองสัญญาณรบกวนที่เกิดขึ้นจาก Sensor ในระบบจริง



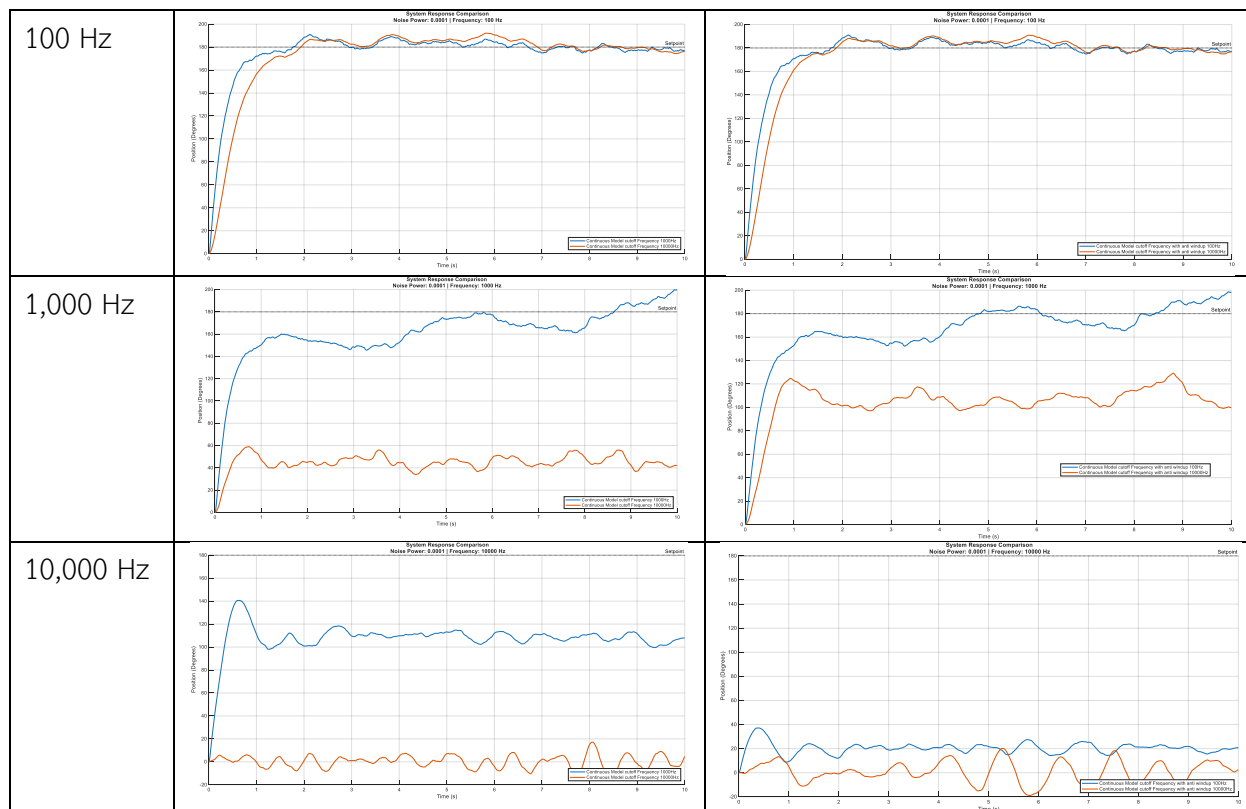
ภาพที่ 56

2. ทำการทดลองเฉพาะ Continuous Model เท่านั้น โดยปรับค่า Filter Coefficient ของ Derivative Term ใน PID Block เพื่อสังเกตว่าการตั้งค่า Cut-Off Frequency ที่ต่างกัน ส่งผลต่อความสามารถในการตัดสัญญาณรบกวนอย่างไร
3. ทดลองเปลี่ยนความถี่ของ Noise ที่ 100 Hz, 1000 Hz และ 10000 Hz โดยในแต่ละความถี่ให้เปรียบเทียบผลระหว่างการใช้ Cut-Off Frequency ต่ำ (100 Hz) และสูง (10000 Hz) ทั้งแบบมีและไม่มี Anti-Windup
4. บันทึกกราฟ Position Response และ Controller Effort ของแต่ละกรณี เพื่อวิเคราะห์ว่าเกิดอะไรขึ้นเมื่อความถี่ของ Noise สูง เมื่อความถี่ต่ำ และเมื่อใส่ Anti-Windup
5. ทดลองปรับค่า Parameter ใน Block Band Limit White Noise ใน Simulink เมื่อใส่เข้าไปใน PID และ PD Control ทั้งแบบไม่มี Low-pass Filter และไม่มี Low-pass Filter
6. ทดลองใส่และไม่ใส่ Anti Wind Up ในระบบ
7. ใส่ Block Saturation ก่อนเข้า Plant

ผลการทดลอง วิธีการแสดงผล

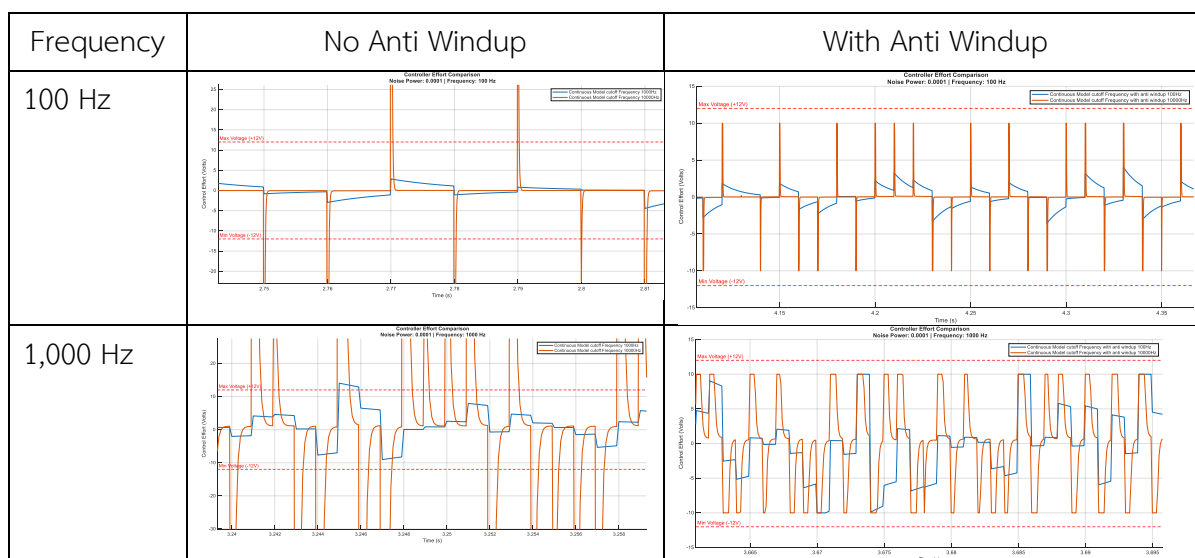
ตารางที่ 20 ตารางเปรียบเทียบผลของ Position Response ระหว่างการใส่ Anti Windup และไม่ใส่ Anti Windup ที่ความถี่แตกต่างกัน

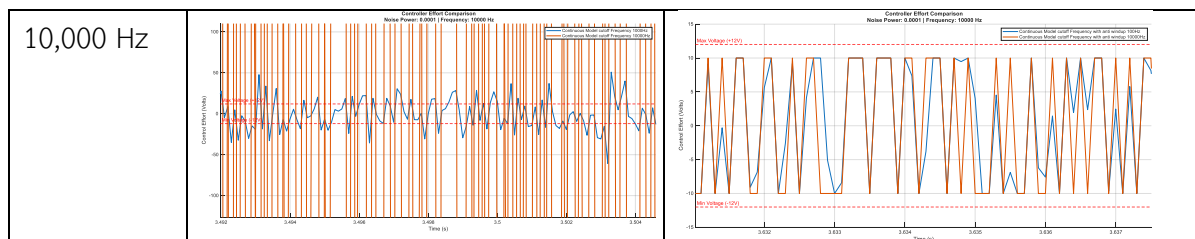
Frequency	No Anti Windup	With Anti Windup
-----------	----------------	------------------



*หมายเหตุ สีฟ้าคือกราฟจาก Controller ที่มี Cut-Off Frequency 100 Hz สีส้มคือกราฟจาก Controller ที่มี Cut-Off Frequency 10,000 Hz

ตารางที่ 21 ตารางเปรียบเทียบผลของ Controller Effort ระหว่างการใส่ Anti Windup และไม่ใส่ Anti Windup ที่ความถี่แตกต่างกัน





*หมายเหตุ สีฟ้าคือกราฟจาก Controller ที่มี Cut-Off Frequency 100 Hz สีส้มคือกราฟจาก Controller ที่มี Cut-Off Frequency 10,000 Hz

สรุปผลการทดลอง

จากการทดลองแสดงให้เห็นว่าที่ความถี่สูง Controller ไม่สามารถตอบสนองกับระบบได้เนื่องจาก Controller เมื่อเจอกับ Noise ที่มีความถี่สูง Controller จะจ่าย Voltage ที่สูงขึ้นสังเกตจากที่ความถี่สูง Controller จะจ่าย Voltage เข้า ไกล่อันต์ ในขณะที่เมื่อจ่าย Noise ที่ Frequency 100 Hz Controller จะจ่าย Voltage น้อยลง

ในส่วนของ Low Pass Filter ที่ Cut Off Frequency 100 Hz ระบบจะตอบสนองต่อ Noise น้อยกว่าที่ Cut-off Frequency 10,000 Hz สังเกตจาก Controller ที่มี Cut-Off Frequency 100 Hz สามารถถึง Response ของระบบได้ใกล้ Setpoint มากกว่า Controller ที่มี Cut-Off Frequency 10,000 Hz

ในส่วนของ Anti Windup นั้นมีส่วนช่วงในการตอบสนองของ Controller แคบางส่วนเท่านั้นสังเกตจาก ข้อมูลเมื่อใส่ Anti Wind up ที่ความถี่ของ Noise 1000 Hz Anti Wind up สามารถถึงค่า Position Response ขึ้นมาได้ แต่เมื่อปรับค่าความถี่ของ Noise ไปที่ 10,000 Hz Anti Wind up ก็ไม่สามารถรักษาเสถียรภาพ ของ ระบบได้

อภิปรายผล

จากการทดลองสอดคล้องกับสมการ *Derivative* ซึ่งการที่ Noise มีความถี่สูงขึ้นความชันของ Noise ก็จะมีค่าสูงขึ้นเช่นเดียวกัน ซึ่งส่งผลให้เมื่อใส่เข้าไปในพจน์ K_d การจ่ายไฟของ Controller ก็จะแปรปรวนขึ้นตาม ความถี่ของ Noise ซึ่ง Low Pass Filter จะเข้ามาช่วยลดความชันของ Noise ทำให้ Controller สามารถทำงาน ได้อย่างขึ้น

ทั้งนี้หากสังเกตจาก *Control Effort* ระบบจะไม่สามารถ *Position Response* ของระบบจะไม่ถึง *Set Point* เมื่อค่า *Controller Effort* เกินค่า *Saturation* ไป

ข้อเสนอแนะ

1. ควรเพิ่มตัวกรอง Low-Pass Filter เพื่อลดทอนความถี่ของสัญญาณรบกวนก่อนนำไปคำนวณในตัวควบคุม ซึ่งจะช่วยให้ระบบทำงานได้เสถียรขึ้น
2. ควรใช้ Anti-Windup ร่วมกับ Low-Pass Filter เสมอ เนื่องจากเมื่อมี Noise ความถี่สูงในระบบจะทำให้ Integral Term สะสมค่าผิดพลาดจนเกิด Windup ส่งผลให้ระบบมี Overshoot สูงและใช้เวลานานในการเข้าสู่สภาวะคงตัว

อ้างอิง

[Band-Limited White Noise - Introduce white noise into continuous system - Simulink](#)

[Measurement Noise Degrades Derivative Action – Control Guru](#)

การทดลองที่ 3.4 การทดลองปรับค่า K_d ที่ส่งผลต่อการตอบสนองต่อ Noise

จุดประสงค์

1. เพื่อศึกษาผลกระทบของการปรับ K_d ต่อการตอบสนองต่อ Noise ต่อ PID Control
2. เพื่อสังเกตผลของการปรับ K_d ที่ส่งผลต่อ Overshoot ของระบบเมื่อไม่มี Noise ในระบบ

สมมติฐาน

การปรับ K_d ส่งผลให้ค่า Overshoot ของระบบมีค่าน้อยลงและส่งผลให้ผลของ Noise มีค่ามากยิ่งขึ้น

ตัวแปร

1. ตัวแปรต้น:
 - 1.1. การปรับค่า K_d ใน Controller แต่ละแบบ
 - 1.2. การเปลี่ยน Controller แต่ละแบบ
2. ตัวแปรตาม:
 - 2.1. Position Response ของระบบ
3. ตัวแปรควบคุม:
 - 3.1. Plant ของระบบ

เอกสารและงานวิจัยที่เกี่ยวข้อง

สัญญาณรบกวน (Noise) เป็นปัจจัยที่มักมาจากเซ็นเซอร์หรือสัญญาณรบกวนทางไฟฟ้า การปรับค่า Gain ของตัวควบคุมส่งผลโดยตรงต่อ Sensitivity ของระบบต่อสัญญาณรบกวน ณ Derivative Gain (K_d) ซึ่งจะสามารถทราบถึงการเปลี่ยนแปลงได้หากค่า K_d มีค่าที่มากระบบจะขยาย Noise ที่มีความถี่สูง ทำให้สัญญาณในระบบมีการสั่นสะเทือนและเกิดความร้อนที่สะสมในระบบ PD ขึ้นดังสมการ

$$u = K_p(e + n) + K_d \frac{d(e + n)}{dt}$$

ซึ่งงานวิจัยที่อ้างอิงระบุว่าหากเพิ่มค่า K_p ส่งผลทำให้ Noise ที่สัญญาณ Output นั้นมีมากใกล้เคียงกับค่า K_d แต่หากปรับค่าที่ Integral Gain (K_i) จะช่วยเฉลี่ยค่าสัญญาณรบกวนได้ในระดับหนึ่ง ในการทดลองปรับค่าสัญญาณ Noise จะที่รู้ถึงขีดจำกัดในการเพิ่มค่า Gain เพราะ หากค่า Gain สูงขึ้นส่งผลทำให้ระบบนั้นตอบสนองได้เร็วขึ้น โดยมีเงื่อนไขอยู่ที่ หากค่า Gain สูงเกินไปที่ระบบจะรับ Noise ได้จะทำให้เกิดการสั่นเทือนทางกลและลดอายุการใช้งานของอุปกรณ์

ขั้นตอนการดำเนินงาน

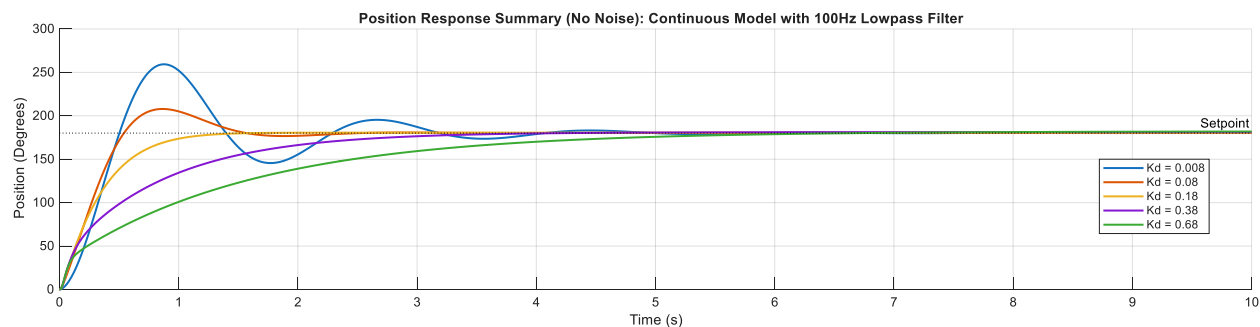
1. ใช้การตั้งค่าระบบเดิมจากการทดลองที่ 3.3 โดยคงสัญญาณ White Noise ไว้ใน Feedback Path

```
Noise_Power = 0.0001;
Noise_Frequency = 0.001;
```

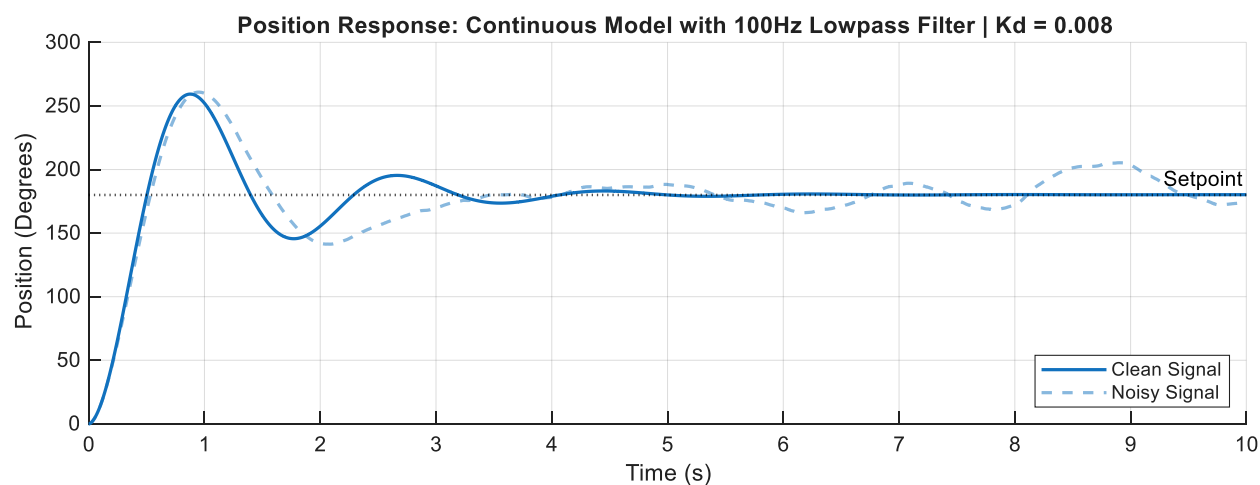
2. ทดลองปรับค่า Parameter ใน Block Band Limit White Noise ใน Simulink เมื่อใส่เข้าไปใน PID และ PD Control ทั้งแบบไม่มี Low-pass Filter และไม่มี Low-pass Filter
3. ปรับค่า K_d หลายค่า ได้แก่ 0.008, 0.08, 0.18, 0.38 และ 0.68 โดยคงค่า K_p และ K_i ไว้คงที่ บันทึกกราฟ Position Response ของแต่ละค่า K_d ทั้งแบบไม่มี Noise (Clean Signal) และมี Noise (Noisy Signal) เพื่อสังเกตว่าการเพิ่มค่า K_d ส่งผลต่อ Overshoot และความไวต่อ Noise อย่างไร

```
%% Input Variable Section
kp_value = 0.4626;
ki_values = 0.004608;
kd_value = [0.008,0.08,0.18,0.28,0.68];
```

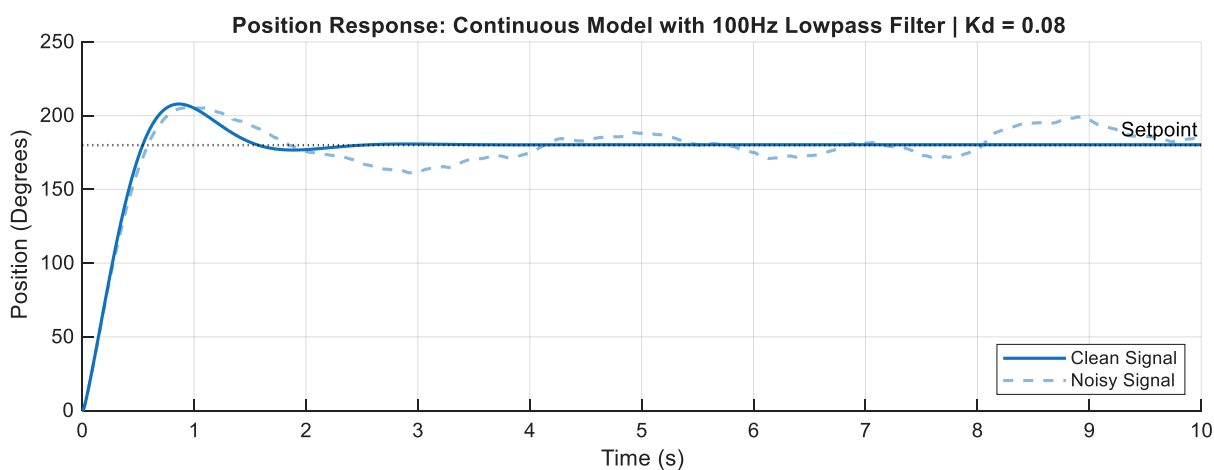
ผลการทดลอง



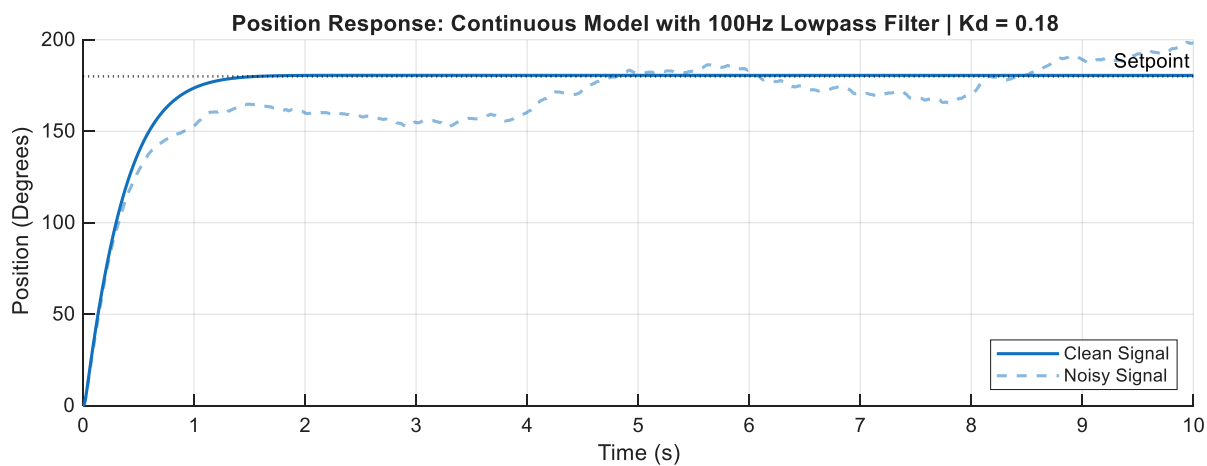
ภาพที่ 57 เปรียบเทียบ Position Response ต่อค่า K_d ที่เปลี่ยนไป



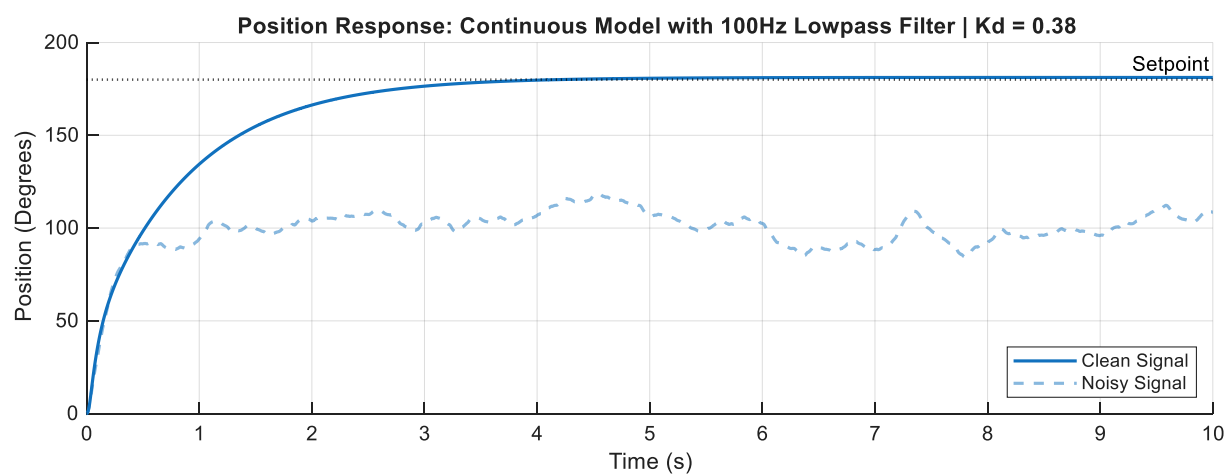
ภาพที่ 58 เปรียบเทียบ Position Response ต่อกับการใส่ Noise และไม่ใส่ Noise ที่ $K_d 0.008$



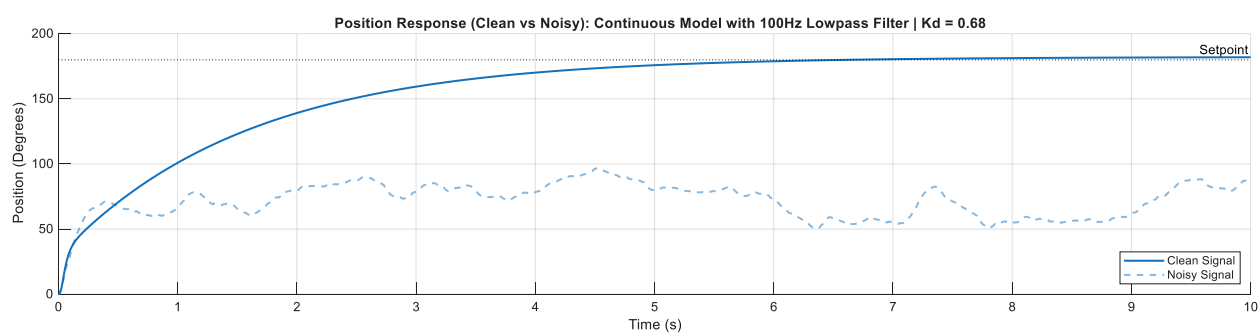
ภาพที่ 59 เปรียบเทียบ Position Response ต่อกับการใส่ Noise และไม่ใส่ Noise ที่ $K_d 0.08$



ภาพที่ 60 เปรียบเทียบ Position Response ต่อกับการใส่ Noise และไม่ใส่ Noise ที่ $K_d = 0.18$



ภาพที่ 61 เปรียบเทียบ Position Response ต่อกับการใส่ Noise และไม่ใส่ Noise ที่ $K_d = 0.38$



ภาพที่ 62 เปรียบเทียบ Position Response ต่อกับการใส่ Noise และไม่ใส่ Noise ที่ $K_d = 0.68$

สรุปผลการทดลอง

จากการทดลองพบว่าเมื่อเพิ่มค่า K_d จะทำให้ระบบ Response ซ้ำลงสังเกตจากที่ค่า K_d 0.008 ระบบจะมี Overshoot ที่สังเกตได้อย่างชัดเจน แต่เมื่อเพิ่ม K_d ไปที่ 0.68 ระบบจะลู่เข้าสู่ Setpoint ซ้ำลงอย่างชัดเจน

การเพิ่มค่า K_d จะทำให้ระบบตอบสนองต่อ Noise แย่ลง โดยสังเกตจากที่ K_d 0.68 ระบบไม่สามารถพาตัวเองไปที่ Setpoint ได้

อภิปรายผล

ในการเพิ่มค่า K_d หากมองในมุมของ root locus จะเป็นการเลื่อน Zero ของระบบไปทางซ้ายมากยิ่งขึ้น ซึ่งส่งผลให้ Path ของ Pole นั้นเลื่อนไปทางซ้ายเช่นเดียวกัน ซึ่งสอดคล้องกับผลการทดลองที่เมื่อเพิ่มค่า K_d ระบบจะตอบสนองซ้ำลง

ในการเพิ่มค่า K_d ระบบจะตอบสนองต่อ Noise รุนแรงมากยิ่งขึ้นเช่นเดียวกับในการทดลองที่ 3.3 สาเหตุมาจากพจน์ sK_d ซึ่งเป็นที่แน่นอนว่าเมื่อเพิ่มค่า K_d ก็จะทำให้จากปกติที่ Noise ส่งผลต่อ *Controller Effort* อยู่แล้วจะยิ่งแปรผันตามกัน

ข้อเสนอแนะ

1. ควรระวังการเพิ่มค่า K_d มากเกินไป มีผลต่อการเกิดสัญญาณรบกวนในระบบให้รุนแรงขึ้นซึ่งทำให้มอเตอร์เกิดการสั่นจนเสียหายได้
2. ควรใช้ Low-Pass Filter คู่กับ Derivative Term เสมอ โดยเลือกค่า Cut-Off Frequency ให้เหมาะสมกับความถี่ของ Noise ในระบบ เพื่อลดผลกระทบของ Noise ต่อ K_d โดยไม่ทำให้การตอบสนองของระบบช้าลงมากเกินไป
3. ในการนำไปใช้งานจริงบนฮาร์ดแวร์ ควรวัดและวิเคราะห์ลักษณะของ Noise ที่เกิดขึ้นจริงในระบบก่อนเพื่อใช้เป็นข้อมูลในการเลือกค่า K_d และค่า Cut-Off Frequency ของ Filter ที่เหมาะสมกับสภาพการใช้งานจริง

อ้างอิง

[Band-Limited White Noise - Introduce white noise into continuous system - Simulink](#)

Measurement Noise Degrades Derivative Action – Control Guru

การทดลองที่ 3.5 การทดลองปรับค่า K_i ที่ส่งผลต่อ Steady State Error

จุดประสงค์

1. เพื่อสังเกตผลกระทบของการปรับ K_i ที่ส่งผลต่อระบบ

สมมติฐาน

การปรับ K_i ส่งผลให้ค่า Overshoot ของระบบมีค่าน้อยลงและส่งผลให้ผลของ Noise มีค่ามากยิ่งขึ้น

ตัวแปร

1. ตัวแปรต้น:
 - 1.1. การปรับค่า K_i ใน Controller แต่ละแบบ
 - 1.2. การเปลี่ยน Controller แต่ละแบบ
2. ตัวแปรตาม:
 - 2.1. Response ของระบบ
3. ตัวแปรควบคุม:
 - 3.1. Plant ของระบบ

เอกสารและงานวิจัยที่เกี่ยวข้อง

ค่า **Steady State Error** ที่เป็ค่าที่แตกต่างระหว่างเป้าหมายกับ output ณ เวลาผ่านไปจนระบบนิ่ง ซึ่ง k_i จะพยายามทำให้ค่าที่เกิดขึ้นของ Steady State Error ลดลงอย่างสมบูรณ์จากการคำนวณการสะสมค่าของความผิดพลาดในอดีตให้เป็นสัญญาณควบคุมที่มี Dynamics หรือ Gravity ซึ่งจะทำงานในส่วน Integralดังสมการ

$$u_1(t) = K_i \int_0^t e(\tau) d\tau$$

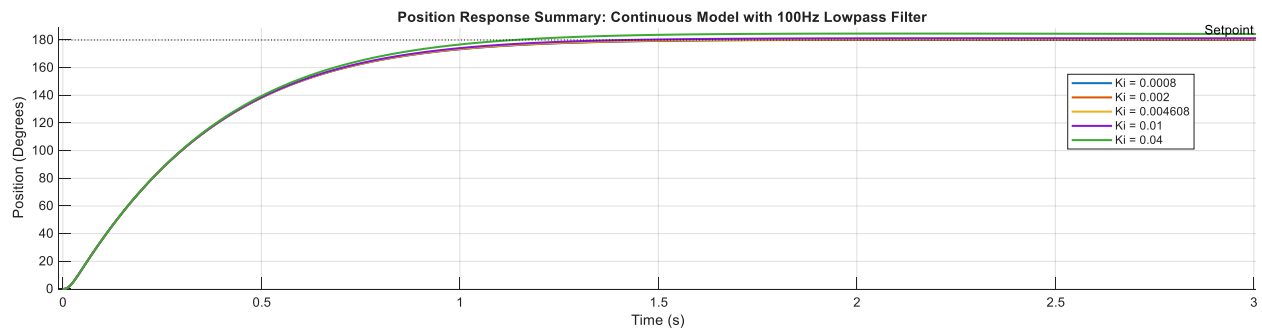
ดังนั้นหากเพิ่ม k_i จะทำให้ระบบมีความแม่นยำมากขึ้น ในเงื่อนไขที่หาก k_i สูงเกินไปจะมีผลให้มีการสั่นเกิดขึ้นในระบบจนทำให้เกิด Integration Windup ซึ่งงานวิจัยอ้างอิงว่าค่าที่เหมาะสมสำหรับ Type 0 เป็น Type 1 จะให้แน่นอนว่า **Steady State Error** ขาเข้าจะเป็นศูนย์เสมอเพื่อไม่ให้สะสมมากเกินไปจนทำให้ระบบควบคุมยาก

ขั้นตอนการดำเนินงาน

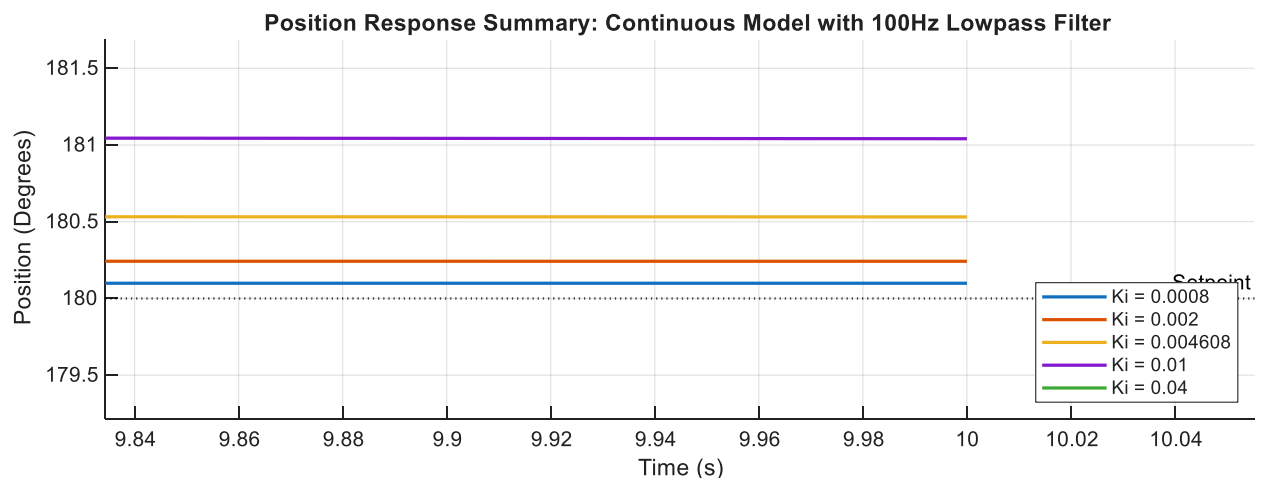
1. ใช้การตั้งค่าระบบเดิมจากการทดลองที่ 3.3 โดยคงสัญญาณ White Noise ไว้ใน Feedback Path
2. ทดลองปรับค่า Parameter ใน Block Band Limit White Noise ใน Simulink เมื่อใส่เข้าไปใน PID และ PD Control ทั้งแบบไม่มี Low-pass Filter และไม่มี Low-pass Filter
3. ปรับค่า K_i หลายค่า ได้แก่ 0.0008, 0.002, 0.004608, 0.01 และ 0.04 โดยคงค่า K_p และ K_d ไว้คงที่ บันทึกกราฟ Position Response พร้อม Peak Time, %Overshoot, Rise Time และ Settling Time ของแต่ละค่า K_i เพื่อสังเกตว่าการเพิ่มค่า K_i ส่งผลต่อ Steady State Error และ Settling Time อย่างไร

```
kp_value = 0.4626;
kd_value = 0.18; % Fixed value
ki_values = [0.0008, 0.002, 0.004608, 0.01, 0.04];
```

ผลการทดลอง



ภาพที่ 63 เปรียบเทียบ Position Response เมื่อปรับค่า K_i ในระบบ



ภาพที่ 64 เปรียบเทียบ Steady State Error ของแต่ละค่า K_i

ตารางที่ 22 ตารางแสดงค่า Peak Time, Percent Overshoot, Rise Time และ Settling Time ที่ได้จากค่า K_i ที่ต่างกัน

K_i Value	Peak Time (s)	Percent Overshoot (%)	Rise Time (s)	Settling Time (s)
0.0008	2.456	0.0618	0.6839	1.1513
0.002	2.4614	0.1448	0.6828	1.1436
0.004608	2.4569	0.3236	0.6804	1.1279
0.01	2.405	0.6862	0.6755	1.0989
0.04	2.1058	2.5684	0.6513	5.0623

สรุปผลการทดลอง

จากการทดลองแสดงให้เห็นว่าเมื่อเพิ่มค่า K_i ในระบบจะทำให้ Steady State Error มีค่าสูงขึ้น และยังส่งผลให้ Percent Overshoot ของระบบมีค่าเพิ่มมากยิ่งขึ้น

อภิปรายผล

การปรับ K_i เพิ่มขึ้นส่งผลให้ ระบบส่ายมากยิ่งขึ้น

ข้อเสนอแนะ

1. ควรเลือกค่า k_i ที่ไม่สูงเกินไป เนื่องจากค่า k_i ที่มากเกินไปจะทำให้ Settling Time เพิ่มขึ้นอย่างมีนัยสำคัญ แม้จะช่วยลด Steady State Error ได้ก็ตาม
2. ควรใช้ Anti-Windup ร่วมกับ Integral Term เสมอ เพื่อป้องกันการสะสมค่า Error ที่มากเกินไปเมื่อระบบอยู่ในสถานะ Saturation
3. ควรพิจารณา Trade-off ระหว่างค่า Steady State Error และ Settling Time ในการเลือกค่า k_i ที่เหมาะสม โดยอ้างอิงจาก Requirement ของระบบเป็นหลัก

อ้างอิง

[Band-Limited White Noise - Introduce white noise into continuous system - Simulink](#)

[Measurement Noise Degrades Derivative Action – Control Guru](#)

4. การออกแบบ Cascade Controller

การทดลองที่ 4.1 การทดลองเพื่อหาค่า Time Constant ที่เหมาะสมของ Reference Feedforward Controller

จุดประสงค์

1. เพื่อให้สามารถออกแบบ Reference Feedforward Controller ได้
2. เพื่อให้สามารถปรับพารามิเตอร์ของ Reference Feedforward Controller ให้สามารถทำงานได้อย่างมีประสิทธิภาพ
3. เพื่อให้สามารถวิเคราะห์ผลของการทำ Reference Feedforward Controller ได้

สมมติฐาน

ค่า Time Constant ที่มากเกินไปจะทำให้ระบบเกิดการตอบสนองช้า ส่วนค่า Time Constant ที่น้อยเกินไปจะทำให้ Reference Feedforward Controller ตอบสนองต่อค่า Time Constant นั้น ๆ ไม่ทัน ซึ่งการจะหาค่า Time Constant ที่เหมาะสมได้ต้องทำการปรับค่า Time Constant และดูพฤติกรรมการตอบสนองของ Reference Feedforward Controller เพื่อหาความถี่ของการตอบสนองที่เหมาะสม

ตัวแปร

1. ตัวแปรต้น:
 - 1.1. การปรับค่า K_d ใน Controller แต่ละแบบ
2. ตัวแปรตาม:

3.

2.1:3.1. พฤติกรรมการตอบสนองของ Reference Feedforward Controller

2.2:3.2. ตัวแปรควบคุม: Reference Feedforward Transfer Function

2.3:3.3. ค่า Input ของ Reference Feedforward

—

2.4:3.4. Plant ของระบบ

2.5:3.5. การใส่ Gravity Compensation

2.6:3.6. เวลาที่ใช้เก็บค่าการตอบสนองของ Reference Feedforward

เอกสารและงานวิจัยที่เกี่ยวข้อง

Reference Feedforward Controller เป็นการควบคุมที่ใช้เพื่อปรับปรุงการตามติด Setpoint โดยป้อนข้อมูลเกี่ยวกับการเปลี่ยนแปลงของ Setpoint เข้าไปในระบบล่วงหน้า ทำให้ระบบสามารถตอบสนองได้เร็วขึ้น และลด Overshoot ที่เกิดจากการเปลี่ยน Setpoint แบบทันทีทันใด

Setpoint Filter แบบ First-Order Reference Feedforward Controller ที่นิยมใช้คือ First-Order Low-Pass Filter ที่กรอง Setpoint ก่อนส่งเข้า Controller

$$F(s) = \frac{1}{1 + \tau s}$$

โดยที่ τ คือ Time Constant ของ Filter มีหน่วยเป็นวินาที

ประโยชน์ของ Reference Feedforward:

- ลด Overshoot เมื่อมีการเปลี่ยน Setpoint แบบฉับพลัน
- ลดความรุนแรงของสัญญาณควบคุม (Control Action) ไม่ให้กระตุ้น Actuator มากเกินไป
- ทำให้ระบบมี Smooth Response และประหยัดพลังงาน

ขั้นตอนการดำเนินงาน

1. จากสมการของมอเตอร์ที่กำลังกำลังให้กับ Vertical Pendulum ใน S Domain

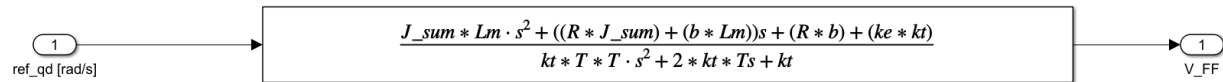
$$\omega_m = \left(\frac{K_m}{J_{\text{รวม}} L_m s^2 + (R_m J_{\text{รวม}} + B_m L_m) s + R_m B_m + K_t K_e} \right) V_{in} - \left(\frac{R_m + L_m s}{J_{\text{รวม}} L_m s^2 + (R_m J_{\text{รวม}} + B_m L_m) s + R_m B_m + K_t K_e} \right) \tau_L$$

เมื่อต้องการหา Reference Feedforward Transfer Function ที่มี ω_m เป็น Input และ V_{in} เป็น

Output จากทฤษฎีบท Superposition กำหนดให้ τ_L เป็น 0 จะสามารถจัดรูปสมการได้ ดังนี้

$$\frac{V_{in}}{\omega_m} = \left(\frac{J_{\text{รวม}} L_m s^2 + (R_m J_{\text{รวม}} + B_m L_m) s + R_m B_m + K_t K_e}{K_m} \right)$$

2. จาก Reference Feedforward Transfer Function ที่ได้จากข้อข้างต้น จะยังไม่สามารถนำมาใช้งานได้ เนื่องจากเลขชี้กำลังสูงสุดของตัวเศษมีมากกว่าตัวส่วนซึ่งจะไม่สามารถหาค่าได้ ดังนั้นจึงต้องมีการใส่ Low Pass Filter ที่มีสูตรดังนี้ $\frac{1}{(1+\tau s)^n}$ โดยที่ n ในที่นี้ให้เท่ากับ 2 เพราะเลขชี้กำลังสูงสุดของตัวเศษมากกว่าตัวส่วนอยู่ 2 จะได้สมการดังนี้ $G_{ff}(s) = \left(\frac{J_{\text{รวม}} L_m s^2 + (R_m J_{\text{รวม}} + B_m L_m) s + R_m B_m + K_t K_e}{K_m} \right) \frac{1}{(1+\tau s)^2}$ จัดรูปจะได้ $G_{ff}(s) = \frac{J_{\text{รวม}} L_m s^2 + (R_m J_{\text{รวม}} + B_m L_m) s + R_m B_m + K_t K_e}{K_m \tau^2 s^2 + 2\tau K_m s + K_m}$
3. นำ Reference Feedforward Transfer Function ที่ได้ ไปเขียนใน Simulink ดังนี้

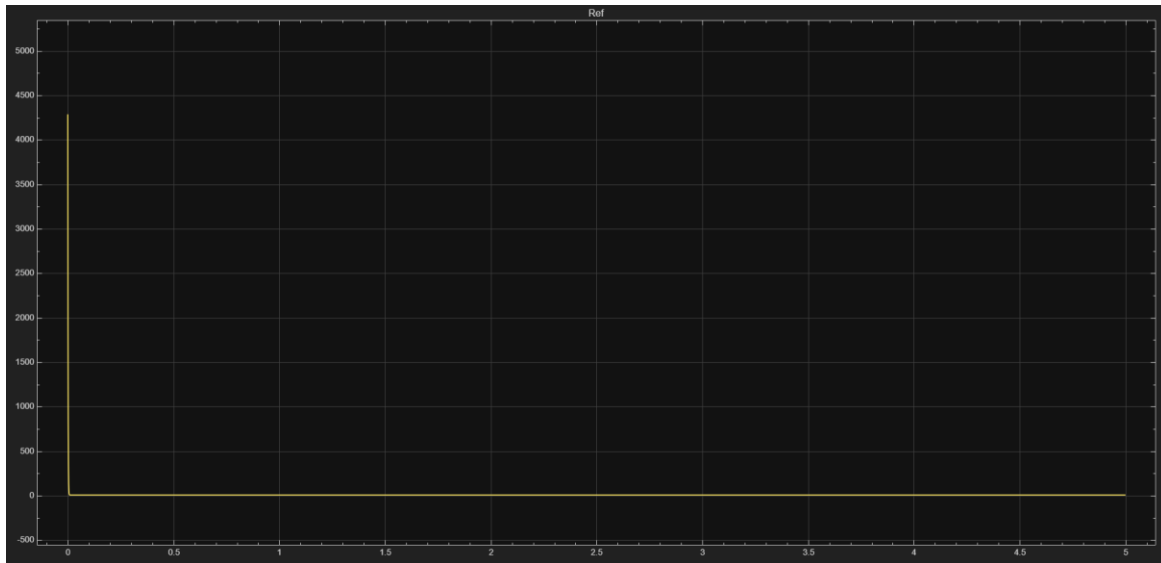


ภาพที่ 65 Reference Feedforward Transfer Function ใน Simulink

4. กำหนดค่า Input เป็นค่าคงที่ 150 rad/s ให้กับ Reference Feedforward และกำหนดค่า Time Constant ขึ้นมาค่าหนึ่ง
5. กด Run จากนั้นดูกราฟ Output ของ Reference Feedforward เพื่อดูพฤติกรรมการตอบสนอง
6. ปรับค่า Time Constant จนกระทั่งลักษณะกราฟที่ได้ไม่มียอดแหลม และไม่ตอบสนองช้าจนเกินไป

ผลการทดลอง

- ค่า Time Constant เป็น 0.001



ภาพที่ 66 กราฟการตอบสนองของ Reference Feedforward ที่ค่า Time Constant เป็น 0.001 โดยมีแกน x เป็นเวลา (Sec)

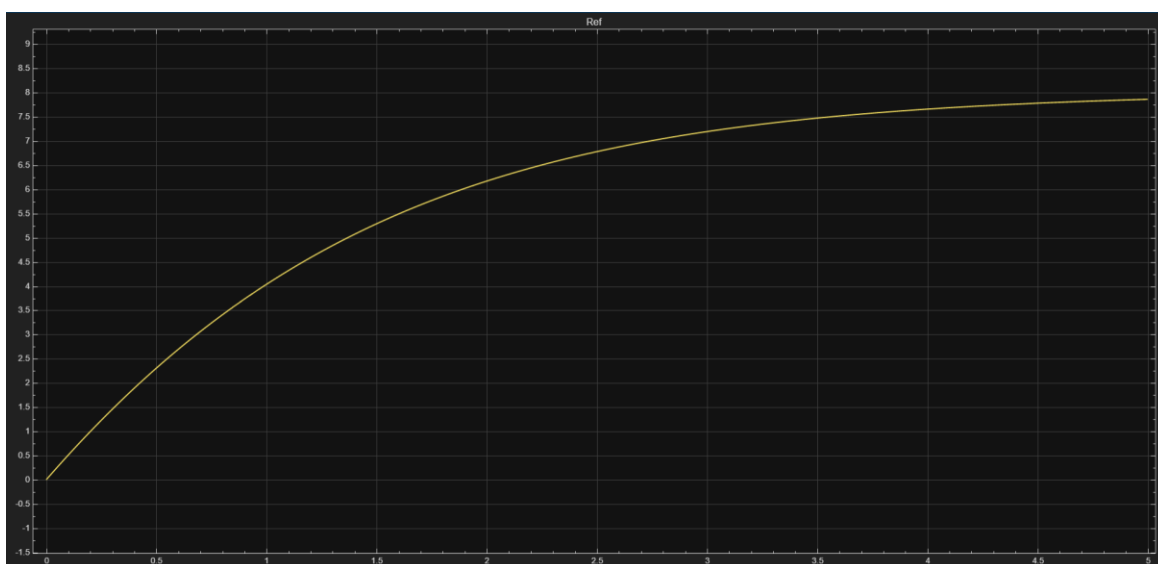
และมีแกน y เป็นแรงดันไฟฟ้า (Volt)

- ค่า Time Constant เป็น 0.1



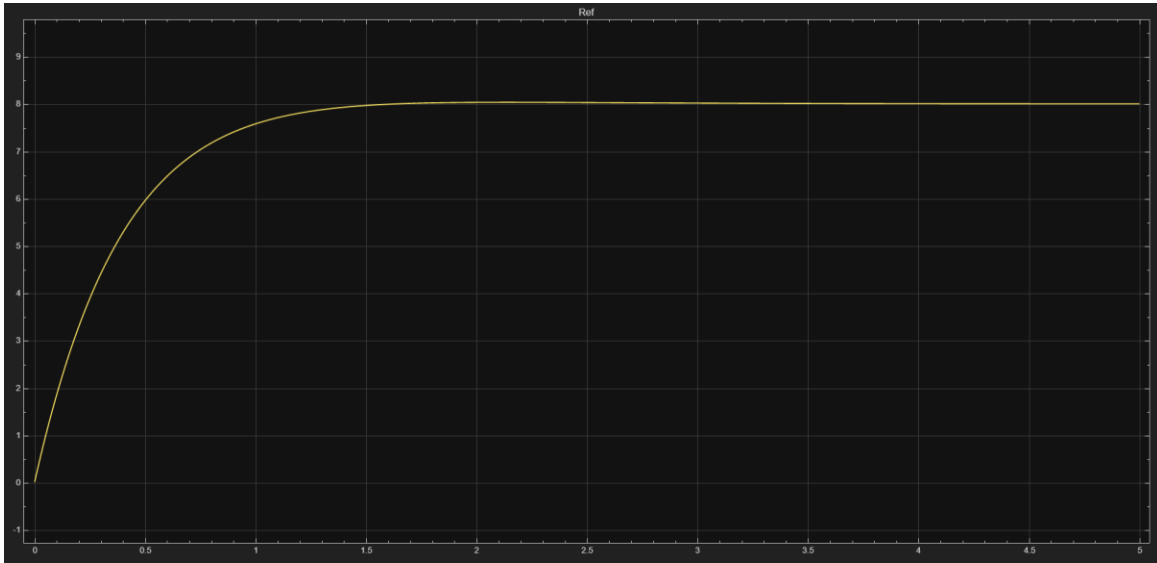
ภาพที่ 67 กราฟการตอบสนองของ Reference Feedforward ที่ค่า Time Constant เป็น 0.1 โดยมีแกน x เป็นเวลา (Sec) และมีแกน y เป็นแรงดันไฟฟ้า (Volt)

- ค่า Time Constant เป็น 1



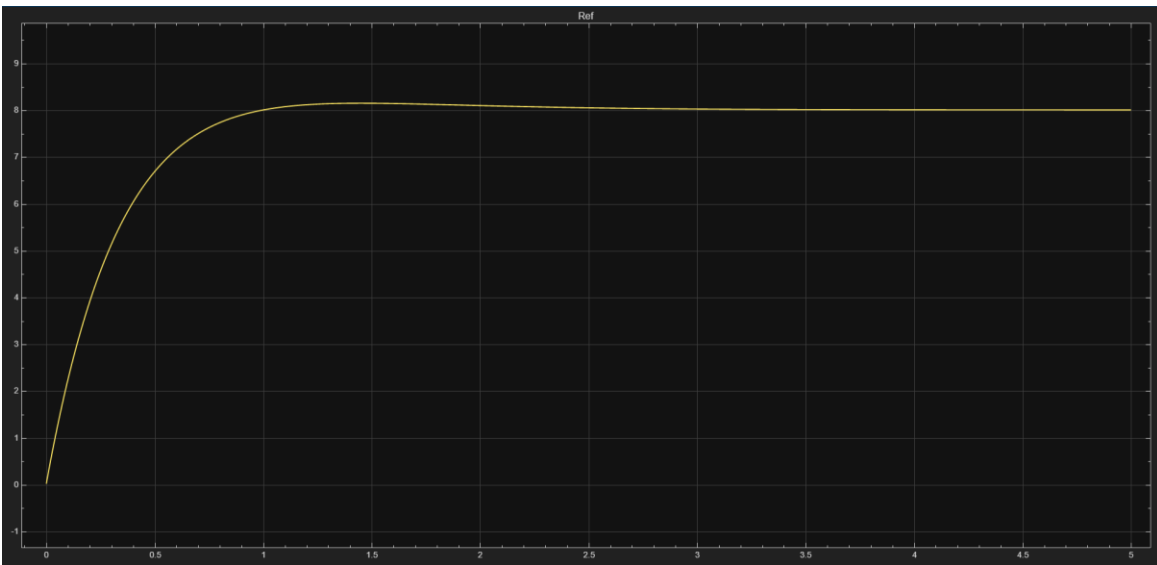
ภาพที่ 68 กราฟการตอบสนองของ Reference Feedforward ที่ค่า Time Constant เป็น 1 โดยมีแกน x เป็นเวลา (Sec) และมีแกน y เป็นแรงดันไฟฟ้า (Volt)

- ค่า Time Constant เป็น 0.5



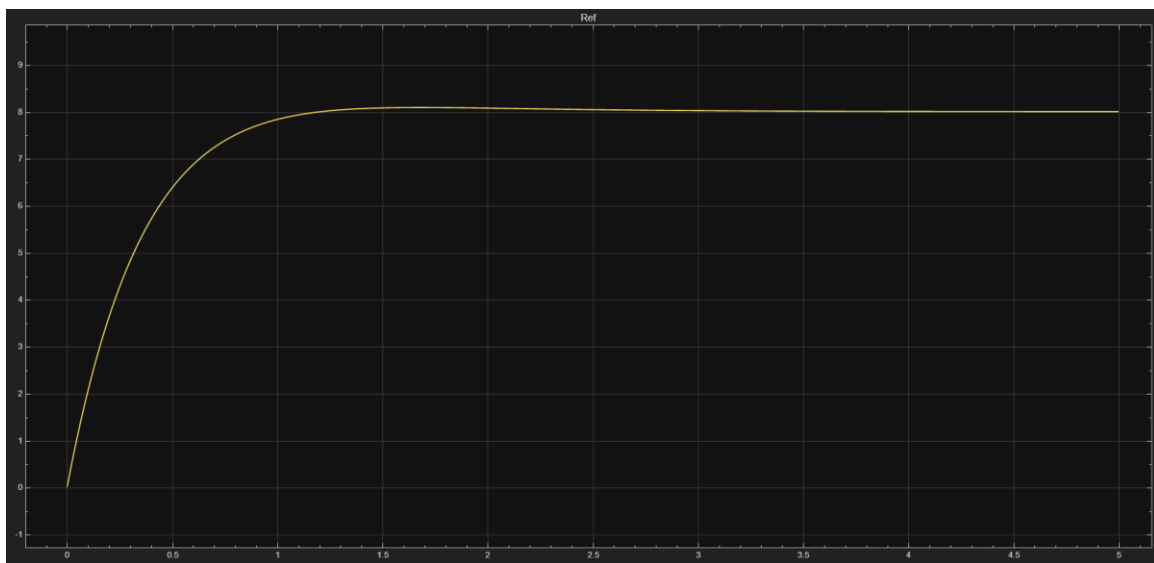
ภาพที่ 69 กราฟการตอบสนองของ Reference Feedforward ที่ค่า Time Constant เป็น 0.5 โดยมีแกน x เป็นเวลา (Sec) และมีแกน y เป็นแรงดันไฟฟ้า (Volt)

- ค่า Time Constant เป็น 0.45



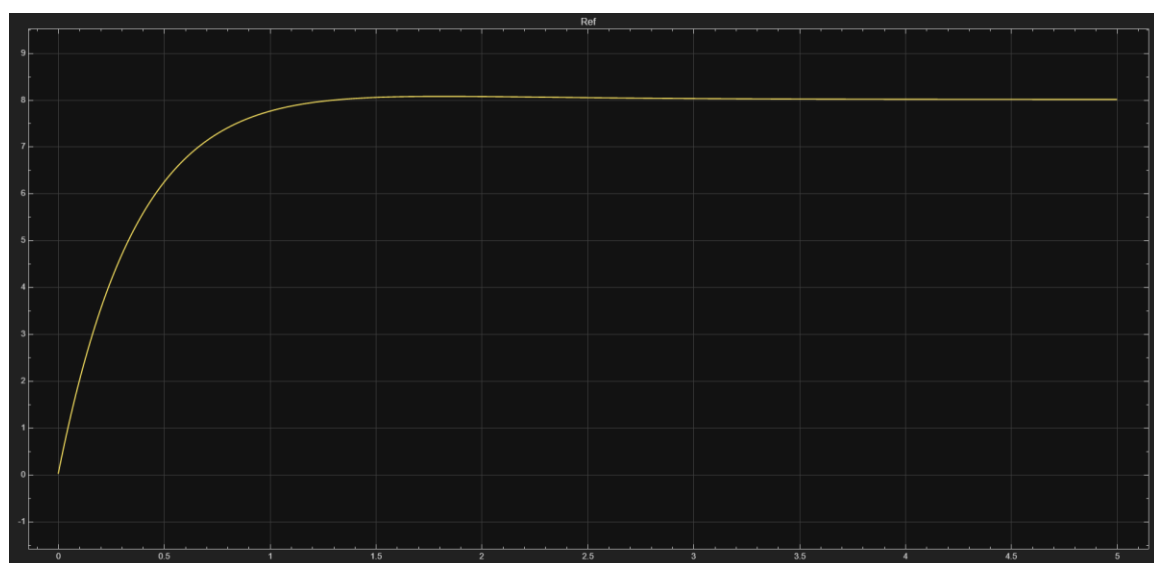
ภาพที่ 70 กราฟการตอบสนองของ Reference Feedforward ที่ค่า Time Constant เป็น 0.45 โดยมีแกน x เป็นเวลา (Sec) และมีแกน y เป็นแรงดันไฟฟ้า (Volt)

- ค่า Time Constant เป็น 0.47



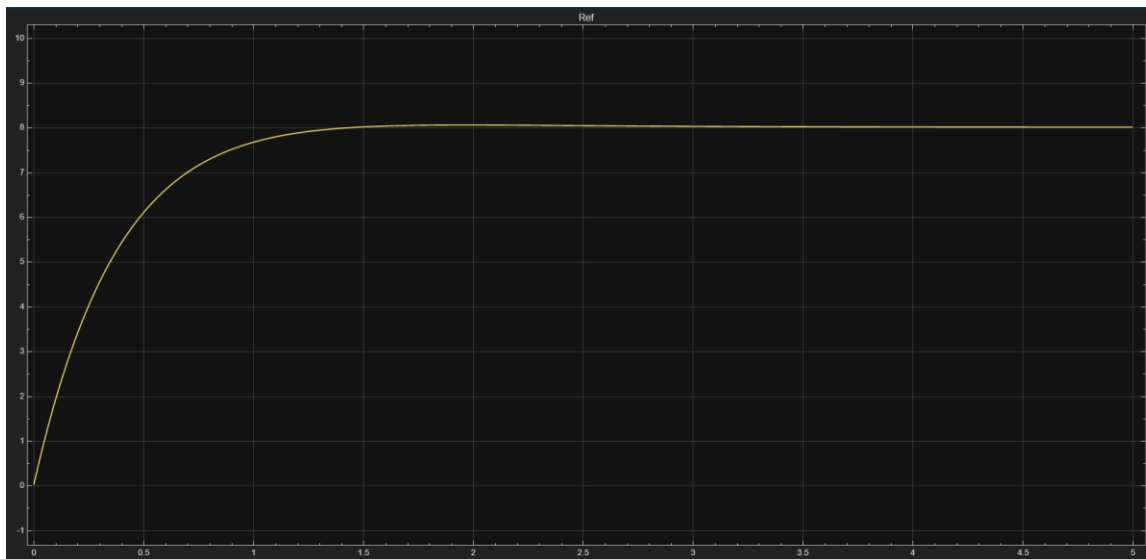
ภาพที่ 71 กราฟการตอบสนองของ Reference Feedforward ที่ค่า Time Constant เป็น 0.47 โดยมีแกน x เป็นเวลา (Sec) และมีแกน y เป็นแรงดันไฟฟ้า (Volt)

- ค่า Time Constant เป็น 0.48



ภาพที่ 72 กราฟการตอบสนองของ Reference Feedforward ที่ค่า Time Constant เป็น 0.48 โดยมีแกน x เป็นเวลา (Sec) และมีแกน y เป็นแรงดันไฟฟ้า (Volt)

- ค่า Time Constant เป็น 0.49



ภาพที่ 73 กราฟการตอบสนองของ Reference Feedforward ที่ค่า Time Constant เป็น 0.49 โดยมีแกน x เป็นเวลา (Sec) และมีแกน y เป็นแรงดันไฟฟ้า (Volt)

สรุปผลการทดลอง

จากการทดลองพบว่าค่า Time Constant ที่เหมาะสม คือ 0.5 วินาที เนื่องจากตั้งแต่ค่า 0.49 วินาทีลงไปจะเกิด Overshoot ขึ้น และค่าที่มากกว่า 0.5 Reference Feedforward จะมีการตอบสนองที่ช้าลง

อภิปรายผล

จากการสังเกตพฤติกรรมของกราฟเมื่อปรับค่า Time Constant จะได้ว่าหากกราฟ Output ของ Feedforward Controller มียอดแหลม หรือ Overshoot ปรากฏขึ้นจะต้องปรับค่า Time Constant ให้เพิ่มขึ้นเพื่อลดความถี่ที่ Feedforward Controller ต้องตอบสนองลง เนื่องจากยอดแหลมเกิดจากการที่ระบบตอบสนองต่อความถี่นั้น ๆ ไม่ทัน แต่หากกราฟที่ได้มีลักษณะโค้งมาก ไปถึงจุดที่ Saturation ได้ช้า จะต้องปรับค่า Time Constant ให้น้อยลงเพื่อเพิ่มความถี่ในการตอบสนองของ Feedforward Controller ให้มีการตอบสนองที่เร็วขึ้น

ข้อเสนอแนะ

1. ควรเริ่มต้นปรับค่า Time Constant จากค่าที่มากก่อน แล้วค่อยลดลงทีละน้อย เพื่อหลีกเลี่ยงการเกิด Overshoot ที่รุนแรงในช่วงแรกของการทดลอง
2. ค่า Time Constant ที่เหมาะสมควรสอดคล้องกับ Bandwidth ของระบบ หากทราบความถี่ธรรมชาติของระบบล่วงหน้าจะช่วยให้การปรับค่าทำได้รวดเร็วและแม่นยำยิ่งขึ้น

3. ในการนำ Reference Feedforward ไปใช้งานจริงบนฮาร์ดแวร์ ควรทดสอบค่า Time Constant ที่ได้จาก Simulation อีกครั้ง เนื่องจากพฤติกรรมของมอเตอร์จริงอาจมีความแตกต่างจากแบบจำลอง ส่งผลให้ค่า Time Constant ที่เหมาะสมอาจเปลี่ยนไปได้

อ้างอิง

Feedforward https://web.stanford.edu/class/archive/ee/ee392m/ee392m.1034/Lecture5_Feedfwrdr.pdf (2003)

An Implementation Recipe for Cascade Control [An Implementation Recipe for Cascade Control – Control Guru](#) (2015)

Closed Loop Tuning of Cascade Controllers Based on Setpoint Experiment [\(PDF\) Closed Loop Tuning of Cascade Controllers Based on Setpoint Experiment](#) (2020)

การทดลองที่ 4.2 การทดลองเพื่อออกแบบ Cascade Control ให้สามารถทำงานได้ตาม Requirement

จุดประสงค์

1. เพื่อให้สามารถออกแบบ Cascade Controller ได้
2. เพื่อให้สามารถปรับพารามิเตอร์ของ Cascade Controller ให้ทำตาม Requirement ได้
3. เพื่อให้สามารถวิเคราะห์ผลของการทำ Cascade Controller ได้
4. เพื่อให้สามารถวิเคราะห์ข้อดี และข้อจำกัดของ Cascade Controller ได้

สมมติฐาน

การที่ Cascade Control จะสามารถทำตาม Requirement ได้ Inner Loop และ Outer Loop ต้องใช้การควบคุมแบบ PID

ตัวแปร

1. ตัวแปรต้น:
 - 1.1. ค่า Parameter ของ Controller ทั้ง Inner Loop และ Outer Loop

2. ตัวแปรตาม:

- 2.1. พฤติกรรมของความเร็วเชิงมุมที่ Pendulum บนมอเตอร์ตอบสนอง
- 2.2. พฤติกรรมของตำแหน่งเชิงมุมที่ Pendulum บนมอเตอร์ตอบสนอง

3. ตัวแปรควบคุม:

- 3.1. Reference Feedforward Controller
- 3.2. Plant ของระบบ
- 3.3. การใส่ Gravity Compensation
- 3.4. เวลาที่ใช้เก็บค่าการตอบสนองของ Cascade Controller
- 3.5. สัญญาณ Reference Input ที่ใส่ให้กับระบบ

เอกสารและงานวิจัยที่เกี่ยวข้อง

Cascade Control เป็นโครงสร้างการควบคุมที่ใช้ตัวควบคุมสองตัวเชื่อมต่อกันแบบอนุกรม โดย Primary Controller (Outer Loop) จะกำหนด Setpoint ให้กับ Secondary Controller (Inner Loop) ระบบนี้ช่วยให้ตอบสนองต่อการรบกวนได้เร็วขึ้น โดยเฉพาะเมื่อมีการรบกวนเกิดขึ้นภายใน Secondary Loop

ระบบ Cascade Control ประกอบด้วย 2 ลูปหลัก:

1. Primary Loop (Outer Loop):

$$u_1(t) = K_{c1} \left[e_1(t) + \frac{1}{T_{i1}} \int e_1(t) dt + T_{d1} \frac{de_1(t)}{dt} \right]$$

โดยที่ $e_1(t) = SP_1 - PV_1$ และ $u_1(t) = SP_2$ (Output ของ Primary Controller คือ Setpoint ของ Secondary Controller)

2. Secondary Loop (Inner Loop):

$$u_2(t) = K_{c2} \left[e_2(t) + \frac{1}{T_{i2}} \int e_2(t) dt + T_{d2} \frac{de_2(t)}{dt} \right]$$

โดยที่ $e_2(t) = SP_2 - PV_2$ และ $u_2(t)$ คือสัญญาณควบคุมที่ส่งไปยัง Actuator

ข้อดีและข้อจำกัดของการควบคุมแบบ Cascade Control โดยข้อจำกัดหรือข้อควรระวังของการใช้งาน มีดังนี้ :

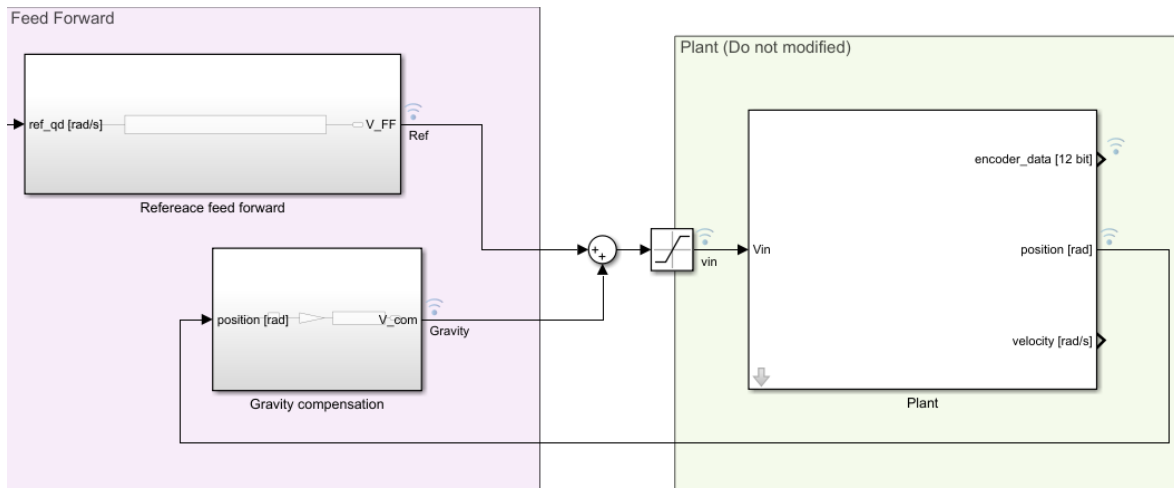
- จูนยากและซับซ้อน : ข้ามขั้นตอนไม่ได้ ต้องจูนลูบความเร็ว (ลูบใน) ให้เสถียรก่อนลูบตำแหน่ง (ลูบนอก) เสมอ และมีพารามิเตอร์ให้ปรับเยอะกว่าระบบลูบเดียว
- ข้อจำกัดด้านการตอบสนอง : ลูบความเร็วต้องเร็วกว่าลูบตำแหน่ง 5-10 เท่า หากกลไกมีข้อจำกัด (เช่น น้ำหนักหรือความเฉื่อยสูง) จะดึงให้ลูบตำแหน่งช้าตามไปด้วย
- มีความไวต่อสัญญาณรบกวน : การหาค่าความเร็วจาก Encoder ความละเอียดต่ำจะเกิด Noise สูง ทำให้มอเตอร์สั่น มีเสียงดัง และเกิดความร้อนสะสมได้
- ปัญหา Windup ซับซ้อน : เมื่อระบบจ่ายพลังงานจนถึง Saturation การมี I-term ในทั้งสองลูบจะทำให้ Error สะสมได้เยอะและแก้ยากกว่า ทำให้เสี่ยงต่อการเกิด Overshoot ที่มาก
- เกิดความล่าช้าสะสม (Phase Lag) : การต่อลูบควบคุมซ้อนกันจะเพิ่มความล่าช้าในระบบ ซึ่งอาจไปลดทอนความเสถียรจนทำให้ระบบเกิดการแกว่งได้ง่ายขึ้น

ต่อมาจะเป็นข้อดีของการใช้ Cascade Control มีดังนี้ :

- จัดการ Disturbance ได้ไว : ลูบความเร็วที่อยู่ด้านในจะจับความผิดปกติ (เช่น โหลดกระชาก หรือความผิด) และชดเชยแรงได้ทันที ก่อนที่ตำแหน่งจะคลาดเคลื่อนจนส่งผลกระทบต่อลูบนอก
- จำกัดความเร็วสูงสุดได้ง่ายและปลอดภัย : สามารถใส่บล็อกจำกัดค่า เช่น Saturation คั่นที่คำสั่งความเร็วที่มาจากเอาต์พุตของลูบตำแหน่งได้เลย ซึ่งสามารถช่วยป้องกันมอเตอร์หมุนเกินขีดจำกัดได้
- ลดผลกระทบจากความไม่เป็นเชิงเส้น : ลูบในที่จูนมาอย่างดีจะช่วยลดปัญหา Friction หรือ Deadband ของกลไกได้ ทำให้ลูบตำแหน่งทำงานได้ดีและแม่นยำขึ้น
- แบ่งปัญหาของการจูนได้ชัดเจน : แยกเป้าหมายออกจากกัน โดยเน้นให้ลูบในควบคุมความเร็วให้เสถียร และให้ลูบนอกเน้นพาไปถึงเป้าหมายโดยไม่ให้มี Overshoot ซึ่งทำให้ไม่สับสนพารามิเตอร์
- มีความเข้ากันได้กับ Feed Forward : โครงสร้างนี้รองรับการป้อนความเร็วล่วงหน้า เช่น Trapezoidal Profile ได้ง่าย และช่วยลด Error ระหว่างการเคลื่อนที่ได้จนเกือบเป็นศูนย์

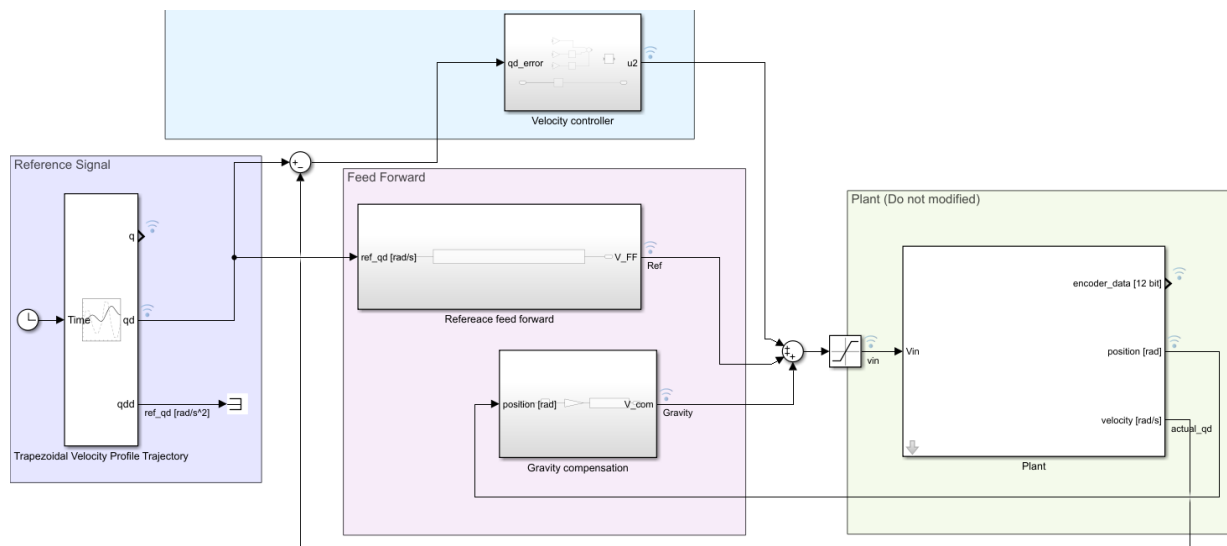
ขั้นตอนการดำเนินงาน

1. นำ Gravity Compensation และ Reference Feedforward Controller ที่ได้จากการทดลองก่อนหน้านี้มาต่อเข้ากับ Plant ดังภาพ



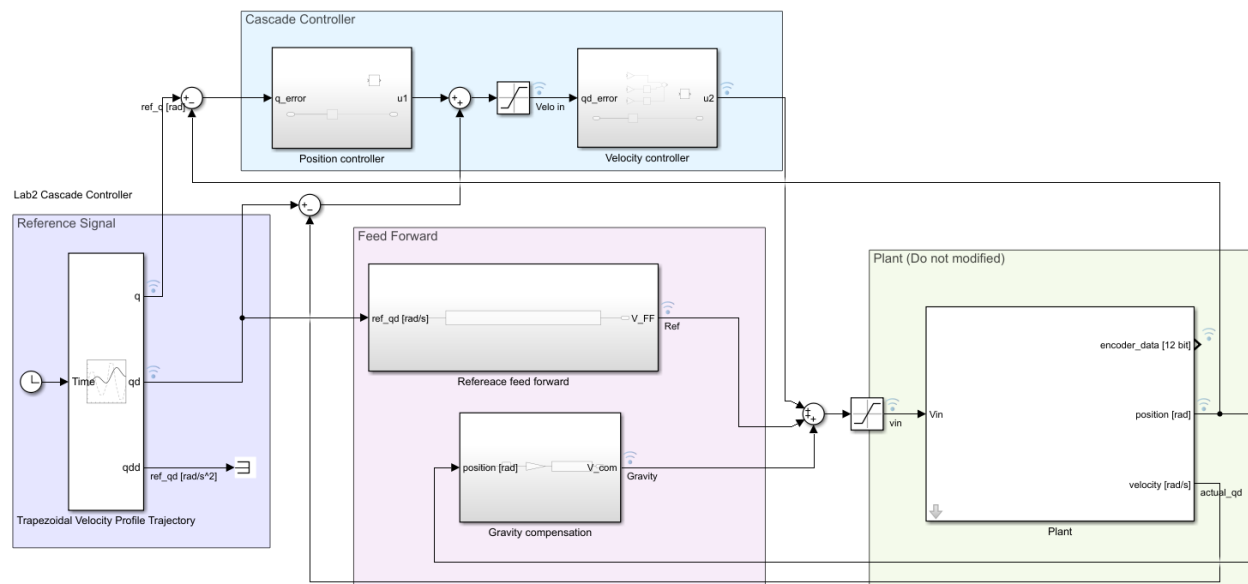
ภาพที่ 74 การต่อ Gravity Compensation และ Reference Feedforward Controller เข้ากับ Plant

2. สร้าง Velocity Controller และนำมาต่อเข้ากับ Plant ดังภาพ



ภาพที่ 75 การต่อ Velocity Controller เข้ากับ Plant

3. จาก Requirement ของความเร็วเชิงมุมที่ต้องการค่า Overshoot ไม่เกิน 2% และค่า Absolute Tracking Error น้อยกว่า 0.02 rad/s จึงต้องปรับค่า Parameter ของ Velocity Controller โดยเริ่มจากการใช้ P Controller ก่อน หากไม่สามารถทำตาม Requirement ได้ ต้องเพิ่มตัวควบคุมเป็น PI, PD หรือ PID Controller ตามความเหมาะสม จนกระทั่งระบบสามารถทำตาม Requirement ได้
4. ทดลองนำ Voltage Saturation Block ออกจากระบบเพื่อสังเกตพฤติกรรมของระบบว่าเปลี่ยนไปหรือไม่อย่างไร บันทึกผล และนำ Voltage Saturation Block มาต่อตามเดิม
5. สร้าง Position Controller และนำมาต่อเข้ากับ Velocity Controller ดังภาพ

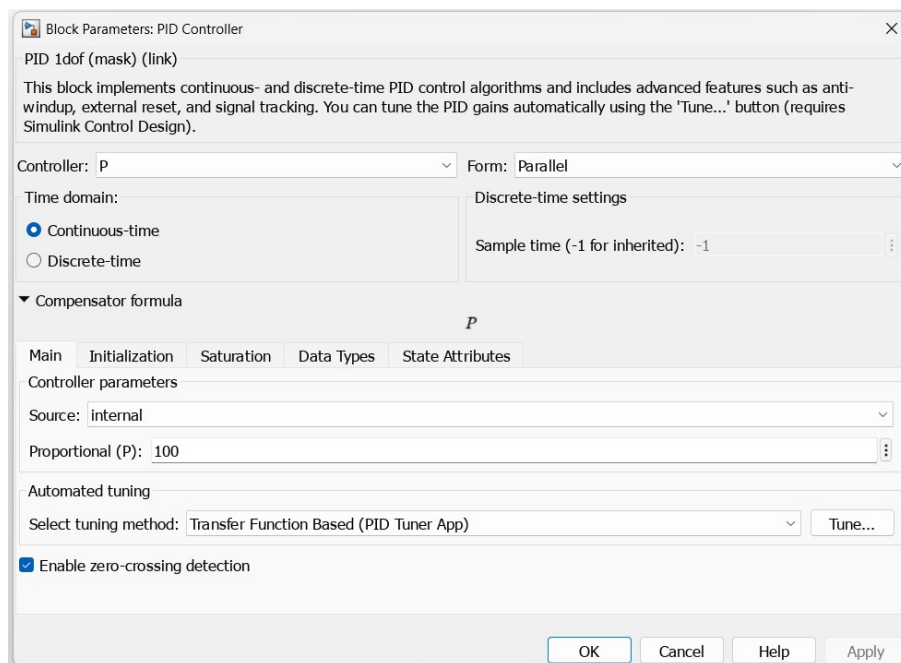


ภาพที่ 76 การต่อ Position Controller เข้ากับ Velocity Controller

6. จาก Requirement ของตำแหน่งเชิงมุมที่ต้องการค่า Overshoot ไม่เกิน 2% และค่า Absolute Tracking Error น้อยกว่า 0.004 rad จึงต้องปรับค่า Parameter ของ Position Controller โดยเริ่มจากการใช้ P Controller ก่อน หากไม่สามารถทำตาม Requirement ได้ ต้องเพิ่มตัวควบคุมเป็น PI, PD หรือ PID Controller ตามความเหมาะสม จนกระทั่งระบบสามารถทำตาม Requirement ได้
7. ทดลองนำ Velocity Saturation Block ออกจากระบบ จากนั้นนำ Voltage Saturation Block ออกจากระบบ จากนั้นนำ Velocity Saturation Block มาต่อตามเดิม เพื่อสังเกตพฤติกรรมของระบบว่าเปลี่ยนไปหรือไม่อย่างไร และบันทึกผล

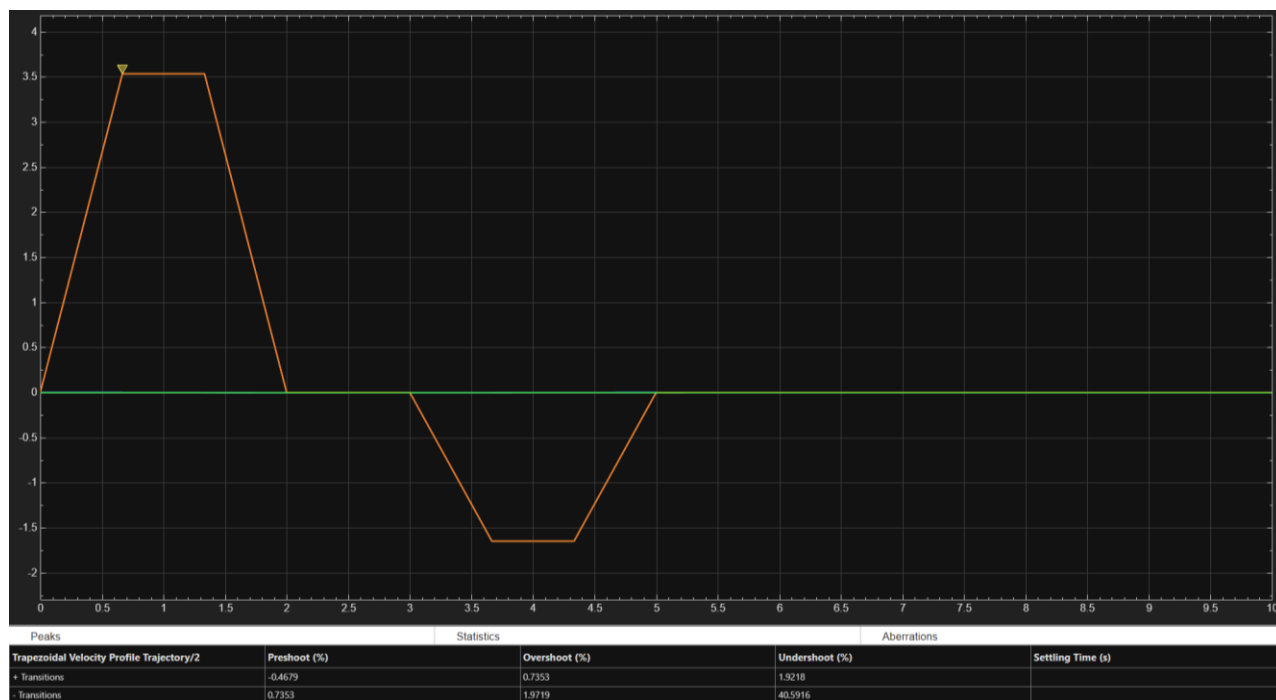
ผลการทดลอง

- ได้ค่า Parameter ของ Velocity Controller ที่สามารถทำงานได้ตาม Requirement โดยใช้ P Controller ดังนี้

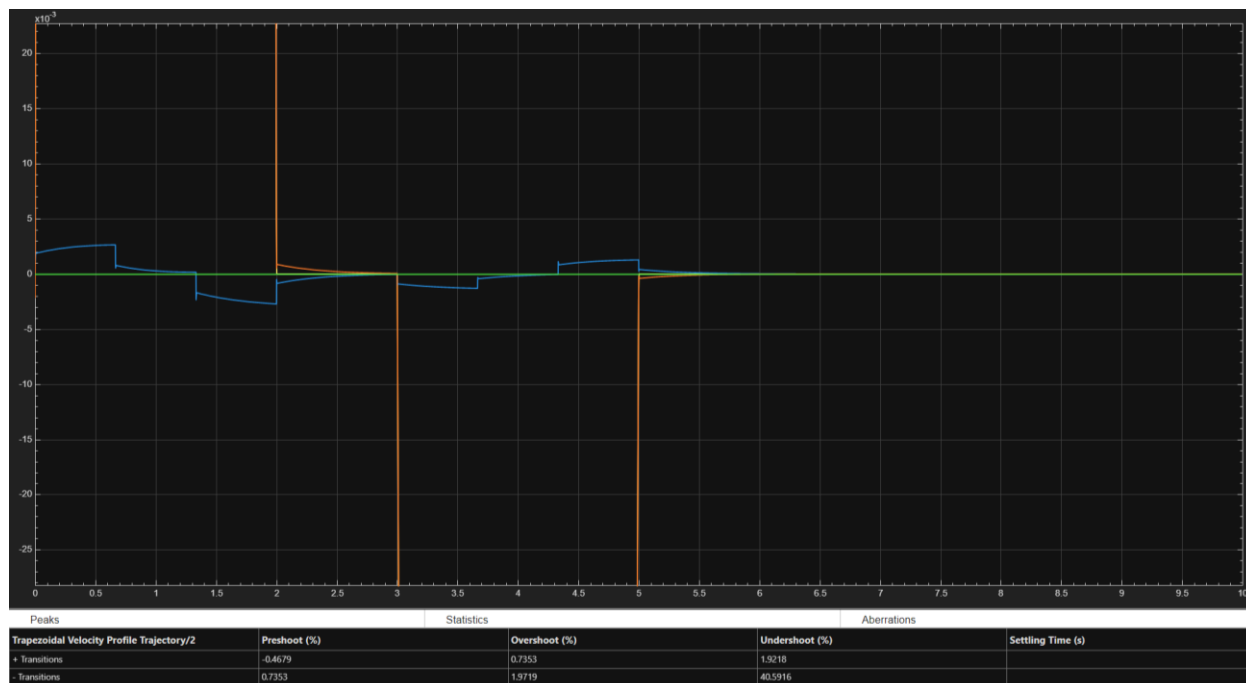


ภาพที่ 77 ค่า Parameter ของ Velocity Controller ที่ใช้ P Controller

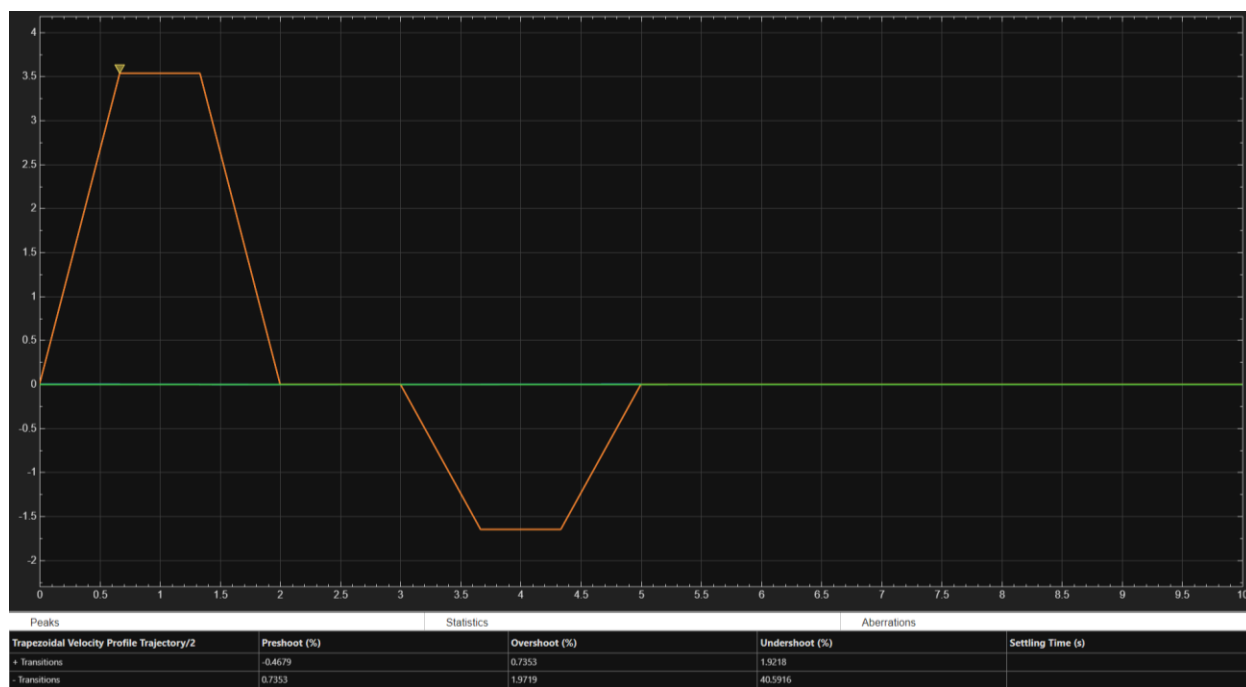
ซึ่งมีกราฟแสดงการตอบสนองของความเร็วเชิงมุมที่มีหน่วยเป็น เรเดียนต่อวินาที เทียบกับเวลาที่มีหน่วยเป็น วินาที ดังนี้



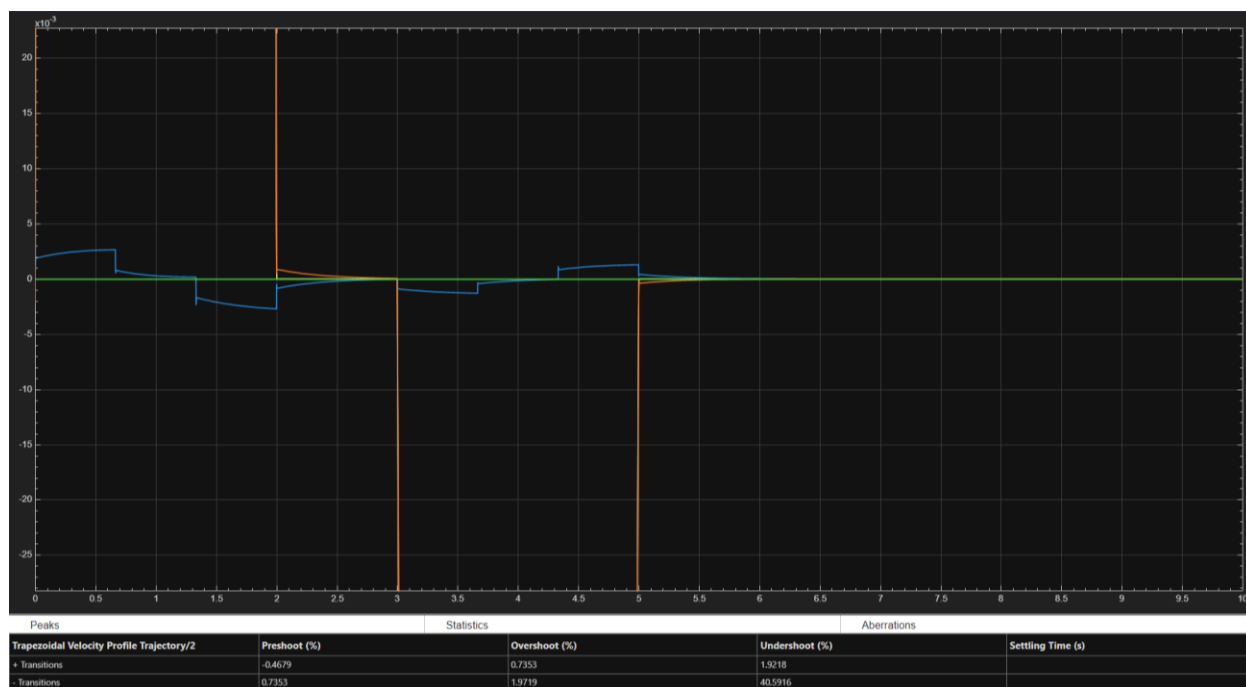
ภาพที่ 78 กราฟเส้นสีส้มที่แสดง Velocity Output ที่ Plant ตอบสนองได้ เทียบกับกราฟเส้นสีเหลืองที่แสดง Velocity Input ที่เป็น Reference ของระบบ แบบมี Voltage Saturation Block



ภาพที่ 79 กราฟเส้นสีฟ้าแสดง Error ระหว่าง Velocity Input ที่เป็น Reference ของระบบกับค่า Velocity Output ที่ Plant ตอบสนองได้ แบบมี Voltage Saturation Block



ภาพที่ 80 กราฟเส้นสีส้มที่แสดง Velocity Output ที่ Plant ตอบสนองได้ เทียบกับกราฟเส้นสีเหลืองที่แสดง Velocity Input ที่เป็น Reference ของระบบ แบบไม่มี Voltage Saturation Block



ภาพที่ 81 กราฟเส้นสีฟ้าแสดง Error ระหว่าง Velocity Input ที่เป็น Reference ของระบบกับค่า Velocity Output ที่ Plant ตอบสนองได้ แบบไม่มี Voltage Saturation Block

*หมายเหตุ ความหมายของเส้นแต่ละสีของ Velocity Controller มีดังนี้ : สีฟ้า คือ ค่า Velocity Error ที่เป็น Input ให้กับ Velocity Controller, สีเหลือง คือ ค่า Input ที่เป็น Reference Velocity ของระบบ, สีส้ม คือ ค่า Velocity Output ของ Plant

- และได้ค่า Parameter ของ Position Controller ที่สามารถทำงานได้ตาม Requirement โดยใช้ PID Controller ดังนี้

Block Parameters: PID Controller

PID 1dof (mask) (link)

This block implements continuous- and discrete-time PID control algorithms and includes advanced features such as anti-windup, external reset, and signal tracking. You can tune the PID gains automatically using the 'Tune...' button (requires Simulink Control Design).

Controller: PID Form: Parallel

Time domain:

☒ Continuous-time

☐ Discrete-time

Discrete-time settings

Sample time (-1 for inherited): -1

Compensator formula

$$P + I \frac{1}{s} + D \frac{N}{1 + N \frac{1}{s}}$$

Main Initialization Saturation Data Types State Attributes

Controller parameters

Source: internal

Proportional (P): 5

Integral (I): 10 ☐ Use I*Ts (optimal for codegen)

Derivative (D): 0.05 ☐ Use externally sourced derivative

Filter coefficient (N): 20 ☒ Use filtered derivative

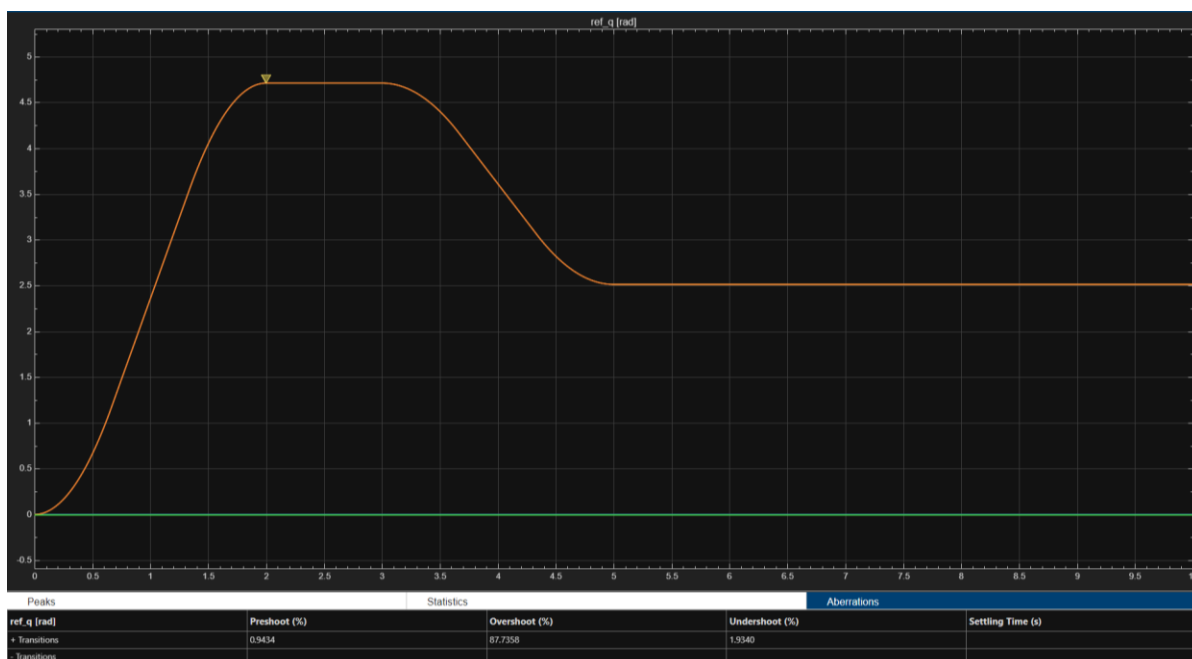
Automated tuning

Select tuning method: Transfer Function Based (PID Tuner App) Tune...

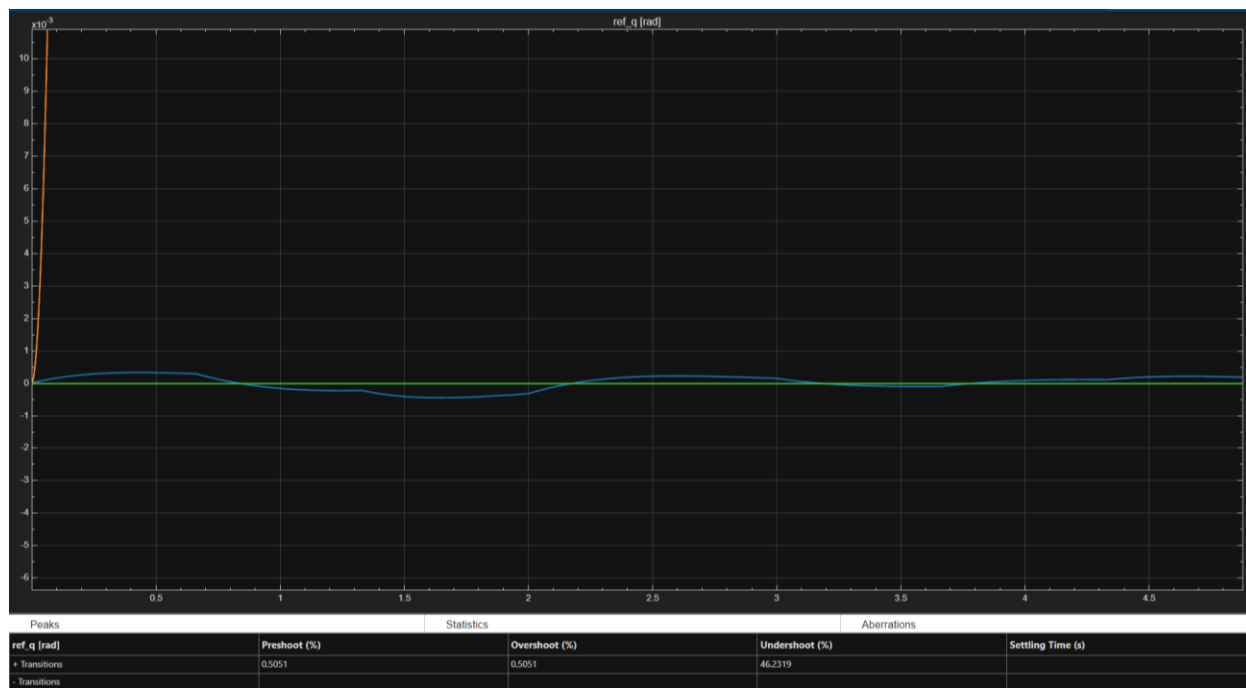
☒ Enable zero-crossing detection

OK Cancel Help Apply

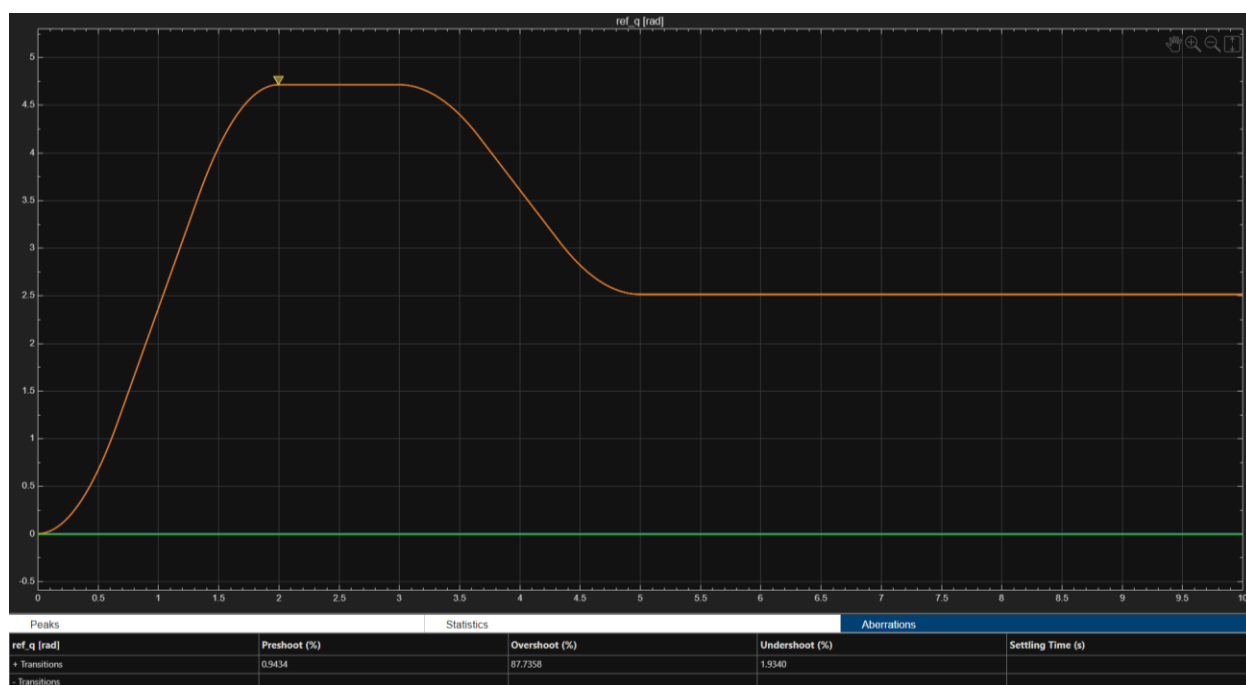
ภาพที่ 82 ค่า Parameter ของ Position Controller ที่ใช้ PID Controller



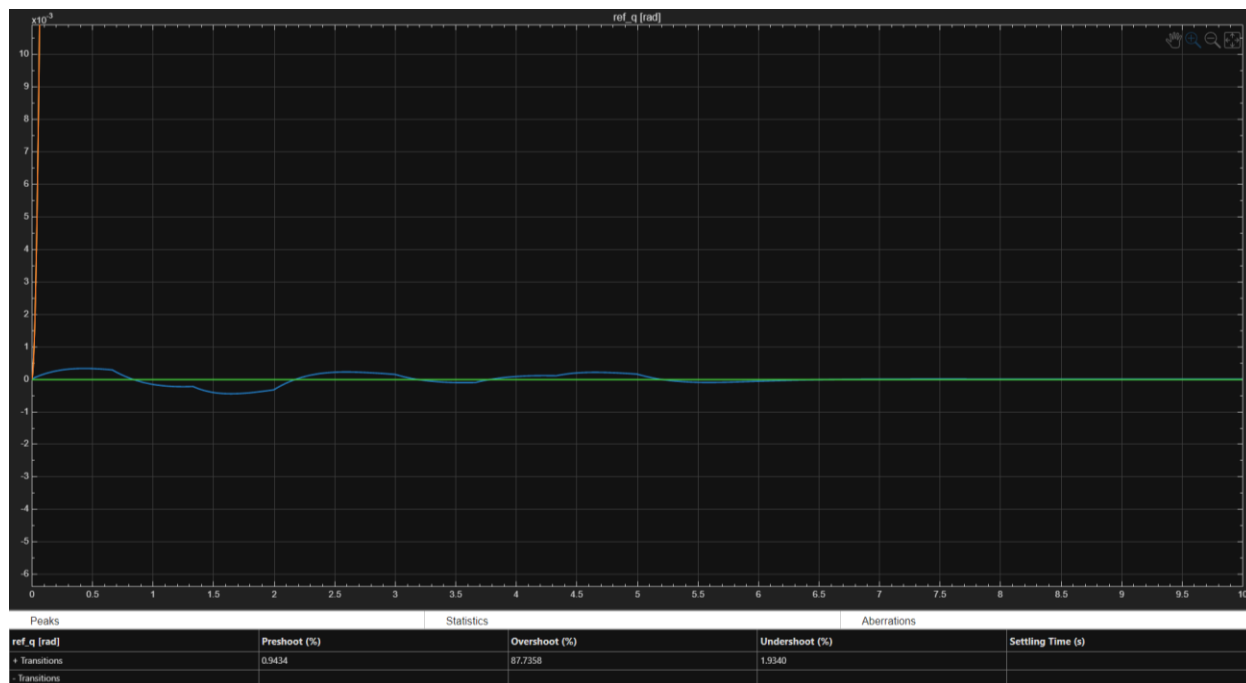
ภาพที่ 83 กราฟเส้นสีส้มที่แสดง Position Output ที่ Plant ตอบสนองได้ เทียบกับกราฟเส้นสีเหลืองที่แสดง Position Input ที่เป็น Reference ของระบบ แบบมี Voltage Saturation Block และ Velocity Saturation Block



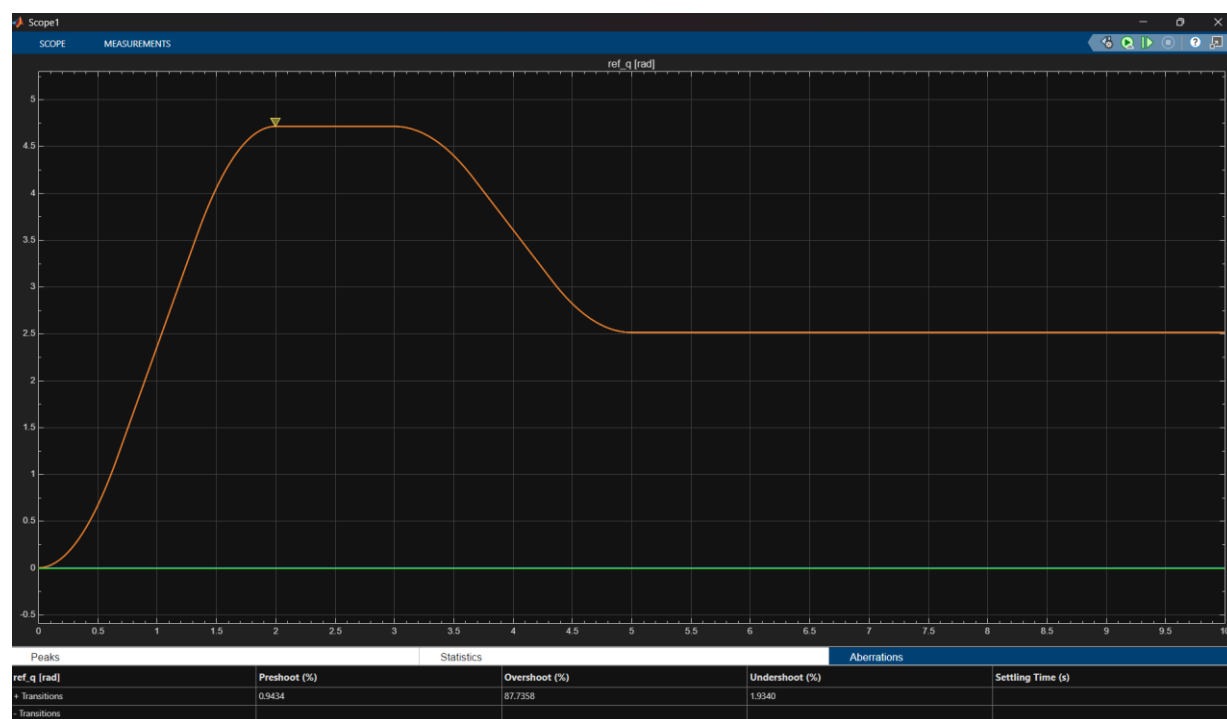
ภาพที่ 84 กราฟเส้นสีฟ้าแสดง Error ระหว่าง Position Input ที่เป็น Reference ของระบบกับค่า Position Output ที่ Plant ตอบสนองได้ แบบมี Voltage Saturation Block และ Velocity Saturation Block



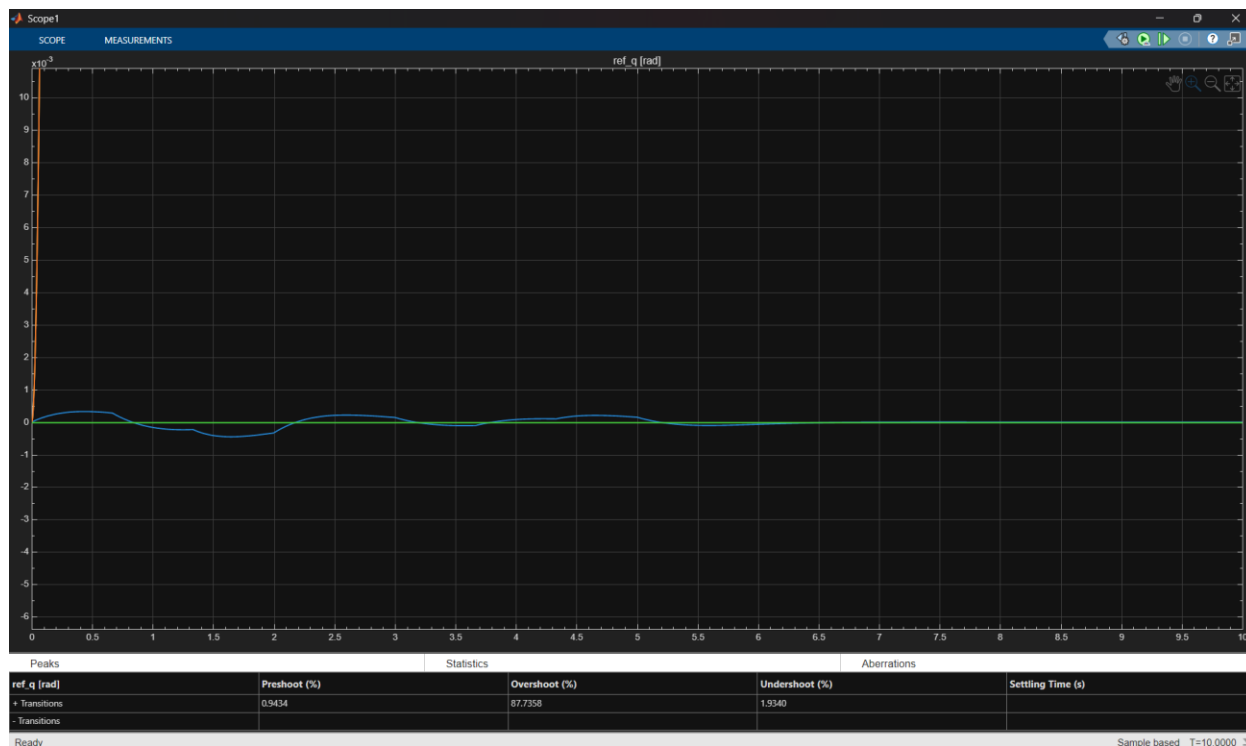
ภาพที่ 85 กราฟเส้นสีส้มที่แสดง Position Output ที่ Plant ตอบสนองได้ เทียบกับกราฟเส้นสีเหลืองที่แสดง Position Input ที่เป็น Reference ของระบบ แบบมี Voltage Saturation Block แต่ไม่มี Velocity Saturation Block



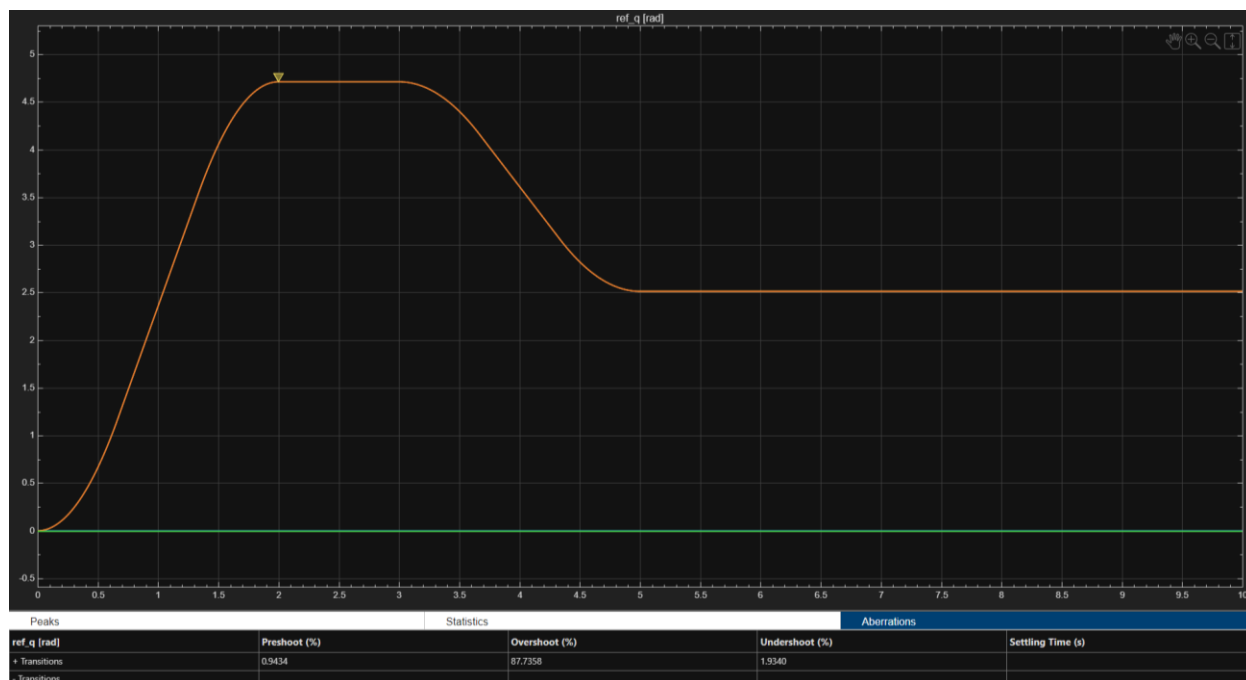
ภาพที่ 86 กราฟเส้นสีฟ้าแสดง Error ระหว่าง Position Input ที่เป็น Reference ของระบบกับค่า Position Output ที่ Plant ตอบสนองได้ แบบมี Voltage Saturation Block แต่ไม่มี Velocity Saturation Block



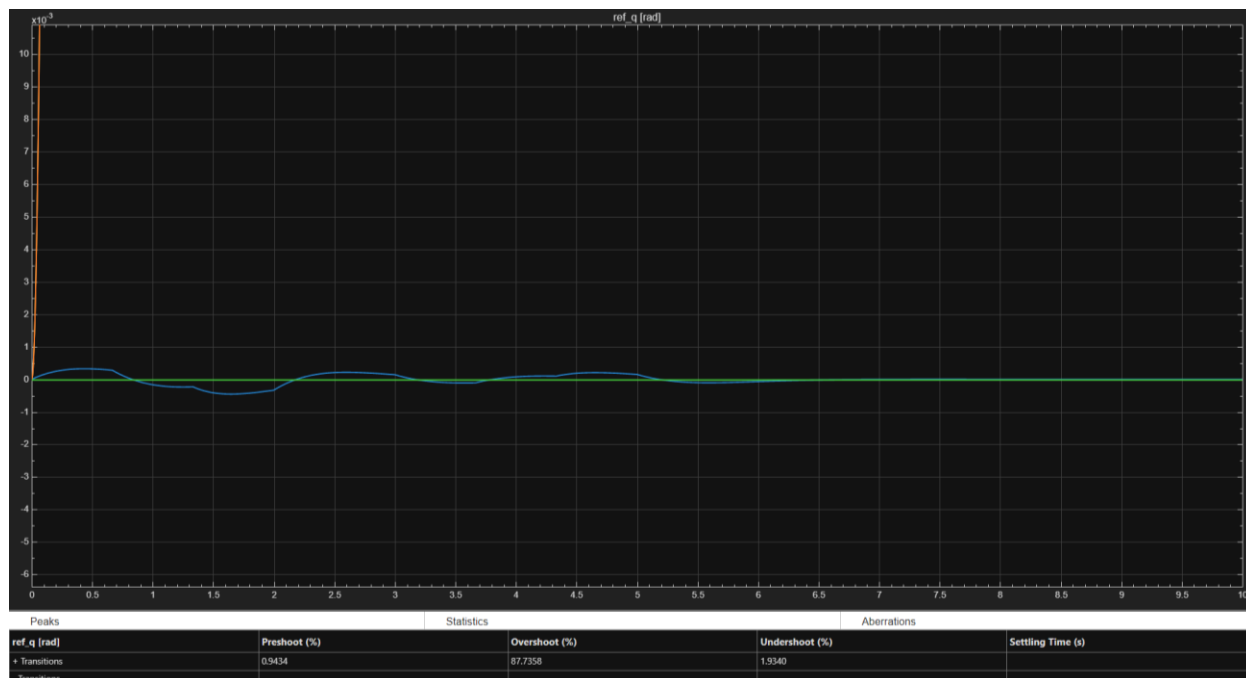
ภาพที่ 87 กราฟเส้นสีส้มที่แสดง Position Output ที่ Plant ตอบสนองได้ เทียบกับกราฟเส้นสีเหลืองที่แสดง Position Input ที่เป็น Reference ของระบบ แบบมี Velocity Saturation Block แต่ไม่มี Voltage Saturation Block



ภาพที่ 88 กราฟเส้นสีฟ้าแสดง Error ระหว่าง Position Input ที่เป็น Reference ของระบบกับค่า Position Output ที่ Plant ตอบสนองได้ แบบมี Velocity Saturation Block แต่ไม่มี Voltage Saturation Block



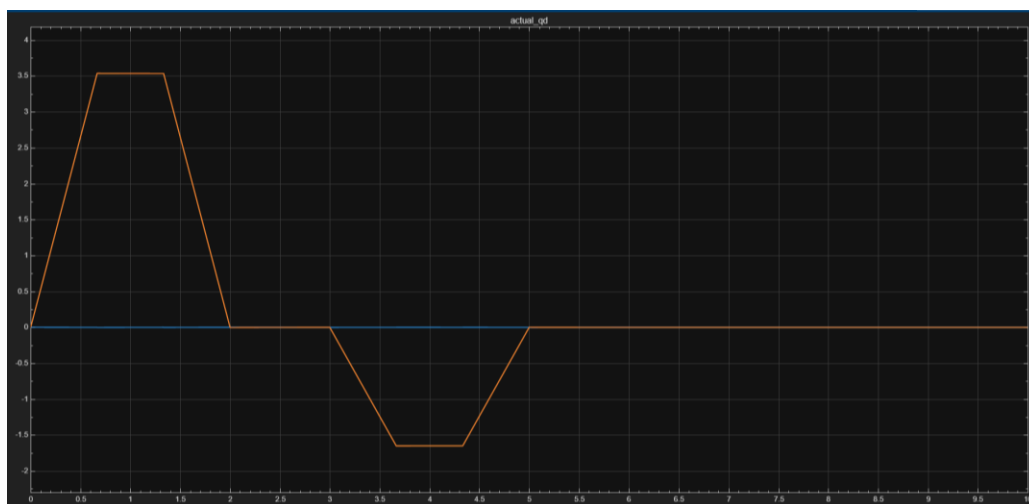
ภาพที่ 89 กราฟเส้นสีส้มที่แสดง Position Output ที่ Plant ตอบสนองได้ เทียบกับกราฟเส้นสีเหลืองที่แสดง Position Input ที่เป็น Reference ของระบบ แบบไม่มี Velocity Saturation Block และ Voltage Saturation Block



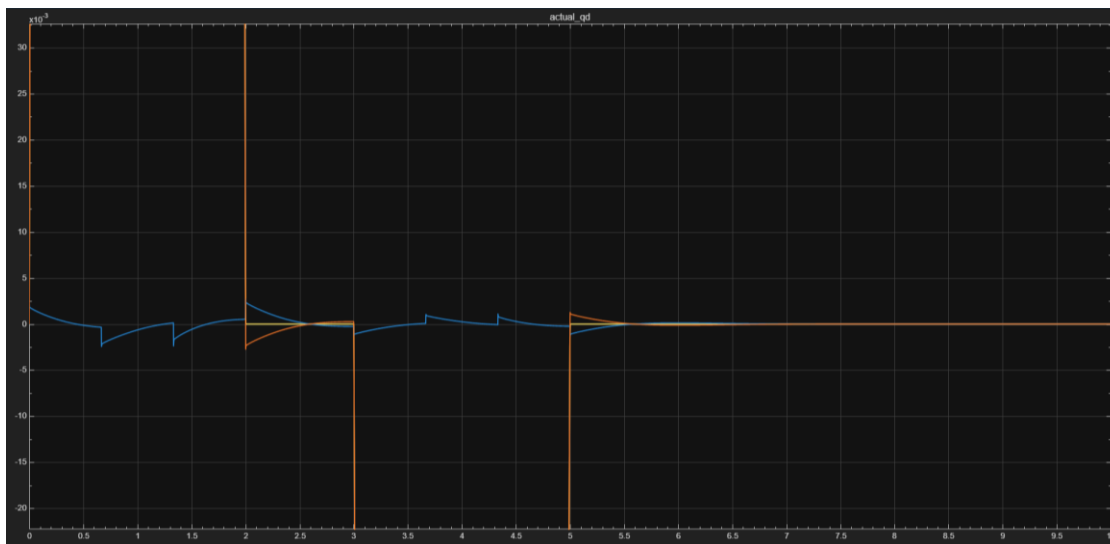
ภาพที่ 90 กราฟเส้นสีฟ้าแสดง Error ระหว่าง Position Input ที่เป็น Reference ของระบบกับค่า Position Output ที่ Plant ตอบสนองได้ แบบไม่มี Velocity Saturation Block และ Voltage Saturation Block

*หมายเหตุ ความหมายของเส้นแต่ละสีของ Position Controller มีดังนี้ : สีฟ้า คือ ค่า Position Error ที่เป็น Input ให้กับ Position Controller, สีเหลือง คือ ค่า Input ที่เป็น Reference Position ของระบบ, สีส้ม คือ ค่า Position Output ของ Plant

- เมื่อใช้ Velocity Controller, Position Controller, Velocity Saturation Block และ Voltage Saturation Block เพื่อดูพฤติกรรมการตอบสนองของ Velocity Output ของ Plant เทียบกับ Reference Velocity



ภาพที่ 91 กราฟเส้นสีส้มที่แสดง Velocity Output ที่ Plant ตอบสนองได้ เทียบกับกราฟเส้นสีเหลืองที่แสดง Velocity Input ที่เป็น Reference ของระบบ แบบไม่มี Velocity Saturation Block และ Voltage Saturation Block



ภาพที่ 92 กราฟเส้นสีฟ้าแสดง Error ระหว่าง Velocity Input ที่เป็น Reference ของระบบกับค่า Velocity Output ที่ Plant ตอบสนองได้ แบบมี Velocity Saturation Block และ Voltage Saturation Block

สรุปผลการทดลอง

Velocity Controller ใช้ P Controller ในการควบคุมให้สามารถทำงานตาม Requirement ที่ค่า Overshoot ไม่เกิน 2% และ Absolute Tracking Error น้อยกว่า 0.02 rad/s ได้ โดยใช้ค่า k_p เท่ากับ 100 และ Position Controller ใช้ PID Controller ในการควบคุมให้สามารถทำงานตาม Requirement ที่ค่า Overshoot ไม่เกิน 2% และ Absolute Tracking Error น้อยกว่า 0.004 rad ได้ โดยใช้ค่า k_p เท่ากับ 5, K_i เท่ากับ 10 และ k_d เท่ากับ 0.01

อภิปรายผล

ในการทำ Cascade Control ต้องทำการปรับค่า Parameter ของ Inner Loop ก่อน จากนั้นจึงค่อยปรับ Parameter ของ Outer Loop และ Controller ใน Cascade Control ไม่จำเป็นต้องเป็น PID เสมอไป

และจากการทดลองพบว่าการใส่ Saturation Block หรือไม่ได้ใส่ Saturation Block ไม่ได้ส่งผลให้การตอบสนองของระบบเกิดการเปลี่ยนแปลง แต่อย่างไรก็ตามควรมีการใส่ Saturation Block เพื่อลดโอกาสเกิดข้อผิดพลาดจากการจ่ายสัญญาณที่เกินขีดจำกัดของระบบ

ข้อเสนอแนะ

1. ควรกำหนดค่าใน Saturation Block ให้เท่ากับขีดจำกัดของระบบจริงเพื่อให้ระบบสามารถทำงานได้อย่างเต็มประสิทธิภาพ และไม่เกิดข้อผิดพลาด

อ้างอิง

PID Cascade Controller Design for an Unstable System [*PID Cascade Controller Design for an Unstable System*](#) (2022)

Designing Cascade Control System with PI Controllers [*Designing Cascade Control System with PI Controllers*](#) (2025)

Procedure for Cascade Control Systems Design: Choice of Suitable PID Tunings [*Procedure for Cascade Control Systems Design: Choice of Suitable PID Tunings*](#) (2008)

[*Designing Cascade Control System with PI Controllers - MATLAB & Simulink*](#)

การทดลองที่ 4.3 การทดลองศึกษาผลจากการปรับความถี่ในการคำนวณของ Cascade Control

จุดประสงค์

1. เพื่อให้เข้าใจพฤติกรรมการตอบสนองของ Cascade Control เมื่อปรับความถี่ในการคำนวณ Inner Loop และ Outer Loop
2. เพื่อให้สามารถเปรียบเทียบความถี่ในการคำนวณ Inner Loop และ Outer Loop ได้
3. เพื่อให้สามารถวิเคราะห์ผลของการปรับความถี่ในการคำนวณ Inner Loop และ Outer Loop ได้

สมมติฐาน

ค่าความถี่ของการคำนวณทั้ง Inner Loop และ Outer Loop ควรเท่ากัน

ตัวแปร

1. ตัวแปรต้น:
 - 1.1. ค่า Sample Time ของ Inner Loop และ Outer Loop
2. ตัวแปรตาม:

- 2.1. พฤติกรรมการตอบสนองของ Pendulum บนมอเตอร์
3. ตัวแปรควบคุม:
 - 3.1. Reference Feedforward Controller
 1. Plant ของระบบ
 2. การใส่ Gravity Compensation
 3. สัญญาณ Reference Input ที่ใส่ให้กับระบบ
 4. Parameter ใน Controller ของ Inner Loop และ Outer Loop

เอกสารและงานวิจัยที่เกี่ยวข้อง

Cascade Control เป็นโครงสร้างการควบคุมที่ใช้ตัวควบคุมสองตัวเชื่อมต่อกันแบบอนุกรม โดย Primary Controller (Outer Loop) จะกำหนด Setpoint ให้กับ Secondary Controller (Inner Loop) ระบบนี้ช่วยให้ตอบสนองต่อการรบกวนได้เร็วขึ้น โดยเฉพาะเมื่อมีการรบกวนเกิดขึ้นภายใน Secondary Loop

ระบบ Cascade Control ประกอบด้วย 2 ลูปหลัก:

1. Primary Loop (Outer Loop):

$$u_1(t) = K_{c1} \left[e_1(t) + \frac{1}{T_{i1}} \int e_1(t) dt + T_{d1} \frac{de_1(t)}{dt} \right]$$

โดยที่ $e_1(t) = SP_1 - PV_1$ และ $u_1(t) = SP_2$ (Output ของ Primary Controller คือ Setpoint ของ Secondary Controller)

2. Secondary Loop (Inner Loop):

$$u_2(t) = K_{c2} \left[e_2(t) + \frac{1}{T_{i2}} \int e_2(t) dt + T_{d2} \frac{de_2(t)}{dt} \right]$$

โดยที่ $e_2(t) = SP_2 - PV_2$ และ $u_2(t)$ คือสัญญาณควบคุมที่ส่งไปยัง Actuator

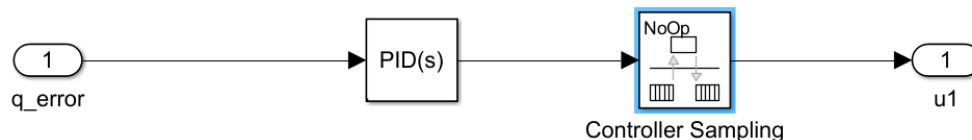
ขั้นตอนการดำเนินงาน

1. ต่อ Controller Sampling Block เข้าไปใน Inner Loop



ภาพที่ 93 การต่อ Controller Sampling Block ใน Inner Loop

2. ต่อ Controller Sampling Block เข้าไปใน Outer Loop



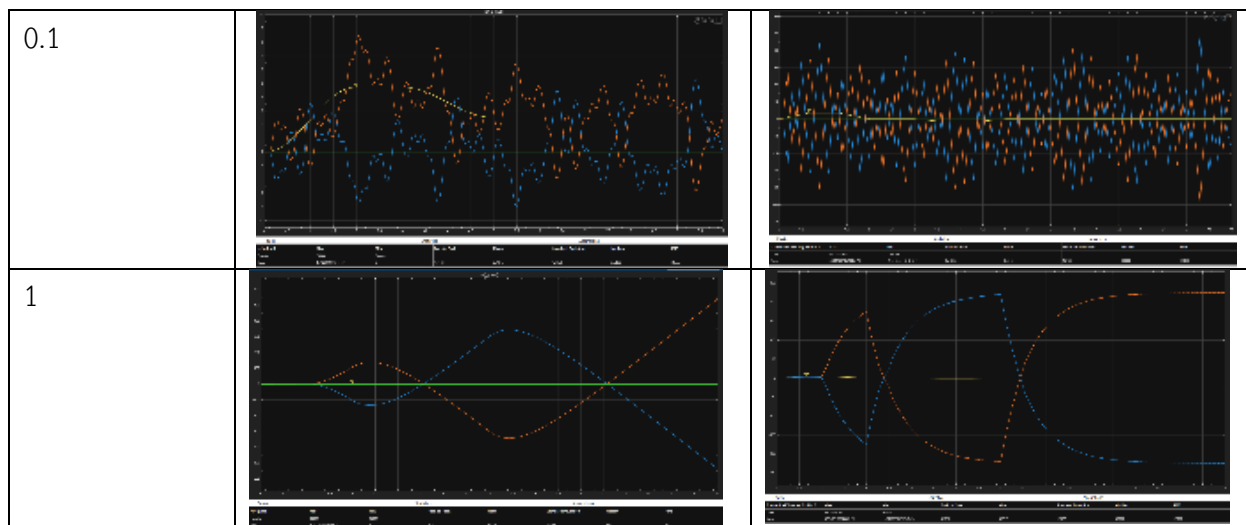
ภาพที่ 94 การต่อ Controller Sampling Block ใน Outer Loop

3. ปรับค่า Sample Time ของ Controller Sampling Block ใน Inner Loop และ Outer Loop โดยปรับค่าให้ Sample Time ของ Inner Loop มากกว่า, น้อยกว่า หรือเท่ากับ Outer Loop
4. ดูพฤติกรรมการตอบสนองของระบบที่เกิดจากความถี่ต่าง ๆ
5. บันทึกผลการทดลอง

ผลการทดลอง

ตารางที่ 23 การตอบสนอง (ความเร็วเชิงมุม และตำแหน่งเชิงมุม) ของระบบที่มีความถี่ต่าง ๆ เมื่อความถี่ของ Inner Loop และ Outer Loop เท่ากัน

ความถี่ของ Inner Loop และ Outer Loop	Position	Velocity
0.001		
0.01		



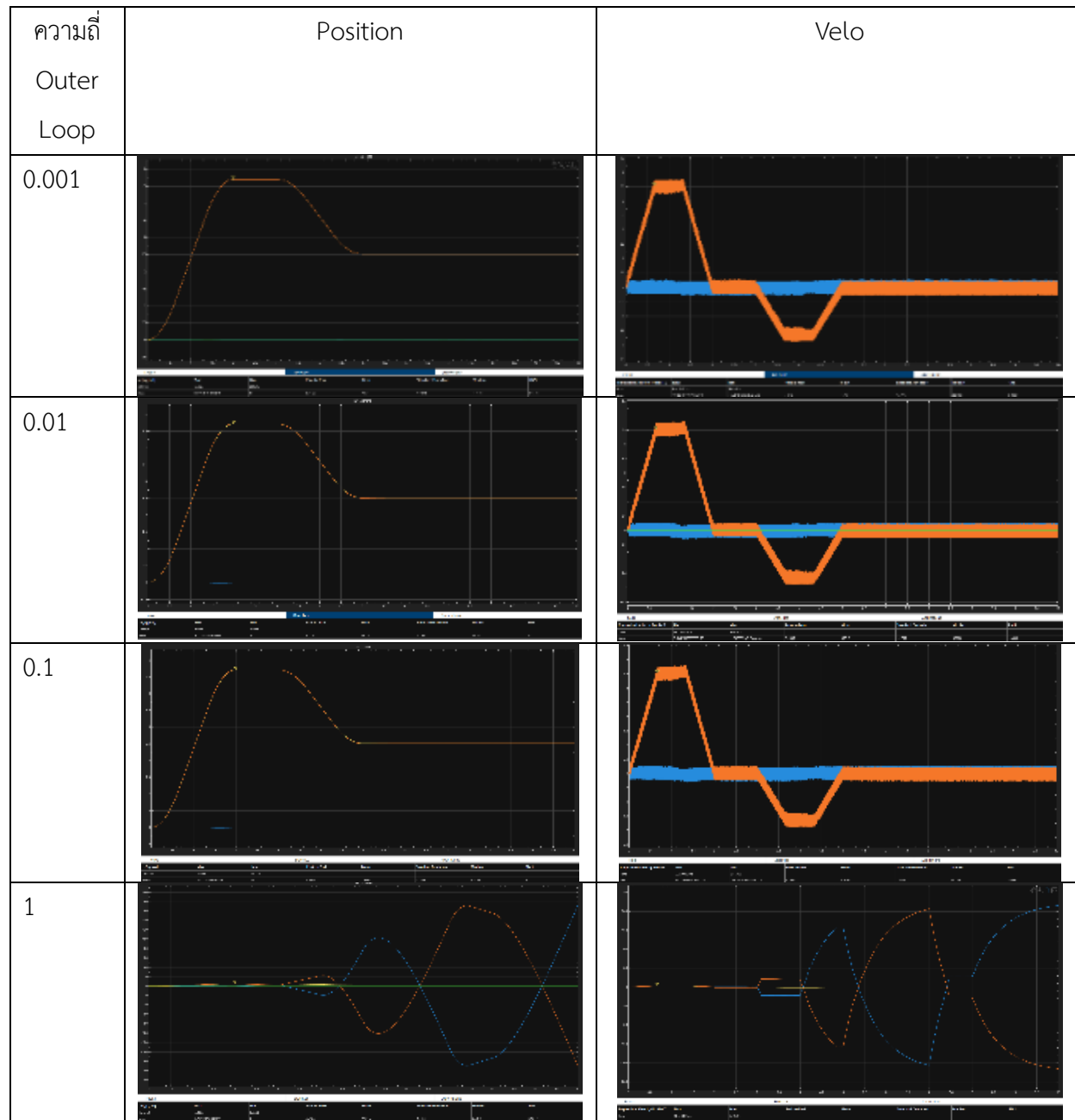
ตารางที่ 24 การตอบสนอง (ความเร็วเชิงมุม และตำแหน่งเชิงมุม) ของระบบที่ความถี่ต่าง ๆ เมื่อความถี่ของ Inner Loop น้อยกว่า Outer Loop

-Outer Loop ความถี่ 0.001

ความถี่ Inner Loop	Position	Velo
0.001		
0.01		
0.1		
1		

ตารางที่ 25 การตอบสนอง (ความเร็วเชิงมุม และตำแหน่งเชิงมุม) ของระบบที่ความถี่ต่าง ๆ เมื่อความถี่ของ Inner Loop มากกว่า Outer Loop

- Inner Loop ความถี่ 0.001



สรุปผลการทดลอง

จากตารางเปรียบเทียบผลการทดลองแสดงให้เห็นว่าการปรับเวลาตอบสนองของ Velocity มีผลต่อสัญญาณอย่างเห็นได้ชัด เนื่องจากเป็น Secondary Loop (Inner Loop) ทำให้การตอบสนองที่ช้ากว่าเพียงเล็กน้อยส่งผลต่อสัญญาณมาก

แต่ไม่ใช่ว่าการปรับ Primary Loop (Outer Loop) ไม่ส่งผลต่อสัญญาณ เห็นได้ว่าในรูปในตารางข้างต้น สัญญาณเริ่มมีการเปลี่ยนแปลงเมื่อเวลาตอบสนองของ Primary Loop ช้ามาก ๆ เช่นกัน

อภิปรายผล

ความถี่การตอบสนองของ Inner Loop ควรมากกว่า Outer Loop เพื่อให้ Outer Loop ตอบสนองได้เร็ว โดยไม่ต้องรอให้ Inner Loop ตอบสนอง

ข้อเสนอแนะ

1. ควรกำหนดค่า Sample Time ของ Inner Loop ให้เร็วกว่า Outer Loop อย่างน้อย 5–10 เท่า เพื่อให้ระบบมีเสถียรภาพ
2. ควรหลีกเลี่ยงการตั้งค่า Sample Time ของ Inner Loop ให้ช้ากว่า Outer Loop เนื่องจากจะทำให้ระบบเกิด Oscillation และไม่สามารถ Converge ได้
3. หากนำไปใช้งานบนฮาร์ดแวร์จริง ควรคำนึงถึงข้อจำกัดด้านความสามารถในการประมวลผลของ Microcontroller ด้วย

อ้างอิง

Designing Cascade Control System with PI Controllers [*Designing Cascade Control System with PI Controllers*](#) (2025)

Procedure for Cascade Control Systems Design: Choice of Suitable PID Tunings [*Procedure for Cascade Control Systems Design: Choice of Suitable PID Tunings*](#) (2008)

The Pros and Cons of Cascade Control [*The Pros and Cons of Cascade Control - Control Station*](#) (2017)

PID Cascade Controller Design for an Unstable System [*PID Cascade Controller Design for an Unstable System*](#) (2022)

ภาคผนวก ก

การทดลองที่ 1

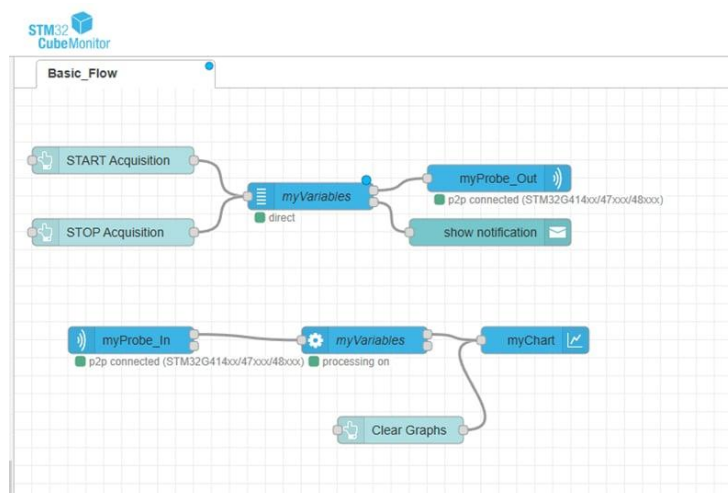
1. Control Design for DC Motor in STM32G474RE

การทดลองที่ 1 การทดลองเพื่อศึกษาผลที่เกิดจากการเปลี่ยนความถี่การจ่ายแรงดันให้กับแบบจำลอง DC Motor ใน STM32G474RE

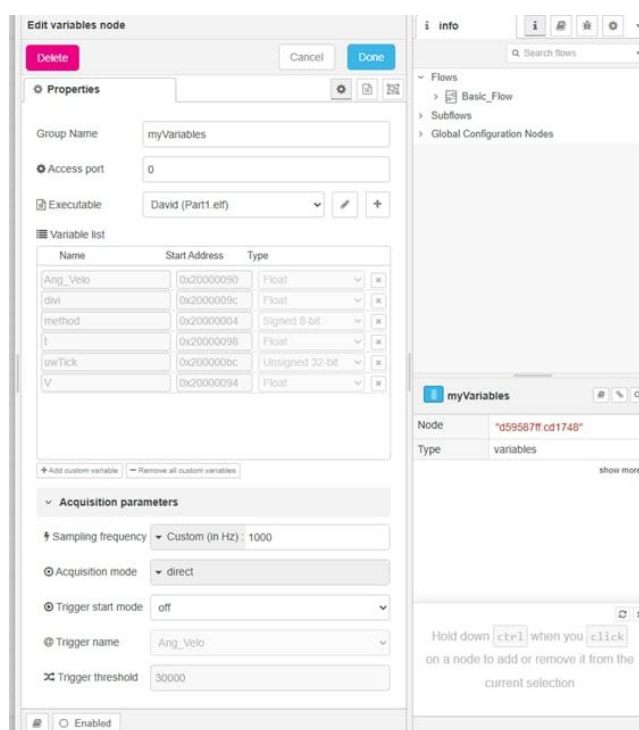
```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim == &htim2) { // สมมติใช้ Timer 2 เช็ตไว้ที่ 1ms
        t += Ts;
        if (t <= 20) {
            // if (t<1) {
            //     V=0;
            // }
            // else {
            //     V=t-1;
            // }
            if (V > 12.0f) V = 12.0f;
            if (V < -12.0f) V = -12.0f;
            V = A * (sinf(2.0f * Pi * f * t));
            Ang_Velo = Apply_Filter(V);
        } else {
            return;
        }
    }
}
```

ภาพที่ 95

โค้ดนี้เป็น Callback Function ที่ถูกเรียกตามความถี่การทำงานที่ตั้งไว้ใน Timer 2 โดยมีการทำงานดังนี้ เมื่อ Timer 2 หมดเวลา จะทำการเพิ่มค่าเวลา t ด้วย Ts จากนั้นตรวจสอบว่า $t \leq 20$ วินาทีหรือไม่ หากยังอยู่ในช่วงเวลาที่กำหนด จะคำนวณแรงดัน Input แบบ Sine Wave ด้วยสมการ $V = A * \sin(2\pi * f * t)$ โดยที่ A คือ Amplitude และ f คือความถี่ที่ต้องการ จากนั้นนำค่า V ที่ได้ไปคำนวณความเร็วเชิงมุมผ่านฟังก์ชัน Apply_Filter() ซึ่งเป็นสมการ Discretization ของแบบจำลอง DC Motor แล้วเก็บผลลัพธ์ไว้ใน Ang_Velo หากเวลาเกิน 20 วินาที จะหยุดการทำงานด้วย return นอกจากนี้ยังมีโค้ดส่วนของสัญญาณ Ramp ที่ถูก Comment ไว้ ซึ่งจะให้ $V = 0$ ในช่วง 1 วินาทีแรก จากนั้นจะเพิ่มค่าตามเวลาแบบ Ramp ที่มี Slope = 1 โดยจำกัดค่าแรงดันไว้ที่ ± 12 โวลต์เพื่อป้องกันแรงดันเกินทำงานด้วย



ภาพที่ 96



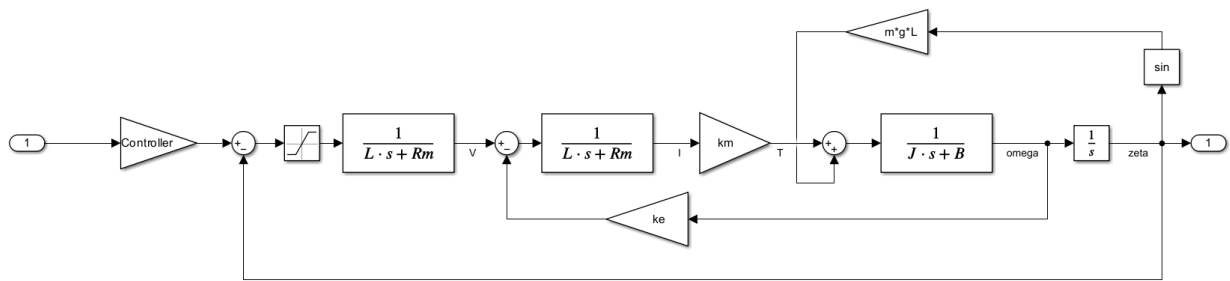
ภาพที่ 97

เป็นตัวบอกค่าความถี่ในการเก็บตัวอย่างของตัวมอเตอร์จำลอง โดยจะปรับให้ความถี่ที่บันทึกตรงกับ ความถี่มอเตอร์จำลองให้เท่ากัน

ภาคผนวก ข

การทดลองที่ 2.1 : คำนวณ Disturbance Feed Forward

Transfer Function ของระบบ



ภาพที่ 98

1. กำหนด Plant ของ Block Diagram ของระบบดังรูปต่อไปนี้

กำหนด Transfer Function ของระบบซึ่งสามารถเขียนเป็นสมการได้ดังต่อไปนี้

$$G(s) = \frac{\theta(s)}{V(s)}$$

ให้ผลจาก Gravity ค่าเป็น Disturbance τ_{load} โดยจะสามารถเขียนสมการของระบบ Pendulum ได้ดังต่อไปนี้

$$sJ_{sum}\omega(s) = k_t I(s) - \tau_{load}(s) - B\omega(s)$$

$$V(s) = R_m I(s) + k_e \omega(s) + sL I(s)$$

โดยกำหนดให้

$\theta(s)$ คือ องศาใน Domain S [degrees]

$V(s)$ คือ Voltage ใน S plane

$\omega(s)$ คือ ความเร็วในการหมุนใน S plane

J คือ ค่า Inertia ของมอเตอร์และ Pendulum รวมกัน [kgm^2]

L	คือ ค่า Inductance ของ Motor [H]
B	คือ ค่าความเสียดทานภายใน Motor
R_m	คือ ค่าความต้านทานของ Motor [Ω]
k_t	คือ ค่า Motor Constant [$\frac{Nm}{\sqrt{p}}$]
k_e	คือ ค่า Back-emf Constant [$\frac{V}{\omega}$]

จัดรูปสมการ $V(s)$ ได้ว่า

$$I(s) = \frac{V(s) - k_e \omega(s)}{sL + R_m}$$

แทนค่าลงในสมการ $sJ_{sum}\omega(s)$ ได้ว่า

$$\omega(s) \left((sJ_{sum} + B\omega(s))(sL + R_m) + k_e k_t \right) = k_t V(s) - \tau_{load}(s)(sL + R_m)$$

$$\omega(s) = \frac{k_t V(s) - \tau_{load}(s)(sL + R_m)}{((sJ_{sum} + B\omega)(sL + R_m) + k_e k_t)}$$

$$\theta(s) = \frac{k_t V(s) - \tau_{load}(s)(sL + R_m)}{s((sJ_{sum} + B\omega)(sL + R_m) + k_e k_t)}$$

$$\theta(s) = \frac{(k_m V(s))}{s((sJ_{sum} + B\omega)(sL + R_m) + k_e k_m))} - \frac{(\tau_{load}(s)(sL + R_m))}{s((sJ_{sum} + B\omega)(sL + R_m) + k_e k_m))}$$

จัดรูปสมการใหม่

$$\theta(s) = G_v(s)V(s) - G_\tau(s)\tau_{load}$$

หา Disturbance Feedforward กำหนดให้ $V(s) = 0$

$$\frac{V(s)}{\tau_{load}(s)} = \frac{G_\tau(s)}{G_v(s)}$$

แทนค่าและจัดรูปสมการ

$$G_{ff}(s) = \frac{sL + R_m}{k_t}$$

เนื่องจาก *transfer function* มี *Degree* ตัวเศษมากกว่าตัวส่วน สามารถแก้ไขได้ 2 วิธี คือ

Assume $L = 0$

$$G_{ff}(s) = \frac{R_m}{k_t}$$

ใส่ *Low Pass Filter* เข้ามาในระบบ

$$G_{ff}(s) = \left(\frac{1}{\tau s + 1} \right) \left(\frac{sL + R_m}{k_t} \right) = \frac{sL + R_m}{k_t \tau s + k_t}$$

โดยกำหนดให้

τ คือ ค่า *period* ของ *Cut-off Frequency* [sec]

การทดลองที่ 2.2 การทดลองเปรียบเทียบความแตกต่างของหา K_p Controller Gain ด้วยวิธีการต่างๆ

คำนวณ root locus Controller Gain

จาก Plant ของ System โดยกำหนดให้ Disturbance Feedforward จากการทดลองที่ 2.1 หักล้างกับ Gravity ทั้งหมดจะได้ Transfer Function คือ

$$G(s) = \frac{k_t}{s(JLs^2 + (JR_m + BL)s + (R_mB + k_e k_t))}$$

$$\text{จัดรูปใหม่ } G(s) = \frac{k_t}{s(As^2 + Bs + C)}$$

เมื่อ

$$A = JL$$

$$B = JR_m + BL$$

$$C = R_mB + k_e k_t$$

ซึ่งจะได้สมการ Characteristic transfer function คือ

$$s(As^2 + Bs + C) = 0$$

จาก Requirement ของการทดลองที่ 2 จะได้ว่า

Requirement	Value	Unit
Overshoot Percentage	≤ 10	%
Peak Time	≤ 3	seconds
Voltage Range	$[-12, 12]$	volts

ภาพที่ 99 Requirement ของการทดลองที่ 2

$$\zeta = \frac{3}{\text{peaktime}} = 0.5912$$

$$\omega_n = \frac{3}{\frac{B}{2\zeta}} = 1.0476 \frac{\text{rad}}{\text{sec}}$$

$$\omega_d = \omega_n(1 - \zeta^2) = 1.2978 \frac{\text{rad}}{\text{sec}}$$

$$\sigma = -\zeta\omega_n$$

เมื่อกำหนดให้

$$\omega_d \quad \text{คือ} \quad \text{damping ratio natural frequency} \left[\frac{\text{rad}}{\text{sec}} \right]$$

$$\omega_n \quad \text{คือ} \quad \text{natural frequency} \left[\frac{\text{rad}}{\text{sec}} \right]$$

$$\zeta \quad \text{คือ} \quad \text{damping ratio}$$

ซึ่งจะได้ Design point คือ $s = \sigma + j = -0.767 + 1.0467j$

นำค่า Design Point แทนลงใน Characteristic transfer function จะได้ว่า

$$|K_p| = 0.06235$$

ภาคผนวก ข

การทดลองที่ 3.1 การทดลองความแตกต่างของวิธีการ ออกแบบ Controller Design และการเลือกใช้ Controller แต่ละประเภท

การคำนวณ P_i ด้วย Root Locus

จาก Requirement ในการทดลองที่ 3

Requirement	Value	Unit
Overshoot Percentage	≤ 5	%
Settling Time	≤ 2.5	seconds
Absolute Steady State Error	$\leq 4 \times 10^{-3}$	rad

ภาพที่ 100 Requirement ของการทดลองที่ 3

จาก Characteristic Transfer Function

$$s(As^2 + Bs + C) = 0$$

จาก PI Controller

$$K_{controller} = K_p + \frac{K_i}{s} = K_p \left(\frac{s + \frac{K_i}{K_p}}{s} \right)$$

ซึ่งสามารถรวม Controller และ Plant ได้ดังนี้

$$L(s) = \frac{K_t K_p \left(s + \frac{K_i}{K_p} \right)}{s^2 (As^2 + Bs + C)}$$

แก้สมการ $As^2 + Bs + C = 0$ หาค่าตำแหน่ง Pole ของ System จะได้ว่า

$$s = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A}$$

$$s = -1.1, -1116.7$$

จาก Requirement สามารถกำหนดตัวแปรต่างๆได้ดังนี้

$$\zeta = 0.6901$$

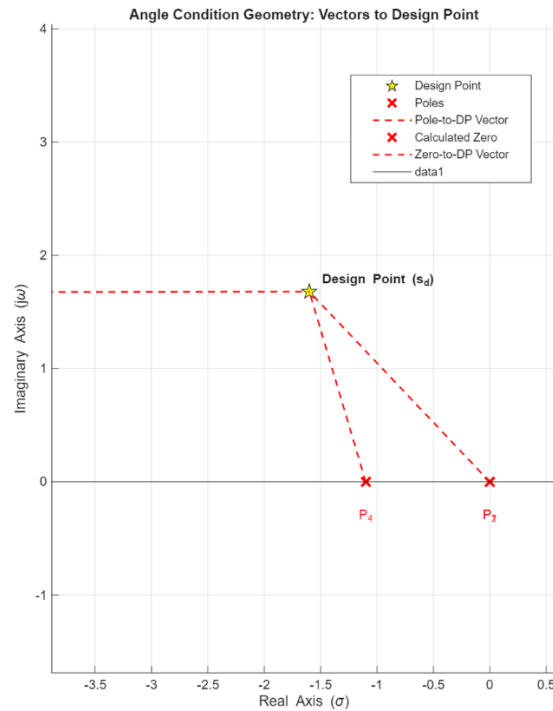
$$\omega_n = 2.3185 \frac{rad}{sec}$$

$$\omega_d = 1.6779 \frac{rad}{sec}$$

จากข้อกำหนดตาม requirement จะได้ Design Point ดังนี้

$$s = -1.6 + 1.678j$$

จากสมการสามารถ Sketch ตำแหน่งของ Pole ได้ดังนี้



ภาพที่ 101 ภาพแสดงตำแหน่ง Pole ของระบบ

จากสมการ

$$\frac{K_t K_p \left(s + \frac{K_i}{K_p} \right)}{s^2 (As^2 + Bs + C)} = -1$$

$$\sum \text{Pole angle} - \sum \text{Zero angle} = 180$$

จากการคำนวณพบว่ามุมของ Zero คือ 193.87 องศา ซึ่งหมายความว่า PI Control ไม่สามารถทำงานได้ตาม Requirement

Sum of Pole Angles: 373.87 degrees
Required Zero Angle (theta_z): 193.87 degrees

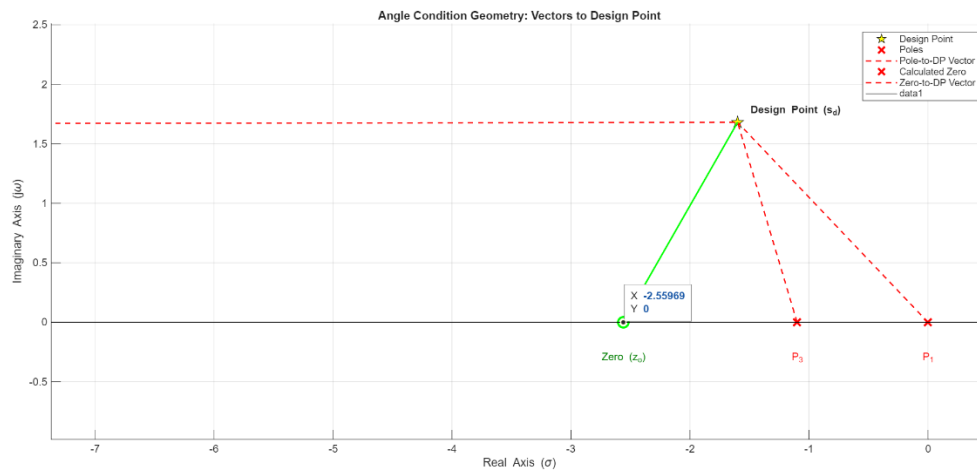
ภาพที่ 102 ผลการคำนวณมุม zero ที่ใช้เพื่อให้ได้ตำแหน่งที่ต้องการ

การคำนวณ PD ด้วย Root Locus

เนื่องจากเป็น Plant และ Requirement เดียวกันดังนั้น

Design Point คือ $s = -1.6 + 1.678j$

ใส่ตำแหน่งของ Pole ทั้งหมดลงใน โค้ดคำนวณตำแหน่ง Zeros



ภาพที่ 103 ภาพแสดง Sketch ของ Zero และ Pole

Calculated Zero Position: -2.5597

Magnitude of the transfer function at the Design Point: 0.0705

Sum of Pole Angles: 240.26 degrees

Required Zero Angle (theta_z): 60.26 degrees

ภาพที่ 104 ภาพแสดงผลการคำนวณตำแหน่ง zero ของ PD Control

ได้สมการ Controller คือ

$$K_p + sK_d = K_d \left(s + \frac{K_p}{K_d} \right) = K_d(s + 2.56)$$

ได้สมการ $L(s)$ คือ

$$\frac{K_d k_t (s + 2.56)}{s(As^2 + Bs + C)} = -1$$

$$K_d = -\frac{s(As^2 + Bs + C)}{k_t(s + 2.56)}$$

แทน Design Point $s = -1.6 + 1.678j$

$$|K_d| = 0.0705$$

$$|K_p| = 0.18048$$

การคำนวณ PID ด้วย Root Locus

จากสมการ PID Controller

$$G_c(s) = K_p + \frac{K_i}{s} + K_d s$$

$$G_c(s) = \frac{K_d s^2 + K_p s + K_i}{s}$$

$$G_c(s) = \frac{K_d(s + z_{pd})(s + z_{pi})}{s}$$

ต่อเนื่องจาก PD Control

$$z_{pd} = 2.56$$

และจากกฎ

$$\sum \text{Pole angle} - \sum \text{Zero angle} = 180$$

ได้ว่า Zero ของ I control ทำมุมกับ Design Point 37.86 องศาได้ตำแหน่ง Zero ได้สมการคือ

$$z_{pi} = 1.6 - \frac{1.68}{\tan(37.80)} = 0.0004016$$

ซึ่งจะได้สมการ Plant ดังนี้

$$G_s(s) = \frac{K_d(s + 2.56)(s + 0.0004016)}{s}$$

$$L(s) = \frac{K_d(s + z_{pd})(s + z_{pi})}{s^2(As^2 + Bs + C)} = -1$$

$$\frac{K_d(s + 2.56)(s + 0.0004016)}{s^2(As^2 + Bs + C)} = -1$$

แทน Design Point $s = -1.6 + 1.678j$

$$|K_d| = 0.18$$

$$G_s(s) = \frac{0.18(s + 2.56)(s + 0.0004016)}{s}$$

$$G_s(s) = \frac{0.18s^2 + 0.4608s + 0.000185}{s}$$

เทียบสัมประสิทธิ์ได้ว่า

$$K_d = 0.18$$

$$K_p = 0.4608$$

$$K_i = 0.000185$$

ภาคผนวก ค

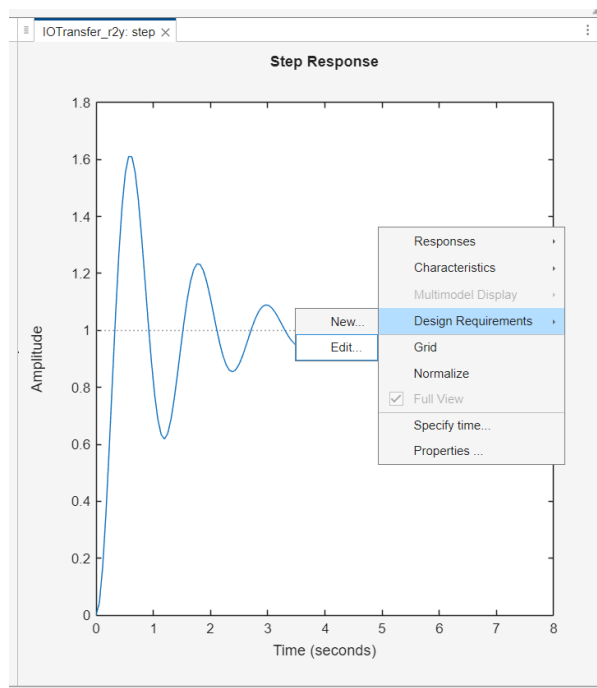
การ Tune PID ด้วย Control System Design Toolbox

ขั้นตอนที่ 1 กำหนด Plant ของระบบและ Run Program

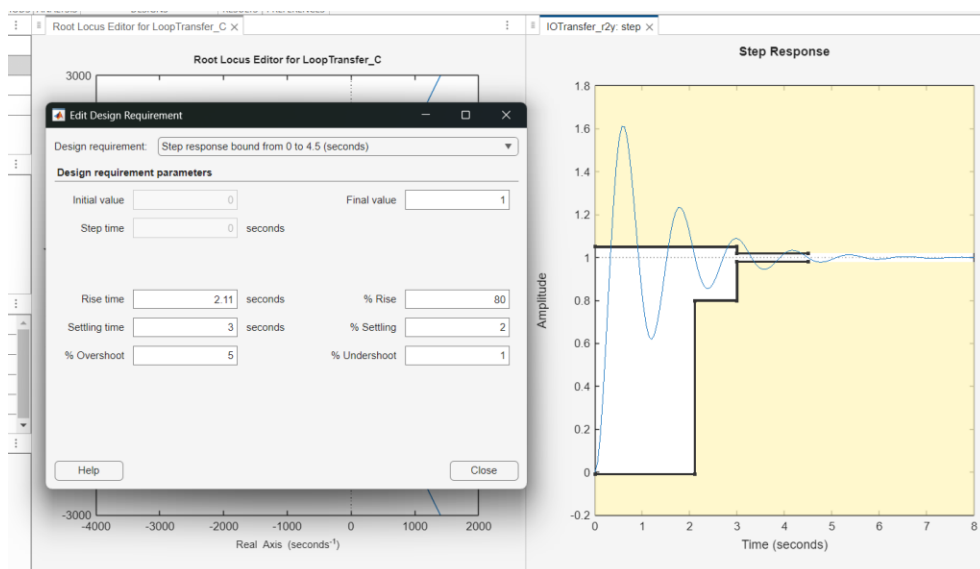
```
num = kt;
den = [ (J_sum*Lm), (J_sum*R + b*Lm), (b*R + kt*ke), 0];
sys_tf = tf(num, den);
controlSystemDesigner("rlocus", sys_tf);
```

ภาพที่ 105 ภาพแสดงคำสั่งสำหรับการเปิด Control System Design ในกรอบสีแดงคือ Function Plant

ขั้นตอนที่ 2 ใส่ Requirement ลงในกราฟ

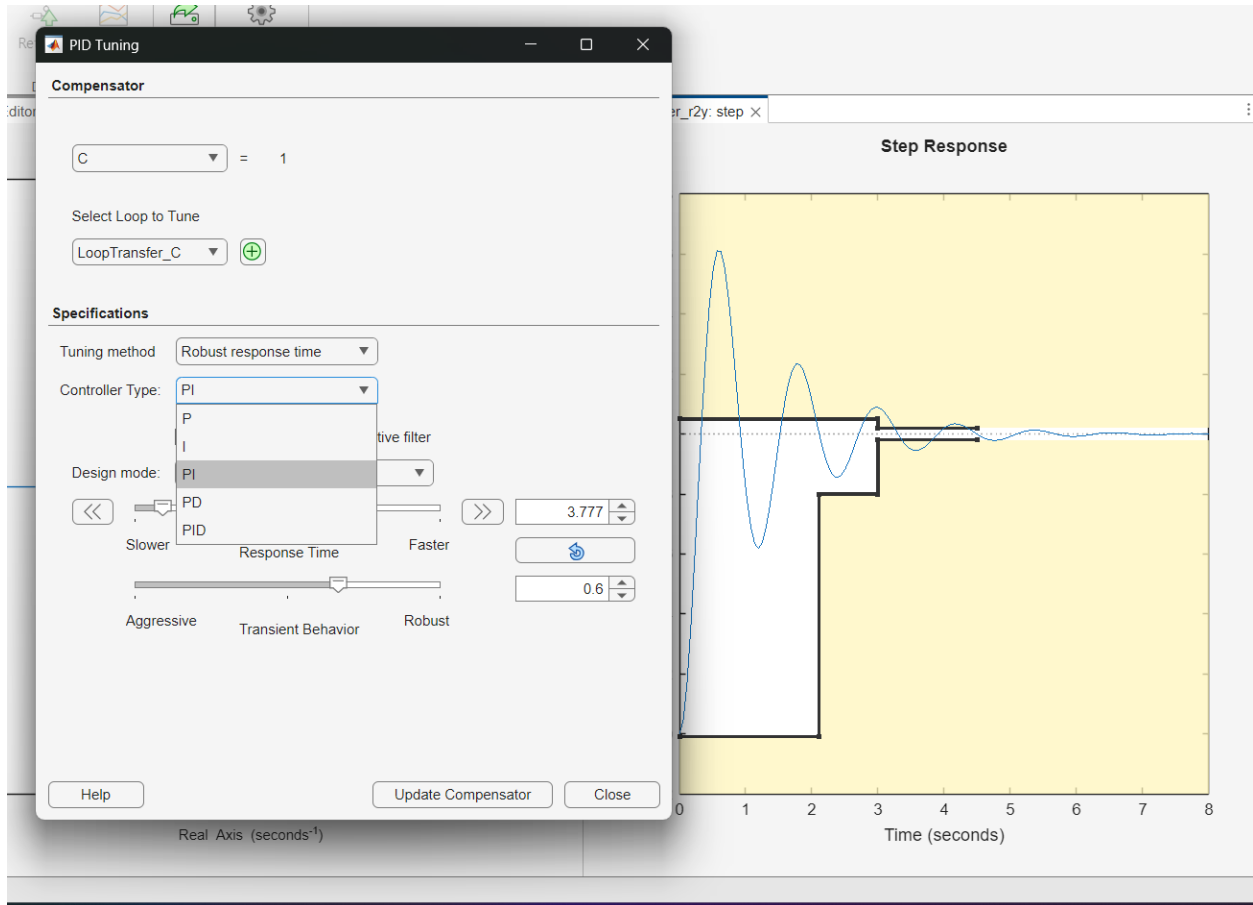


ภาพที่ 106



ภาพที่ 107

ขั้นตอนที่ 3 เลือก Controller ต้องการ Tune

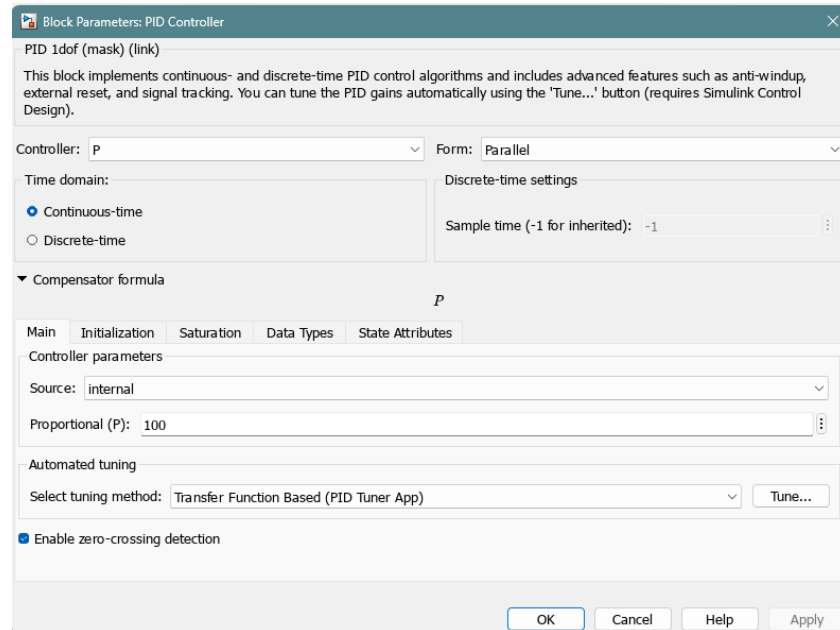


ภาพที่ 108

ขั้นตอนที่ 4 ปรับสไลด์ bar และกดอัปเดตจนกว่า กราฟจะลู่เข้า Requirement ทั้งหมด

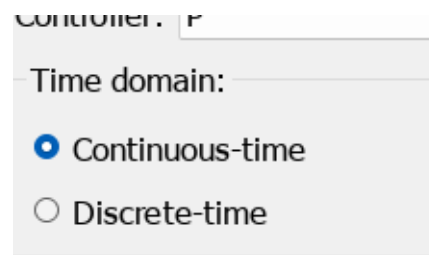
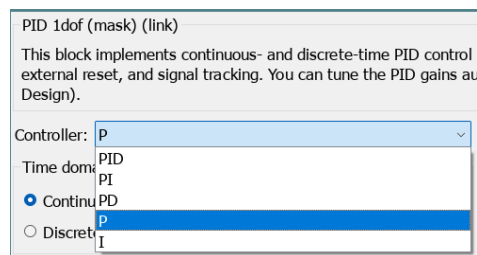
การ Tune PID ด้วย PID Controller

ขั้นตอนที่ 1 กดเข้าบล็อก PID Controller



ภาพที่ 109 แสดงหน้าต่าง Block PID Controller

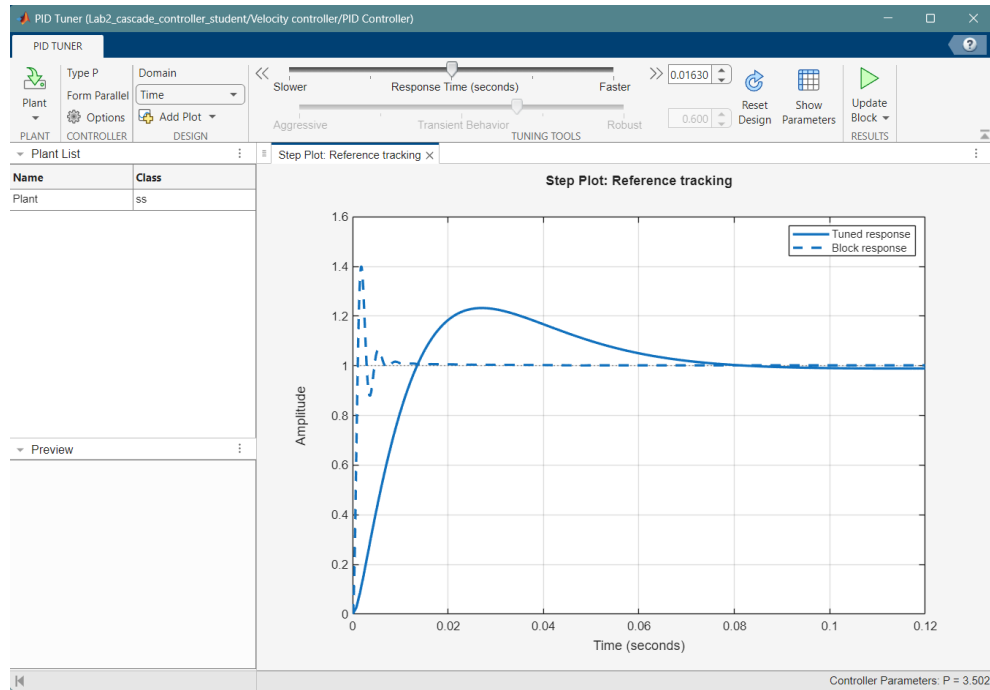
ขั้นตอนที่ 2 เลือก Controller ที่ต้องการจะใช้ และ Time domain



ภาพที่ 110 และ 111 แสดงการเลือก Controller และ Time Domain

ขั้นตอนที่ 3 กด Apply ที่มุมขวาล่าง หลังจากนั้นกด Tune...

ขั้นตอนที่ 4 ปรับ Slide bar ให้ได้ค่า Parameters ตรงตาม Requirement โดยดู Parameters ได้จากการกด Show Parameters



ภาพที่ 112 แสดงหน้าต่าง PID Tuner

The 'Show Parameters' dialog box displays the following data:

Controller Parameters		
	Tuned	Block
P	3.5021	100
I	n/a	n/a
D	n/a	n/a
N	n/a	n/a

Performance and Robustness		
	Tuned	Block
Rise time	0.00975 seconds	0.000703 seconds
Settling time	0.07 seconds	0.0063 seconds
Overshoot	23.3 %	40 %
Peak	1.23	1.4
Gain margin	-23.1 dB @ 19.3 rad/s	-52.2 dB @ 19.3 rad/s
Phase margin	60 deg @ 123 rad/s	32.8 deg @ 1.69e+03 rad/s
Closed-loop stability	Stable	Stable

ภาพที่ 113 แสดงหน้าต่าง Show Parameters

ขั้นตอนที่ 5 กด Update Block จากนั้นค่าที่ Tune มาได้จะไปใส่ที่บล็อกให้เลย และสามารถใช้บล็อก PID Controller ได้เลย